

INFORMATIQUE • TLE TI

MODULE 3 : SYSTÈMES D'INFORMATION ET BASES DE DONNÉES

LEÇON N13

MODÉLISATION DYNAMIQUE DES SYSTÈMES D'INFORMATION AVEC UML (CAS D'UTILISATION ET SÉQUENCE)

DURÉE CONSEILLÉE : 18 H

DANS CETTE LEÇON

Cette leçon introduit la modélisation dynamique des systèmes d'information avec UML : le diagramme de cas d'utilisation, qui décrit les services offerts aux acteurs, et le diagramme de séquence, qui décrit l'ordre chronologique des échanges entre objets. L'élève apprendra à identifier acteurs et scénarios, à utiliser les relations d'inclusion, d'extension et de généralisation, puis à produire ces diagrammes avec un logiciel professionnel.

- À la fin de cette leçon, je sais identifier les acteurs et délimiter le périmètre d'un système d'information
- À la fin de cette leçon, je sais décrire les scénarios (nominal et alternatifs) d'un cas d'utilisation
- À la fin de cette leçon, je sais représenter un diagramme de cas d'utilisation complet dans le formalisme UML
- À la fin de cette leçon, je sais représenter les relations d'inclusion («include»), d'extension («extend») et de généralisation/spécialisation entre cas d'utilisation
- À la fin de cette leçon, je sais construire un diagramme de séquence avec lignes de vie, objets et messages
- À la fin de cette leçon, je sais distinguer et utiliser les types de messages (synchrone, asynchrone, retour, création)



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|--|--------------------------------------|
| 1. Acteurs, périmètre et cas d'utilisation | 6. Synthèse et méthodologie d'examen |
| 2. Relations entre cas d'utilisation | 7. Activité d'intégration |
| 3. Le diagramme de séquence : composants | 8. Méthodes et exercices résolus |
| 4. Types de messages et scénarios complexes | 9. Exercices |
| 5. Production avec un logiciel professionnel | 10. Corrigés des exercices |
| | 11. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais identifier les acteurs et délimiter le périmètre d'un système d'information
- À la fin de cette leçon, je sais décrire les scénarios (nominal et alternatifs) d'un cas d'utilisation
- À la fin de cette leçon, je sais représenter un diagramme de cas d'utilisation complet dans le formalisme UML
- À la fin de cette leçon, je sais représenter les relations d'inclusion («include»), d'extension («extend») et de généralisation/spécialisation entre cas d'utilisation
- À la fin de cette leçon, je sais construire un diagramme de séquence avec lignes de vie, objets et messages
- À la fin de cette leçon, je sais distinguer et utiliser les types de messages (synchrone, asynchrone, retour, création)
- À la fin de cette leçon, je sais produire un diagramme de cas d'utilisation et un diagramme de séquence à l'aide d'un logiciel professionnel (ArgoUML, StarUML, WinDesign)

PRÉREQUIS

- Notion de système d'information et de base de données (Module 3, début d'année)
- Modélisation statique avec UML : classes, objets, attributs (leçons précédentes de Tle)

- Notion d'acteur et de processus métier vue en classe de Première (SI et organisation)
- Analyse fonctionnelle simple : identifier les besoins d'un utilisateur
- Utilisation de base d'un logiciel de dessin ou de bureautique

1 ACTEURS, PÉRIMÈTRE ET CAS D'UTILISATION

1.1 Situation d'accroche et problématique

À la MTN Mobile Money Agency de Mokolo, en plein cœur de Yaoundé, les files d'attente s'allongent dès 8 heures du matin. Observe bien ce qui se passe : le client dicte son numéro, l'agent saisit le montant sur son téléphone, le système central de l'opérateur vérifie le solde, envoie un code de confirmation, et parfois la banque partenaire est sollicitée pour valider l'opération. Tout ce petit monde échange des informations dans un **ordre précis** : une seule erreur de séquence — un code saisi trop tard, un solde vérifié après le débit au lieu d'avant — et le retrait entier est bloqué, voire l'argent est perdu.

Le directeur régional veut informatiser et fiabiliser ce service. Mais avant d'écrire la moindre ligne de code PHP ou SQL, une question fondamentale se pose : comment décrire clairement **QUI fait QUOI** et **DANS QUEL ORDRE** ? Comment représenter cela de manière normalisée, compréhensible à la fois par les informaticiens, les gestionnaires et les décideurs ?

C'est exactement le rôle des **diagrammes de cas d'utilisation** et des **diagrammes de séquence** d'UML, que nous allons étudier dans cette leçon. Commençons par les fondations : les acteurs, le périmètre du système et les cas d'utilisation.

1.2 Rappel : UML, un langage de modélisation normalisé

DÉFINITION UML

UML (Unified Modeling Language, langage de modélisation unifié) est un langage graphique de modélisation normalisé qui permet de représenter un système informatique sous forme de diagrammes, avant et pendant sa réalisation. Ce n'est pas un langage de programmation : on n'exécute pas un diagramme UML, on s'en sert pour réfléchir, communiquer et concevoir.

Pourquoi un tel langage ? Parce qu'un système d'information réel (Mobile Money, gestion des notes au lycée, plateforme `impots.cm`) est trop complexe pour être décrit en une seule conversation. UML fournit un vocabulaire graphique commun : un diagramme dessiné à Yaoundé sera compris de la même façon par un développeur de Douala ou de Nairobi.

On distingue deux grandes familles de modélisation :

Famille	Question posée	Exemples de diagrammes
Modélisation statique (structurelle)	De quoi le système est-il fait? Quelles données, quelles classes?	Diagramme de classes, diagramme d'objets, diagramme de composants
Modélisation dynamique (comportementale)	Que fait le système? Qui interagit avec lui, dans quel ordre?	Diagramme de cas d'utilisation, diagramme de séquence, diagramme d'activité

Dans cette leçon, nous nous concentrons sur la modélisation **dynamique** : le diagramme de cas d'utilisation (la vue « qui fait quoi ») et le diagramme de séquence (la vue « dans quel ordre »).

LESAVAIS-TU ? UML, une norme internationale

UML a été normalisé par l'**OMG** (Object Management Group) en 1997, à partir des travaux de Grady Booch, Ivar Jacobson et James Rumbaugh. La version UML 2.x compte 14 types de diagrammes, répartis en 7 diagrammes structurels (classes, objets, composants, déploiement, paquetages, structure composite, profils) et 7 diagrammes comportementaux (cas d'utilisation, séquence, activité, états-transitions, communication, vue d'ensemble des interactions, timing). Au Baccalauréat TI, seuls quelques-uns sont exigibles : retiens surtout le diagramme de cas d'utilisation, le diagramme de séquence et le diagramme de classes.

1.3 Les acteurs : qui interagit avec le système ?

Revenons à notre agence de Mokolo. Qui participe à un retrait ? Le client, bien sûr. L'agent de la boutique. Mais aussi le serveur central de MTN, et parfois la banque partenaire. Remarque une chose essentielle : aucun de ces participants n'est « à l'intérieur » du système informatique que nous concevons — ils sont tous à l'extérieur et dialoguent avec lui.

DÉFINITION Acteur

Un **acteur** est une entité externe au système — personne, groupe de personnes, ou autre système informatique — qui interagit directement avec le système : elle lui fournit des informations ou en reçoit. Un acteur représente un rôle joué vis-à-vis du système, et non une personne précise.

En UML, un acteur se représente par un **bonhomme** (un « stickman ») portant son nom de rôle en dessous : Client, Agent, Système bancaire.

On distingue deux catégories d'acteurs :

- L'**acteur principal** : c'est lui qui **déclenche** le cas d'utilisation pour obtenir un résultat qui lui est utile. Par convention, on le place à **gauche** du système. Exemple : le Client qui demande un retrait.
- L'**acteur secondaire** : il est **sollicité par le système** au cours du déroulement (il fournit un service au système). Par convention, on le place à **droite**. Exemple : le Système bancaire que le serveur Mobile Money interroge pour valider une opération.

ATTENTION Un acteur est un RÔLE, pas une personne

Erreur classique au Baccalauréat : écrire « M. Ngo Bassa » ou « la guichetière Aïcha » comme acteur. Faux ! Un acteur est un rôle : Client, Agent, Guichetier. M. Ngo Bassa peut être Client le matin (il retire de l'argent) et Agent l'après-midi (il tient sa propre boutique) : c'est la même personne, mais deux rôles différents. Autre piège : oublier les acteurs non humains. Un serveur central, une application bancaire, un système de notification SMS sont des acteurs à part entière dès lors qu'ils échangent avec ton système.

Critère	Acteur principal	Acteur secondaire
Qui déclenche ?	L'acteur déclenche le cas d'utilisation	Le système le sollicite pendant l'exécution
Bénéfice	Il reçoit le résultat utile du service	Il rend un service au système
Position conventionnelle	À gauche du rectangle système	À droite du rectangle système
Exemples camerounais	Le client qui retire de l'argent ; l'élève qui consulte ses notes en ligne ; le contribuable qui déclare sur <code>impots.cm</code>	Le serveur central MTN MoMo ; le système bancaire partenaire (Afriland, SGC) ; le serveur SMS qui envoie la confirmation

1.4 Le périmètre du système : la frontière entre le dedans et le dehors

Avant de dessiner quoi que ce soit, il faut répondre à une question de bon sens : qu'est-ce qui relève de mon système, et qu'est-ce qui est en dehors ? C'est la question du **périmètre**.

DÉFINITION Périmètre du système

Le **périmètre du système** (ou frontière du système) délimite ce que le système fait (l'intérieur : les fonctionnalités à développer) de ce qui est externe (les acteurs, les autres systèmes). En UML, il se représente par un rectangle portant le nom du système en haut ; les cas d'utilisation sont placés à l'intérieur du rectangle, les acteurs à l'extérieur.

Cette frontière est une décision de conception, pas une évidence. Dans notre exemple, le serveur central de MTN est-il à l'intérieur ou à l'extérieur du système « application de l'agence » ?

Si nous concevons l'application de l'agent, le serveur central est externe : c'est un acteur secondaire. Si nous concevons toute la plateforme Mobile Money nationale, il passe à l'intérieur. Le périmètre dépend donc du point de vue adopté — et il doit être fixé avant de lister les cas d'utilisation.

1.5 Le cas d'utilisation : un service rendu à un acteur

DÉFINITION Cas d'utilisation

Un **cas d'utilisation** (use case) est un ensemble d'actions réalisées par le système qui produit un résultat observable et utile pour un acteur. Il se représente par une ellipse contenant un nom verbal (un verbe à l'infinitif + complément) : Retirer de l'argent, Consulter le solde, S'authentifier.

Trois conditions pour qu'une ellipse soit un bon cas d'utilisation :

1. c'est le **système** qui réalise les actions (pas l'acteur) ;
2. le résultat est **observable** par l'acteur (un reçu, un solde affiché, un virement effectué) ;
3. le résultat a une **valeur métier** : il répond à un besoin réel de l'acteur.

La liaison entre un acteur et un cas d'utilisation s'appelle une **association** : elle se dessine par une **ligne pleine simple** (sans flèche dans la notation de base) entre le bonhomme et l'ellipse. Elle signifie : « cet acteur participe à ce cas d'utilisation, il dialogue avec le système pendant son exécution ». Le sens de lecture est : l'acteur — réalise / participe à — le cas d'utilisation.

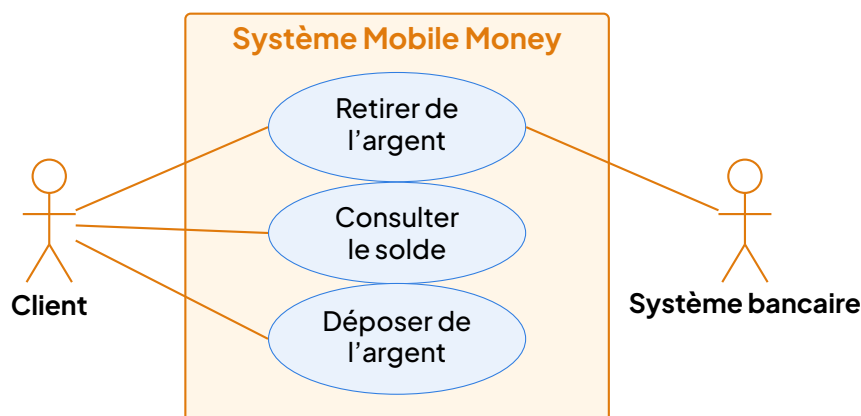


Diagramme de cas d'utilisation simplifié du système Mobile Money : le Client (acteur principal, à gauche) déclenche les cas ; le Système bancaire (acteur secondaire, à droite) est sollicité lors du retrait.

MÉTHODE Identifier les acteurs et les cas d'utilisation à partir d'un cahier des charges

Face à un texte de cahier des charges (souvent un paragraphe au Baccalauréat), procède en quatre étapes :

1. Qui utilise le système ? Souligne toutes les entités qui dialoguent avec lui (personnes et systèmes externes) : ce sont les candidats acteurs. Regroupe-les ensuite par rôle (plusieurs personnes peuvent jouer le même rôle).
2. Quel service chaque acteur attend-il ? Pour chaque acteur, liste ce qu'il demande au système : chaque service devient un cas d'utilisation, nommé par un verbe à l'infinitif.
3. Quel résultat observable ? Vérifie que chaque cas produit un résultat utile et visible pour l'acteur. Sinon, ce n'est pas un cas d'utilisation autonome (c'est peut-être une simple étape d'un autre cas).
4. Trace la frontière. Dessine le rectangle système, place les cas dedans, les acteurs dehors (principaux à gauche, secondaires à droite), puis relie par des associations.

1.6 Le scénario d'un cas d'utilisation : raconter le déroulement

Une ellipse et un nom ne suffisent pas : « Retirer de l'argent » peut se passer de mille façons. Pour décrire précisément le comportement, on rédige le **scénario** du cas d'utilisation.

DÉFINITION Scénario d'un cas d'utilisation

Un **scénario** est une séquence ordonnée d'interactions entre les acteurs et le système, décrivant un déroulement particulier du cas d'utilisation. On distingue :

- le **scénario nominal** : le déroulement normal, où tout se passe bien ;
- les **scénarios alternatifs** : des variantes prévues (ex. le client choisit un reçu papier ou SMS) ;
- les **scénarios d'exception** : les cas d'échec (solde insuffisant, code PIN erroné, réseau indisponible).

La description textuelle d'un cas précise aussi les **préconditions** (ce qui doit être vrai avant : « le client possède un compte actif ») et les **postconditions** (ce qui est vrai après : « le compte est débité, le client a reçu les espèces »).

EXEMPLE Scénario nominal du cas « Retirer de l'argent »

Acteur principal : Client. Acteur secondaire : Système bancaire.

Précondition : le client possède un compte Mobile Money actif et approvisionné.

Postcondition : le compte est débité du montant, le client a reçu les espèces et un SMS de confirmation.

1. Le client demande un retrait et communique son numéro de téléphone à l'agent.

2. L'agent saisit le numéro et le montant dans l'application ; le système affiche le nom du titulaire pour vérification.
3. Le système envoie une notification au téléphone du client, qui saisit son code PIN pour valider.
4. Le système vérifie le code PIN et le solde du compte.
5. Le système débite le compte, crédite le compte de l'agent et confirme l'opération au Système bancaire.
6. Le système envoie un SMS de confirmation au client ; l'agent remet les espèces et le reçu.

Scénario d'exception (étape 4) : si le solde est insuffisant, le système affiche « Solde insuffisant », aucune somme n'est débitée, et le cas se termine en échec. Si le code PIN est erroné trois fois, le compte est temporairement bloqué.

EXERCICE RÉSOLU 1**Construire le diagramme de cas d'utilisation d'une bibliothèque scolaire**

Énoncé. Le lycée de Bafoussam veut informatiser sa bibliothèque. Le document de cadrage précise : « Les élèves empruntent et rendent des ouvrages auprès du bibliothécaire, qui enregistre chaque opération dans le futur logiciel. Les élèves peuvent aussi consulter le catalogue en ligne. Le proviseur consulte les statistiques d'emprunt. Chaque soir, le logiciel envoie automatiquement un rappel par SMS aux élèves en retard, via la plateforme SMS de l'opérateur. » Identifier les acteurs et les cas d'utilisation, puis décrire le périmètre.

SOLUTION

Solution détaillée.

Étape 1 — Qui utilise le système ? On relève : les élèves, le bibliothécaire, le proviseur, et la plateforme SMS de l'opérateur (acteur non humain). Regroupés par rôle : Élève, Bibliothécaire, Proviseur, Plateforme SMS.

Étape 2 — Quels services ?

- Élève : Consulter le catalogue (en ligne, sans intermédiaire).
- Bibliothécaire : Enregistrer un emprunt, Enregistrer un retour.
- Proviseur : Consulter les statistiques d'emprunt.
- Système (déclenché par le temps) : Envoyer un rappel de retard, qui sollicite la Plateforme SMS.

Étape 3 — Résultats observables ? Oui pour chacun : catalogue affiché, emprunt enregistré avec date de retour, statistiques affichées, SMS effectivement envoyé.

Étape 4 — Frontière et placement. Le rectangle « Gestion de la bibliothèque » contient

les cinq ellipses. Acteurs principaux à gauche : Élève, Bibliothécaire, Proviseur. Acteur secondaire à droite : Plateforme SMS (sollicitée par le cas Envoyer un rappel de retard). Remarque : l'élève n'interagit pas directement avec les cas d'emprunt/retour, car c'est le bibliothécaire qui saisit dans le logiciel — l'élève n'est donc pas associé à ces ellipses. C'est une subtilité fréquemment sanctionnée à l'examen.

À RETENIR L'essentiel de la section

- UML est un langage de modélisation normalisé (OMG, 1997) ; on distingue la modélisation statique (structure) de la modélisation dynamique (comportement).
- Un acteur est un rôle externe (humain ou système) qui interagit avec le système : principal (il déclenche, à gauche) ou secondaire (il est sollicité, à droite).
- Le périmètre est le rectangle qui sépare l'intérieur (cas d'utilisation) de l'extérieur (acteurs).
- Un cas d'utilisation est un ensemble d'actions du système produisant un résultat observable et utile ; il se note par une ellipse au nom verbal, reliée aux acteurs par des associations (lignes pleines).
- Chaque cas se détaille par des scénarios : nominal, alternatifs, exceptions, encadrés par des préconditions et postconditions.

EXERCICE 1 À toi de jouer

La startup camerounaise KamerPay veut lancer une application de paiement de factures d'électricité (ENEO) par mobile. Le client saisit son numéro de compteur, consulte le montant dû, paie via son compte mobile ; l'application interroge le serveur ENEO pour connaître la dette et confirmer le paiement, puis envoie un reçu par SMS. (1) Identifie les acteurs en précisant principal/secondaire. (2) Liste les cas d'utilisation avec leurs noms verbaux. (3) Rédige le scénario nominal du cas « Payer une facture » en 5 étapes, avec précondition et postcondition. (4) Propose un scénario d'exception.

CORRIGÉ Exercice : KamerPay

(1) Acteur principal : Client (il déclenche le paiement). Acteurs secondaires : Serveur ENEO (fournit la dette, reçoit la confirmation) et éventuellement Plateforme SMS (envoie le reçu). (2) Cas d'utilisation : Consulter le montant dû, Payer une facture, Recevoir un reçu (ou Envoyer le reçu côté système). (3) Scénario nominal de Payer une facture — précondition : le client possède un compte mobile approvisionné ; 1. le client saisit son numéro de

compteur ; 2. le système interroge le serveur ENEO et affiche le montant dû ; 3. le client confirme et saisit son code PIN ; 4. le système débite le compte mobile et notifie ENEO du règlement ; 5. le système envoie le reçu par SMS. Postcondition : la facture est soldée chez ENEO et le client dispose d'un reçu. (4) Exception : si le serveur ENEO ne répond pas (étape 2), le système affiche « Service indisponible, réessayez plus tard » et aucun débit n'est effectué.

2 RELATIONS ENTRE CAS D'UTILISATION

Après avoir identifié les acteurs et dressé la liste des cas d'utilisation qui délimitent le périmètre de notre système Mobile Money, nous faisons face à un écueil classique : la redondance. Plusieurs cas d'utilisation partagent des comportements communs (l'authentification, la journalisation, la notification), d'autres ne sont que des variantes les unes des autres (paiement par carte vs paiement en espèces), et certains n'apparaissent que sous conditions particulières (impression du reçu, envoi de SMS). Sans mécanisme de structuration, le diagramme devient illisible, difficile à maintenir et source d'incohérences. C'est précisément le rôle des **relations entre cas d'utilisation** : factoriser le commun, expliciter les variantes et conditionner les extensions optionnelles. Le langage UML en définit trois principales : «**include**» (inclusion), «**extend**» (extension) et la **généralisation/spécialisation**. Maîtriser leur notation — surtout le sens des flèches — est un passage obligé pour réussir l'épreuve pratique du Baccalauréat.

2.1 Pourquoi relier les cas d'utilisation : factoriser, clarifier, éviter les redondances

Imaginons le système MiniPay, une application de transfert d'argent inspirée des services MTN Mobile Money ou Orange Money au Cameroun. Les cas d'utilisation «Retirer de l'argent», «Déposer de l'argent» et «Transférer de l'argent» exigent tous que le client soit identifié. Si nous dupliquons la séquence d'authentification (saisie du PIN, vérification auprès du serveur USSD, gestion des erreurs) dans chacun de ces trois cas, nous créons trois fois plus de travail d'analyse, trois fois plus de risques d'incohérence (un changement de règle de PIN oblige à modifier trois endroits) et un diagramme brouillon.

DÉFINITION Relations structurelles entre cas d'utilisation

Les relations entre cas d'utilisation permettent de structurer le modèle dynamique en :

- Factorisant les comportements communs et obligatoires («**include**»).
- Conditionnant des comportements optionnels ou exceptionnels («**extend**»).
- Organisant des variantes d'un même service par héritage (**généralisation/spécialisation**).

Elles ne modifient pas la sémantique globale du système : elles ne font qu'organiser sa description pour la rendre maintenable, réutilisable et lisible.

ATTENTION Piège conceptuel : ne pas confondre structure et séquence

Une relation «**include**» ou «**extend**» n'est pas un appel de fonction ni un ordre temporel. Elle exprime une dépendance statique entre descriptions de comportements. L'ordre d'exécution (qui se passe avant qui) se modélise dans le diagramme de séquence (Section 3), pas dans le diagramme de cas d'utilisation.

2.2 Relation d'inclusion <<include>> : le obligatoire et systématique

DÉFINITION Relation d'inclusion <<include>>

La relation d'inclusion indique qu'un cas de base (le cas inclusif) intègre obligatoirement et systématiquement le comportement d'un cas inclus (le cas factorisé). À chaque exécution du cas de base, le cas inclus s'exécute exactement une fois (sauf boucles internes au cas inclus).

Notation UML : une flèche pointillée orientée du cas de base vers le cas inclus, étiquetée <<include>>.

PROPRIÉTÉ Règles de l'inclusion

- Obligation : le cas de base ne peut pas se réaliser complètement sans le cas inclus.
- Systématicité : l'inclusion a lieu à chaque occurrence du cas de base, sans condition.
- Réutilisation : un même cas inclus peut être inclus par plusieurs cas de base (factorisation mutualisée).
- Indépendance : le cas inclus n'a pas connaissance du cas de base qui l'inclut (faible couplage).

EXEMPLE Exemple Mobile Money : Authentification factorisée

Dans MiniPay, les cas «Retirer », «Déposer »et «Transférer »incluent tous le cas «S'authentifier ».

- «Retirer »<<include>> «S'authentifier »
- «Déposer »<<include>> «S'authentifier »
- «Transférer »<<include>> «S'authentifier »

Le cas «S'authentifier »est factorisé une seule fois. Si la banque centrale (BEAC) impose une double authentification (PIN + OTP SMS), on ne modifie que «S'authentifier ».

MÉTHODE Comment identifier un «include» légitime? (Test du « Sans cela, ça ne marche pas »)

1. Isoler un fragment de comportement commun à plusieurs cas d'utilisation.
2. Se demander : «Le cas de base peut-il atteindre son objectif final sans exécuter ce fragment? »
3. Si la réponse est NON (le fragment est indispensable, obligatoire, systématique) → c'est un <<include>>.
4. Si la réponse est OUI (le fragment est optionnel, conditionnel, ou représente une étape de décomposition) → ce n'est pas un <<include>> (voir <<extend>> ou décomposition en sous-cas).

ATTENTION Erreur majeure au Bac : Inversion de la flèche «include»

C'est l'erreur numéro 1 en modélisation UML au Baccalauréat camerounais.

- Faux : Flèche du cas inclus vers le cas de base. (Cela signifierait « S'authentifier inclut Retirer », absurde).
- Vrai : Flèche du cas de base vers le cas inclus. Mnémotechnique : «Le gros (cas de base) pointe vers le petit (cas factorisé réutilisable) ». Ou : «Include = I go Inside » (la flèche entre dans le cas inclus).

2.3 Relation d'extension <<extend>> : l'optionnel et conditionnel

DÉFINITION Relation d'extension <<extend>>

La relation d'extension indique qu'un cas d'extension (le cas étendant) ajoute un comportement optionnel à un cas de base (le cas étendu) sous une condition précise (point d'extension). Le cas de base est complet et fonctionnel sans l'extension.

Notation UML : une flèche pointillée orientée du cas d'extension vers le cas de base, étiquetée <<extend>>. On peut annoter la condition près de la flèche (ex. : [si reçu demandé]).

PROPRIÉTÉ Règles de l'extension

- Optionnalité : le cas de base fonctionne parfaitement sans l'extension.
- Conditionnalité : l'extension ne se déclenche que si une condition est remplie (choix

utilisateur, règle métier, erreur, heure de la journée...).

- Point d'extension : le cas de base déclare explicitement où l'extension peut s'insérer (ex. : «après validation du montant »).
- Non-intrusivité : le cas de base ne connaît pas l'existence de ses extensions (ouvert/-fermé).

EXEMPLE Exemple Mobile Money : Impression du reçu et Alerte solde faible

- «Imprimer reçu »<<extend>> «Retirer de l'argent » [si client appuie sur « Imprimer »]
- «Alerter solde faible »<<extend>> «Retirer de l'argent » [si solde < 1000 FCFA après retrait]
- «Envoyer notification SMS »<<extend>> «Transférer » [si bénéficiaire a activé les notifications]

Le retrait d'argent est un service complet sans impression ni alerte. Ces ajouts sont des services à valeur ajoutée conditionnels.

MÉTHODE Choisir entre «include» et «extend» : l'arbre de décision

1. Le comportement additionnel est-il indispensable pour que le cas de base atteigne son objectif principal?
2. OUI → <<include>> (ex. : payer pour commander, s'authentifier pour consulter).
3. NON → Est-il déclenché par une condition explicite (choix, règle, erreur) après que le cas de base a commencé?
4. OUI → <<extend>> (ex. : imprimer reçu, envoyer SMS, appliquer promo).
5. NON (c'est juste une étape séquentielle décrite par l'analyste) → Pas de relation. Décrivez l'enchaînement dans le scénario textuel ou le diagramme de séquence.

ATTENTION Piège fréquent : «include» pour décomposer en étapes (Anti-pattern)

EXEMPLE Mauvaise modélisation : Décomposition fonctionnelle

Un élève modélise «Acheter en ligne »avec : «Choisir articles »<<include>> «Acheter », «Payer »<<include>> «Acheter », «Valider commande »<<include>> «Acheter ».

Pourquoi c'est faux : <<include>> n'est pas un outil de décomposition algorithmique (étape 1, étape 2, étape 3). Ces « sous-cas » n'ont aucun sens métier autonomes (on ne « Choisir articles » seul comme service final). Cela pollue le diagramme et casse la réutilisabilité. Solution : Un seul cas «Acheter en ligne »avec un scénario textuel détaillé (ou diagramme de séquence).

2.4 Généralisation / Spécialisation entre cas d'utilisation : les variantes d'un service**DÉFINITION** Généralisation / Spécialisation entre cas d'utilisation

La généralisation organise une relation parent/enfant entre un cas d'utilisation général (parent, abstrait) et un ou plusieurs cas d'utilisation spécialisés (enfants, concrets). L'enfant hérite du comportement, des acteurs, des relations (include/extend) du parent, et y ajoute ou redéfinit des spécificités.

Notation UML : une flèche pleine à triangle vide (pointe fermée) orientée du cas spécialisé (enfant) vers le cas général (parent). Identique à l'héritage de classes.

PROPRIÉTÉ Sémantique de l'héritage de cas d'utilisation

- Factorisation de la structure : Acteurs, pré/post-conditions, inclusions/extensions communes sont définies au niveau parent.
- Spécialisation du comportement : L'enfant précise comment le service est réalisé (algorithme, technologie, règles).
- Substitution de Liskov : Partout où le système attend le cas général, un cas spécialisé peut s'exécuter sans casser la cohérence.

EXEMPLE Exemple Mobile Money : Hiérarchie de paiement

- Parent (abstrait) : «Effectuer un paiement »(inclut «S'authentifier », étendu par «Notifier »).

- Enfant 1 : « Paiement par Mobile Money » (spécifie : USSD, PIN, solde compte).
- Enfant 2 : « Paiement par Carte Bancaire » (spécifie : Numéro carte, CVV, 3D-Secure, passerelle VISA/Mastercard).
- Enfant 3 : « Paiement en Espèces (Agent) » (spécifie : Code agent, remise physique, reçu papier).

L'acteur « Client » est associé au parent « Effectuer un paiement ». Par héritage, il accède aux trois spécialisations.

2.5 Généralisation entre acteurs : profils et habilitations

La généralisation s'applique aussi aux **acteurs**. Elle modélise une relation « est-un » (héritage de rôle).

DÉFINITION Généralisation d'acteurs

Un acteur spécialisé (enfant) hérite de toutes les associations (participation aux cas d'utilisation) de l'acteur général (parent), et peut en avoir des supplémentaires.

Notation UML : Flèche pleine à triangle vide de l'acteur spécialisé vers l'acteur général.

EXEMPLE Exemple Mobile Money : Hiérarchie d'acteurs

- Acteur général : « Utilisateur » (associé à : « Consulter solde », « Changer PIN »).
- Spécialisation 1 : « Client Particulier » (hérite de Utilisateur + « Transférer », « Payer Marchand »).
- Spécialisation 2 : « Agent » (hérite de Utilisateur + « Encaisser Dépôt », « Servir Retrait », « Gérer Float »).
- Spécialisation 3 : « Administrateur Système » (hérite de Utilisateur + « Bloquer Compte », « Auditer Transactions »).

Cela évite de redessiner les flèches vers « Consulter solde » pour chaque type d'utilisateur.

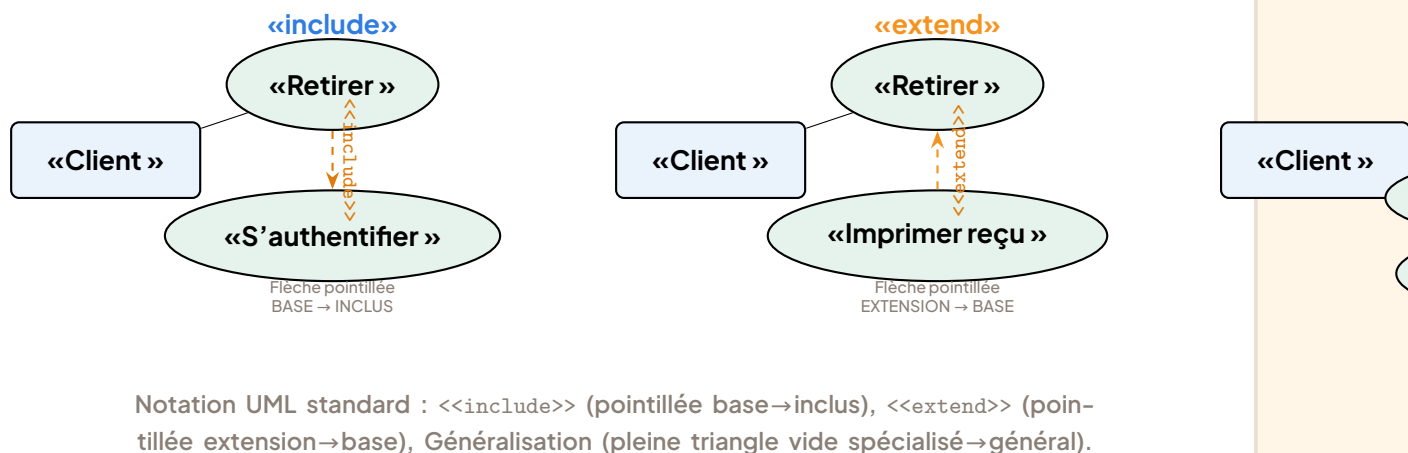
LESAVAIS-TU ? Le piège de la « Décomposition fonctionnelle » déguisée en « include »

Historiquement, les méthodes pré-UML (Merise, SADT) encourageaient la décomposition hiérarchique des fonctions (FO → F1, F2, F3). Les premiers utilisateurs d'UML ont transposé ce réflexe en abusant de <<include>> pour « casser » un cas d'utilisation en étapes. La

norme UML 2.x et les guides de bonnes pratiques (Alistair Cockburn, Writing Effective Use Cases) insistent : un cas d'utilisation = un objectif utilisateur observable et à valeur métier. Si « Saisir le montant » n'est pas un objectif utilisateur final (personne ne se connecte juste pour saisir un montant), ce n'est pas un cas d'utilisation, donc pas de <<include>>. Cette erreur reste la plus répandue dans les copies du Bac et les premiers projets professionnels.

2.6 Synthèse visuelle : Notation comparative des trois relations

Le diagramme ci-dessous résume la notation exacte (forme de flèche, trait, orientation) pour les trois relations. C'est une **fiche de révision** à connaître par cœur pour l'examen pratique.



2.7 Diagramme complet Mobile Money : intégration des trois relations

Voici le diagramme de cas d'utilisation consolidé pour le système MiniPay. Il intègre acteurs spécialisés, inclusions obligatoires, extensions conditionnelles et généralisation de paiement. C'est un modèle **type Bac** : complet, lisible, sans redondance.

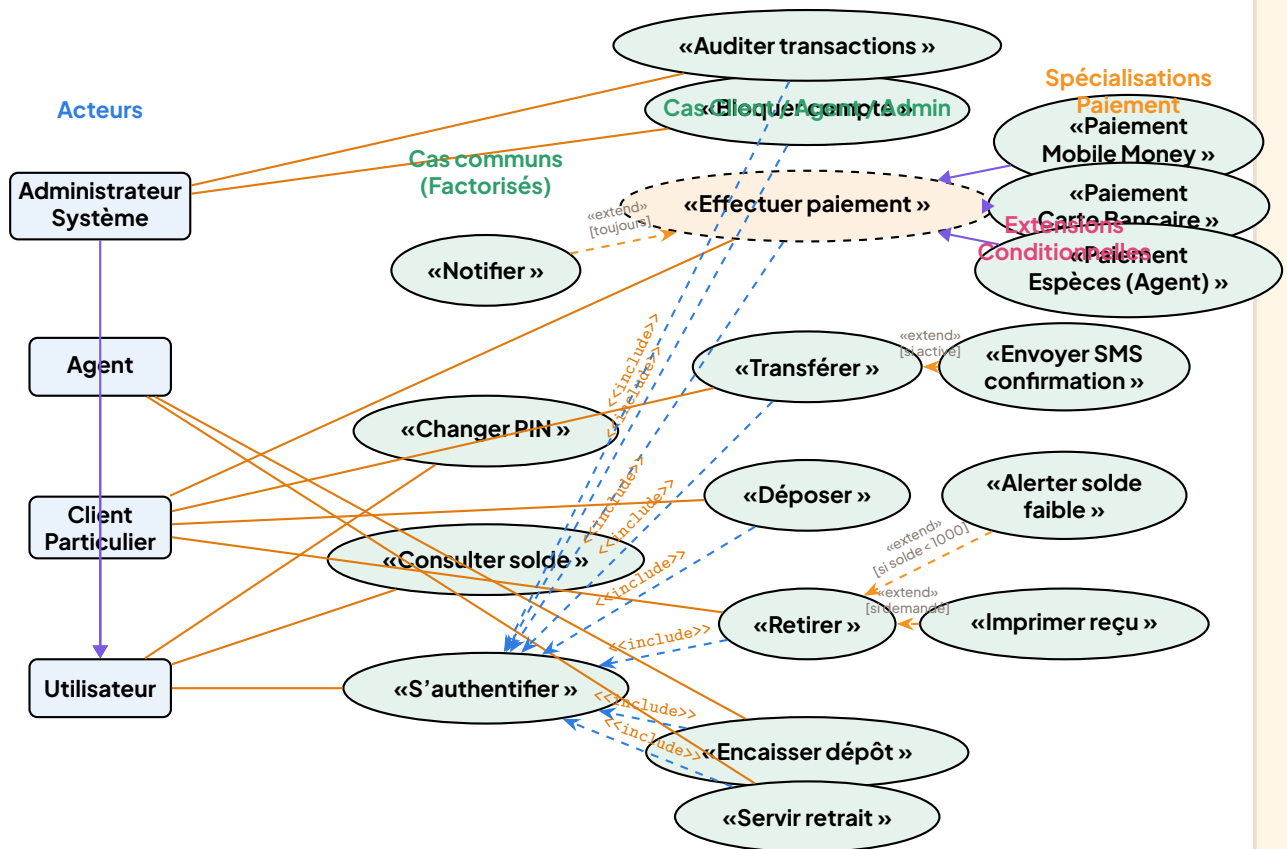


Diagramme de cas d'utilisation complet MiniPay intégrant : généralisation d'acteurs (haut gauche), inclusion obligatoire <<include>> vers «S'authentifier» (flèches bleues pointillées), extension conditionnelle <<extend>> (flèches orange pointillées), généralisation de cas d'utilisation «Paieement» (flèches violettes pleines).

EXERCICE RÉSOLU 2 Correction d'un diagramme type Bac : Inversion des flèches

Énoncé : Un candidat propose le diagramme suivant pour le système «Gestion de bibliothèque» :

- Acteur : «Adhérent »
- Cas : «Emprunter livre », «Réserver livre », «S'identifier »
- Relations tracées :
 1. Flèche pointillée de «S'identifier »vers «Emprunter livre »étiquetée <<include>>.
 2. Flèche pointillée de «Emprunter livre »vers «Imprimer reçu »étiquetée <<extend>>.
 3. Flèche pleine triangle vide de «Paieement »vers «Paieement Carte ».

Identifiez et corrigez les trois erreurs de notation.

SOLUTION

Solution détaillée :

1. Erreur 1 (Include inversée) : La flèche `<<include>>` part de «S'identifier »(inclus) vers «Emprunter »(base). Correction : Inverser la flèche. «Emprunter livre »`<<include>>` «S'identifier ». Justification : Emprunter ne peut se faire sans s'identifier. La base pointe vers l'inclus.
2. Erreur 2 (Extend inversée) : La flèche `<<extend>>` part de «Emprunter »(base) vers «Imprimer reçu »(extension). Correction : Inverser la flèche. «Imprimer reçu »`<<extend>>` «Emprunter livre ». Justification : L'extension (optionnelle) pointe vers la base qu'elle enrichit. Condition : [si adhérent le demande].
3. Erreur 3 (Généralisation inversée) : La flèche pleine triangle vide part du parent «Paie-ment »vers l'enfant «Paie-ment Carte ». Correction : Inverser la flèche. «Paie-ment Carte »-|> «Paie-ment ». Justification : En UML, la flèche d'héritage (généralisation) pointe toujours de l'enfant (spécialisé) vers le parent (général), pour les classes et pour les cas d'utilisation/acteurs.

Règle mnémotechnique unique : «La flèche pointe vers ce qui est réutilisé ou général ».

- Include : Base → Inclus (réutilisé).
- Extend : Extension → Base (général/enrichi).
- Generalization : Spécialisé → Général.

EXPÉRIENCE TP guidé : Modéliser les relations sur StarUML / ArgoUML

Objectif : Produire le diagramme MiniPay ci-dessus avec un outil professionnel. Logiciel : StarUML (recommandé) ou ArgoUML / WinDesign. Durée : 30 min.

Manipulation pas à pas :

1. Créer le projet : File → New → UML Model. Nommer `MiniPay_UseCase`.
2. Acteurs : Palette «Use Case Diagram »→ outil `Actor`. Créer «Utilisateur », «Client Particulier », «Agent », «Administrateur ».
3. Généralisation Acteurs : Outil `Generalization` (flèche pleine triangle). Cliquer «Client Particulier »→ glisser vers «Utilisateur ». Répéter pour Agent et Admin. Vérifier : triangle vide pointe vers Utilisateur.
4. Cas d'utilisation : Outil `Use Case`. Créer tous les cas du diagramme (auth, consulter, retirer, déposer, transférer, paiement, paiementMM, paiementCarte, paiementEspèces, imprimer, alerter, SMS, encaisser, servir, bloquer, auditer).
5. Propriété Abstract : Sélectionner «Effectuer paiement »→ Properties → cocher `isAbstract = true`. Il devient italique/pointillé.

6. Associations Acteur-Cas : Outil *Association* (trait plein). Relier acteurs aux cas (héritage propage les liens visuellement sur certains outils, sinon relier explicitement).
7. Inclusions : Outil *Include* (flèche pointillée). Ordre crucial : Cliquer d'abord sur la BASE (ex. «Retirer»), ensuite sur l'INCLUS (ex. «S'authentifier»). Répéter pour tous. Vérifier étiquette <<include>>.
8. Extensions : Outil *Extend* (flèche pointillée). Ordre crucial : Cliquer d'abord sur l'EXTENSION (ex. «Imprimer reçu »), ensuite sur la BASE (ex. «Retirer »). Ajouter contrainte : double-clic flèche → constraint → [si demandé].
9. Généralisation Cas : Outil *Generalization*. Cliquer SPÉCIALISÉ (→ «Paieement Mobile ») vers GÉNÉRAL (→ «Effectuer paieement »).
10. Mise en page : Aligner, espacer, colorier (Format → Fill Color) pour distinguer : Acteurs (bleu), Cas concrets (vert), Cas abstrait (orange), Extensions (rose).
11. Export : File → Export Diagram → PNG/PDF pour le rapport.

Observations attendues :

- L'outil refuse de créer une inclusion si on inverse l'ordre (StarUML) ou la crée inversée (ArgoUML ancien) → Vigilance absolue sur le sens de clic.
- Le cas abstrait «Effectuer paieement » n'a pas d'acteur direct (hérité).
- L'extension «Notifier » sur le parent «Paieement » s'applique automatiquement aux trois enfants.

Interprétation : Ce diagramme est maintenant une spécification contractuelle valide pour les développeurs (backend API, frontend USSD/App) et les testeurs (cas de test : retrait avec/sans impression, paiement carte vs espèces).

À RETENIR Check-list de validation avant de rendre un diagramme de cas d'utilisation

1. Acteurs : Tous nommés (rôle métier), pas de « Système » comme acteur principal. Généralisations factorisées ?
2. Cas d'utilisation : Noms verbe + objet (objectif utilisateur). Pas de « Gérer X », « Faire Y ». Cas abstraits en italique.
3. «include» : Flèche pointillée BASE → INCLUS. Condition : «Obligatoire & Systématique ». Pas de décomposition en étapes.
4. «extend» : Flèche pointillée EXTENSION → BASE. Condition explicite notée sur la flèche. Base fonctionnelle seule.
5. Généralisation : Flèche PLEINE triangle vide ENFANT → PARENT (acteurs ET cas). Héritage des associations vérifié.
6. Lisibilité : Pas de flèches qui se croisent inutilement. Légende si couleurs personnalisées.

sées. Titre du diagramme + version + auteur + date.

EXERCICE 2 Entraînement Bac : Système de Gestion de Notes «NotePro »

Modéliser le diagramme de cas d'utilisation pour une application web utilisée dans un lycée camerounais. Acteurs : «Élève », «Professeur », «Administrateur », «Parent ». Exigences :

1. Tout utilisateur doit «Se connecter »(authentification) pour accéder à ses services.
2. «Professeur » : «Saisir notes », «Modifier note »(si délai non dépassé), «Publier bulletin ».
3. «Élève » : «Consulter notes », «Télécharger bulletin ».
4. «Parent » : «Consulter notes enfant », «Recevoir alerte SMS »(si note < 10/20).
5. «Administrateur » : «Gérer utilisateurs », «Configurer seuils alerte ».
6. «Publier bulletin »déclenche optionnellement «Générer PDF »et «Envoyer notification ».
7. «Saisir notes »et «Modifier note »sont des variantes de «Éditer note ».

Consignes : Dessiner le diagramme (papier ou outil). Identifier explicitement : 2 généralisations acteurs, 1 généralisation cas, 3 inclusions, 2 extensions. Justifier chaque choix.

CORRIGÉ Exercice NotePro

Acteurs & Généralisation :

- «Utilisateur »(parent abstrait) ← «Élève », «Professeur », «Parent », «Administrateur ». (4 flèches pleines triangle vide vers Utilisateur).
- «Utilisateur »– «Se connecter »(association unique factorisée).

Cas d'utilisation & Généralisation :

- «Éditer note »(abstrait) ← «Saisir notes », «Modifier note ». (2 flèches pleines triangle vide vers Éditer note).
- «Professeur »– «Saisir notes », «Modifier note », «Publier bulletin ».

Inclusions («include») :

- «Saisir notes »<<include>> «Se connecter »(hérité via Utilisateur mais explicite si Professeur seul).
- «Modifier note »<<include>> «Se connecter ».
- «Publier bulletin »<<include>> «Se connecter ».

- «Consulter notes »<<include>> «Se connecter »(Élève, Parent).
- «Gérer utilisateurs »<<include>> «Se connecter »(Admin).

Note : Souvent on relie «Utilisateur »à «Se connecter »et l'héritage suffit. Mais au Bac, on préfère souvent expliciter sur les cas concrets pour montrer la maîtrise de <<include>>.

Extensions («extend») :

- «Générer PDF »<<extend>> «Publier bulletin » [si format PDF demandé].
- «Envoyer notification »<<extend>> «Publier bulletin » [si notification activée].
- «Recevoir alerte SMS »<<extend>> «Consulter notes enfant » [si note < 10].

Vérification sens flèches :

- Include : Base (Saisir, Publier, Consulter) → Inclus (Se connecter). ≡
- Extend : Extension (Générer PDF, Alerte SMS) → Base (Publier, Consulter). ≡
- Generalization : Spécialisé (Saisir, Modifier, Élève, Prof) → Général (Éditer, Utilisateur). ≡

3

LE DIAGRAMME DE SÉQUENCE : COMPOSANTS

Dans la section précédente, nous avons appris à structurer un diagramme de cas d'utilisation grâce aux relations d'inclusion, d'extension et de généralisation. Ce diagramme répond à la question : que peut faire le système, et pour qui ? Mais il reste muet sur une question tout aussi essentielle pour le développeur : comment les objets du système collaborent-ils, et dans quel ordre, pour réaliser concrètement un cas d'utilisation ? C'est précisément le rôle du **diagramme de séquence**, le diagramme dynamique le plus utilisé d'UML.

3.1 Rôle du diagramme de séquence

DÉFINITION Diagramme de séquence

Le **diagramme de séquence** est un diagramme comportemental UML qui représente les **interactions** entre les objets (et l'acteur) d'un système dans le temps, pour un scénario donné d'un cas d'utilisation. Il montre qui envoie un message à qui, et dans quel ordre chronologique.

Reprenons notre exemple fil conducteur : le cas d'utilisation « Consulter le solde » d'une application Mobile Money. Le diagramme de cas d'utilisation nous dit que le **Client** peut consulter son solde. Mais pour programmer cette fonctionnalité, il faut savoir que le client saisit une demande sur l'**Interface**, que celle-ci la transmet à un **Contrôleur**, que celui-ci interroge la **Base de données**, puis que le résultat remonte jusqu'à l'écran. Cette chorégraphie ordonnée, c'est exactement ce que décrit le diagramme de séquence.

PROPRIÉTÉ Complémentarité avec le diagramme de cas d'utilisation

Le diagramme de cas d'utilisation offre une vue statique et fonctionnelle (les services offerts) ; le diagramme de séquence offre une vue dynamique et temporelle (le déroulement d'un service). En pratique, on réalise généralement un diagramme de séquence par scénario important de chaque cas d'utilisation significatif.

LESAVAIS-TU ? Un diagramme = un scénario, pas tout le système

Une erreur de débutant consiste à vouloir faire tenir tout le système dans un seul diagramme de séquence. En réalité, un diagramme de séquence correspond généralement à UN scénario d'UN cas d'utilisation. Pour le cas « Retirer de l'argent », on dessinera par exemple un diagramme pour le scénario nominal (retrait réussi) et un autre pour le scénario alternatif « code PIN incorrect ». Les grands éditeurs de logiciels camerounais et les intégrateurs qui développent des passerelles vers les API de MTN Mobile Money ou d'Orange Money travaillent exactement ainsi : chaque flux de transaction est documenté par son propre diagramme de séquence avant d'être codé.

3.2 L'objet (ou instance) et l'acteur

DÉFINITION Objet / instance

Dans un diagramme de séquence, un **objet** (ou **instance**) est représenté par un rectangle placé en haut du diagramme, contenant son nom sous la forme `nomObjet:NomClasse`, le tout souligné. Le soulignement est la convention UML qui distingue une instance d'une classe.

Trois écritures sont acceptées selon le besoin :

- `maBase:BaseDeDonnees` : objet nommé et classé (la plus complète) ;
- `:BaseDeDonnees` : objet anonyme, seule la classe compte ;
- `maBase` : objet nommé dont la classe est évidente dans le contexte.

L'**acteur** qui déclenche le scénario (par exemple le Client) figure dans le diagramme : il est dessiné avec son symbole habituel (le « bonhomme »), placé tout à gauche, et il possède lui aussi une ligne de vie. Son nom d'instance (souligné) est écrit à côté du symbole.

ATTENTION Ne pas confondre classe et objet

Écrire `BaseDeDonnees` sans deux-points ni soulignement désigne une classe, pas une ins-

tance : c'est une faute de notation en diagramme de séquence. Retiens la règle : souligné = instance. Au baccalauréat, un correcteur vérifie systématiquement ce détail.

3.3 La ligne de vie et l'axe du temps

DÉFINITION Ligne de vie

La **ligne de vie** est une ligne verticale en pointillés tracée sous chaque objet (ou acteur). Elle représente l'existence de l'objet pendant le scénario. Le temps s'écoule de haut en bas : un message placé plus bas sur le diagramme se produit après un message placé plus haut.

C'est la grande intuition à retenir : l'axe horizontal répartit les participants (qui?), l'axe vertical représente le TEMPS (quand?). Lire un diagramme de séquence, c'est donc le lire **de haut en bas**, comme une partition de musique.

DÉFINITION Barre d'activation

La **barre d'activation** (ou période d'activité) est un rectangle fin et vertical superposé à la ligne de vie. Elle indique la période pendant laquelle l'objet est actif, c'est-à-dire en train d'exécuter une opération (il calcule, il interroge, il attend une réponse). Elle commence à la réception d'un message déclencheur et se termine à la fin du traitement, souvent marquée par l'envoi du message de retour.

3.4 Les messages

DÉFINITION Message

Un **message** est une flèche horizontale partant de la ligne de vie de l'émetteur et aboutissant à la ligne de vie du destinataire. Il est étiqueté par le nom de l'opération invoquée, suivi éventuellement de ses paramètres entre parenthèses : `verifierSolde(numeroCompte)`. Les messages sont souvent numérotés (1, 2, 3...) dans l'ordre chronologique pour faciliter la lecture.

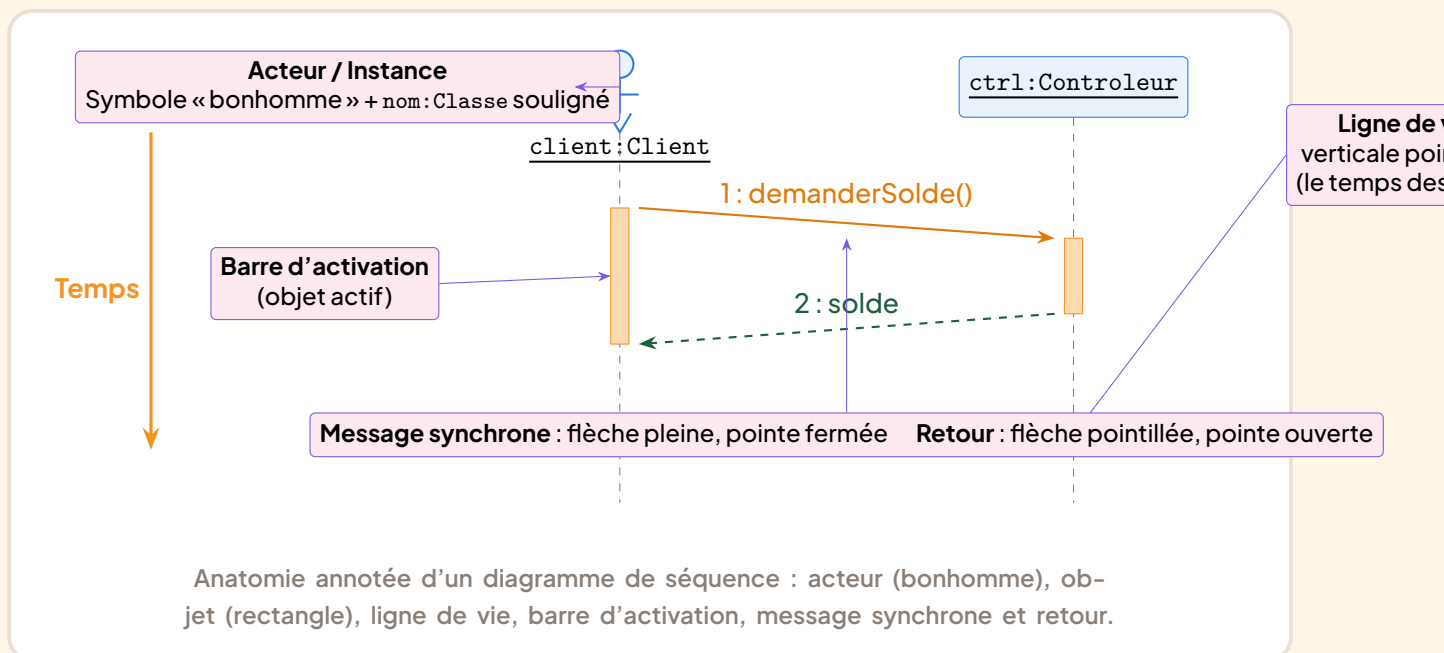
On distingue à ce stade deux formes essentielles (les types seront approfondis dans la section suivante) :

- le **message synchrone** (appel d'opération) : flèche pleine à pointe fermée ; l'émetteur se bloque en attendant la réponse ;
- le **message de retour** : flèche en pointillés à pointe ouverte, dans le sens inverse ; il ramène

le résultat (par exemple `solde = 25 000 FCFA`).

PROPRIÉTÉ Création et destruction d'objets

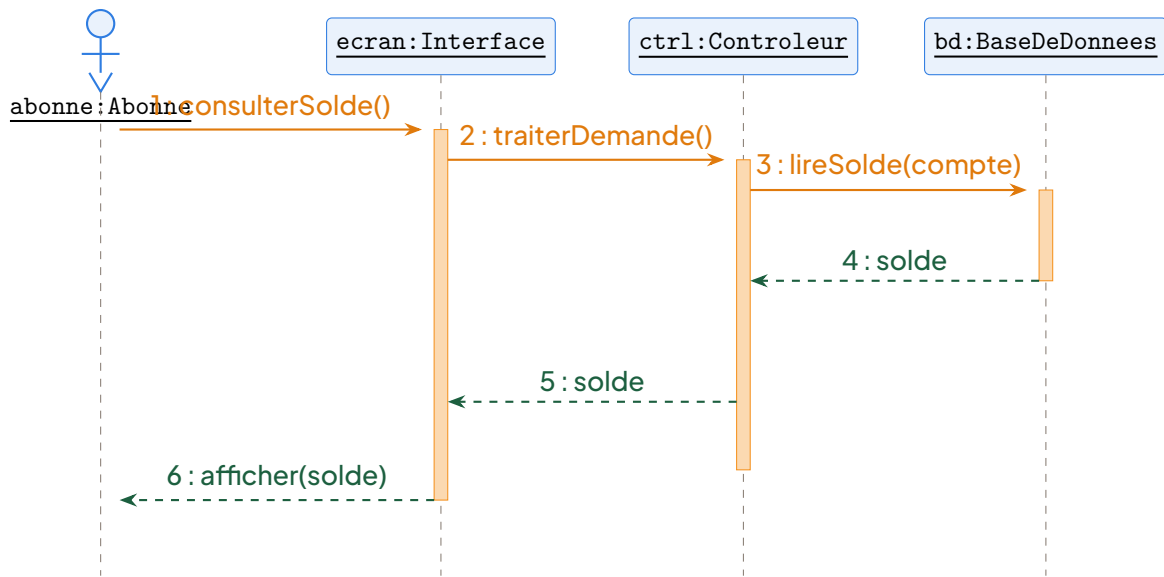
Un **message de création** est une flèche (souvent stéréotypée `<<create>>`) qui aboutit directement sur le rectangle de l'objet créé, placé plus bas que les autres : l'objet naît à cet instant du scénario. La **destruction** d'un objet est marquée par une croix X au bout de sa ligne de vie, à l'endroit où il cesse d'exister.



3.5 Lecture chronologique et exemple guidé

Lire un diagramme de séquence se fait toujours **de haut en bas**, en suivant les numéros des messages. Appliquons cela au scénario « Consulter le solde » :

1. le **Client** saisit sa demande sur l'**Interface** ;
2. l'**Interface** transmet la demande au **Contrôleur** ;
3. le **Contrôleur** interroge la **Base de données** ;
4. la **Base de données** retourne le solde au **Contrôleur** ;
5. le **Contrôleur** renvoie le résultat à l'**Interface** ;
6. l'**Interface** affiche le solde au **Client**.

**MÉTHODE** Construire un diagramme de séquence en 5 étapes

1. Identifier les objets : à partir du scénario du cas d'utilisation, liste l'acteur déclencheur et chaque objet qui participe (interface, contrôleur, base de données...).
2. Ordonner les participants : place-les en haut, de gauche à droite, l'acteur tout à gauche, en regroupant les objets qui échangent beaucoup pour limiter les croisements de flèches.
3. Tracer les lignes de vie : une verticale pointillée sous chaque participant.
4. Tracer les messages dans l'ordre chronologique : du haut vers le bas, numérotés, chacun étiqueté par `operation(parametres)` ; ajoute les barres d'activation pendant les traitements.
5. Ajouter les retours : flèches pointillées à pointe ouverte ramenant les résultats, puis vérifie la cohérence (tout message appelant a-t-il son retour ? l'ordre est-il respecté ?).

ATTENTION Erreur fréquente : ignorer l'axe du temps

L'erreur la plus classique au baccalauréat est d'oublier que l'axe vertical représente le TEMPS et de placer les messages dans le désordre : par exemple dessiner le retour de la

base de données au-dessus de la requête qui la demande. Deux messages qui se croisent verticalement dans le mauvais ordre rendent le scénario impossible. Vérifie toujours : la numérotation 1, 2, 3... doit descendre strictement le long du diagramme. Autre piège : faire partir un message d'une ligne de vie hors de toute barre d'activation alors que l'objet est censé être inactif.

EXEMPLE Retrait Mobile Money avec authentification

Voici le déroulement du scénario nominal « Retirer de l'argent » chez un opérateur de mobile money, qui servira de base à l'exercice résolu ci-dessous :

1. l'Abonné saisit `retirer(montant)` sur le menu USSD;
2. l'Interface demande au Contrôleur de `traiterRetrait(montant)`;
3. le Contrôleur demande le code PIN : `saisirPIN()` ; l'Abonné le fournit;
4. le Contrôleur envoie `verifierPIN(compte, pin)` à la `BaseDeDonnees`, qui retourne `pinValide`;
5. le Contrôleur envoie `debiter(compte, montant)`; la base retourne `confirmation`;
6. le Contrôleur notifie l'Interface, qui affiche « Retrait de 10 000 FCFA effectué » à l'Abonné.

Remarque : chaque vérification (PIN, solde suffisant) donne lieu à un aller-retour Contrôleur ↔ `BaseDeDonnees`, placé plus bas que le précédent.

EXERCICE RÉSOLU 3 Construire le diagramme de séquence du retrait Mobile Money

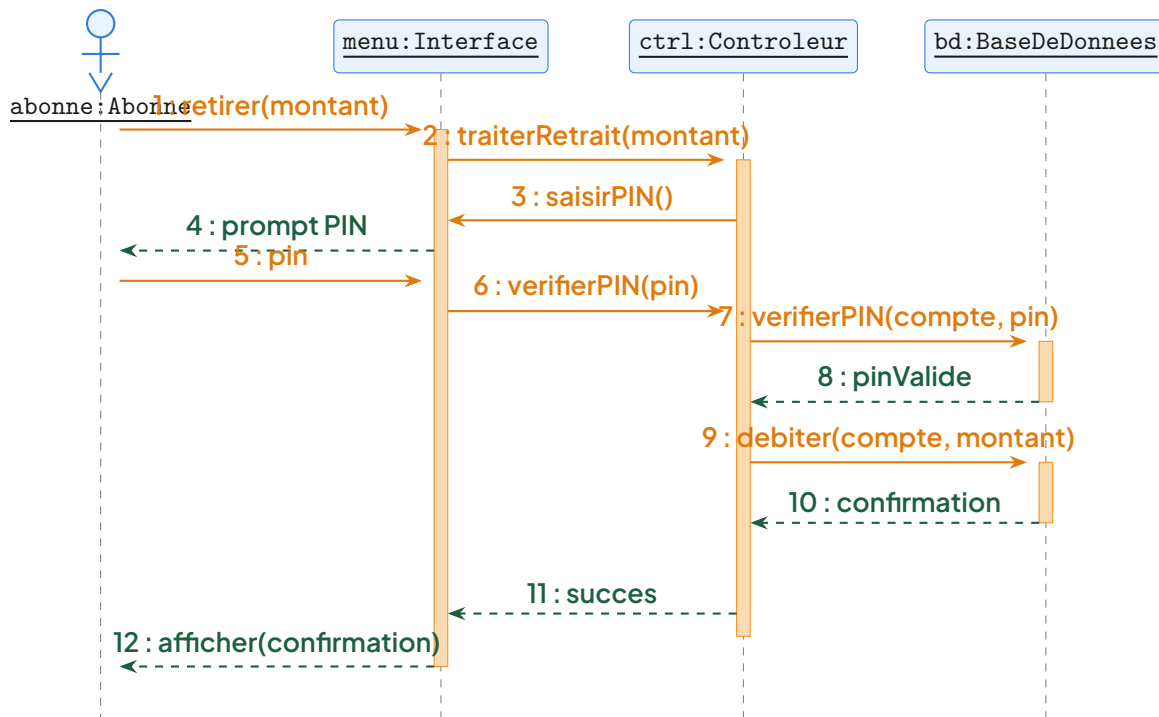
Énoncé. Une startup de Douala développe une passerelle de retrait Mobile Money. Le scénario nominal du cas d'utilisation « Retirer de l'argent » est : l'abonné demande un retrait, saisit son code PIN, le système vérifie le PIN puis le solde en base, débite le compte et confirme. Construis le diagramme de séquence en appliquant la méthode en 5 étapes.

SOLUTION

Étape 1 — Objets : l'acteur `abonne:Abonne` et trois instances : `menu:Interface`, `ctrl:Controleur`, `bd:BaseDeDonnees`.

Étape 2 — Ordre : Abonné (à gauche, bonhomme), puis Interface, Contrôleur, BaseDeDonnees (à droite) : c'est l'ordre naturel de propagation.

Étapes 3 à 5 — Lignes de vie, messages et retours :



Vérification : les numéros 1 à 12 descendent strictement ; chaque appel au Contrôleur ouvre une barre d'activation ; chaque requête à la base possède son retour en pointillés à pointe ouverte. Le diagramme est cohérent et complet.

À RETENIR L'essentiel de la section

- Le diagramme de séquence représente les interactions entre objets dans le temps, pour un scénario d'un cas d'utilisation ; il complète dynamiquement le diagramme de cas d'utilisation.
- Composants : acteur (bonhomme + nom:Classe souligné), objet (rectangle + nomObjet:Classe souligné), ligne de vie (verticale pointillée), barre d'activation (rectangle fin = période d'activité), message (flèche horizontale étiquetée `operation(parametres)`).
- Le temps s'écoule de haut en bas : la lecture est chronologique et les messages sont numérotés dans l'ordre.
- Message synchrone = flèche pleine, pointe fermée ; Retour = flèche pointillée, pointe ouverte ; Création = flèche vers le rectangle de l'objet (souvent `<<create>>`) ; Destruction = croix X en fin de ligne de vie.
- Construction en 5 étapes : identifier, ordonner, lignes de vie, messages ordonnés, retours.

Maintenant que les composants sont maîtrisés, la section suivante approfondira les **types de messages** (synchrone, asynchrone) et la représentation des **scénarios complexes** (alternatives, boucles, parallélisme) grâce aux fragments combinés.

4 TYPES DE MESSAGES ET SCÉNARIOS COMPLEXES

Dans la section précédente, nous avons appris à construire le squelette d'un diagramme de séquence : les objets alignés horizontalement, leurs lignes de vie verticales en pointillés, les barres d'activation et les messages qui circulent entre eux. Mais nous avons volontairement dessiné toutes les flèches de la même façon. Or, dans la réalité d'un système d'information, tous les échanges ne se ressemblent pas : quand tu composes ton code PIN sur un distributeur GIMAC à Douala, la machine attend la réponse de la banque avant de continuer ; en revanche, quand elle envoie un SMS de notification, elle n'attend rien du tout et poursuit son travail. UML distingue donc plusieurs **types de messages**, chacun avec une notation graphique précise. C'est ce que nous étudions maintenant, avant de voir comment numéroter les messages et couvrir tous les scénarios d'un cas d'utilisation.

4.1 Le message synchrone : l'émetteur attend la réponse

DÉFINITION Message synchrone

Un **message synchrone** est un message dont l'émetteur se bloque en attendant que le récepteur ait terminé le traitement demandé et renvoyé sa réponse. Il se représente par une flèche pleine (trait continu) à tête pleine (triangle noir rempli).

L'intuition est simple : c'est un appel téléphonique. Quand tu appelles le service client de CAM-TEL, tu restes en ligne, tu ne fais rien d'autre, jusqu'à ce que ton interlocuteur te réponde. En programmation, c'est exactement le comportement d'un appel de fonction classique : l'instruction suivante ne s'exécute qu'après le retour de la fonction.

PROPRIÉTÉ Message synchrone et barre d'activation (admise)

Nous admettons la propriété suivante : tout message synchrone provoque l'apparition d'une barre d'activation chez le récepteur, barre qui dure jusqu'à l'émission du message de retour correspondant. Pendant toute cette durée, le récepteur est actif (il exécute le traitement) et l'émetteur est suspendu (il attend). Une barre d'activation ne peut donc pas être plus courte que l'intervalle entre la réception du message synchrone et l'envoi de son retour.

ATTENTION La flèche pleine n'est pas une flèche de retour

Erreur extrêmement fréquente au Baccalauréat : dessiner la réponse avec une flèche pleine à tête pleine. C'est faux. La flèche pleine à tête pleine est réservée à la demande synchrone (l'appel). La réponse se dessine toujours en pointillés (voir plus bas). Un diagramme où l'aller et le retour ont la même flèche pleine est pénalisé, car il suggère que le récepteur émet à son tour une demande synchrone bloquante, ce qui créerait une attente mutuelle impossible.

4.2 Le message asynchrone : l'émetteur continue sans attendre

DÉFINITION Message asynchrone

Un **message asynchrone** est un message dont l'émetteur n'attend pas de réponse : il poursuit immédiatement son propre déroulement après l'envoi. Il se représente par une flèche à trait continu et à tête ouverte (simple V, non rempli).

C'est l'équivalent d'un SMS ou d'un dépôt de lettre à la boîte : tu envoies, puis tu continues ta journée. Le message asynchrone est idéal pour modéliser les notifications, les événements, les signaux : envoi d'un SMS Orange Money après une transaction, publication d'un événement « commande passée » vers un module de statistiques, etc.

EXEMPLE Synchronisme ou asynchrone ? Le cas du retrait Mobile Money

Comparons deux échanges d'une même transaction de retrait :

- Vérification du code PIN : synchrone. L'application envoie `verifierPIN(1234)` au serveur de la banque et se bloque : impossible de délivrer l'argent avant de savoir si le code est correct. Flèche pleine à tête pleine.
- Envoi du SMS de confirmation : asynchrone. Une fois l'argent remis, l'application demande au module de notification `envoyerSMS("Retrait de 20 000 FCFA effectué")` et continue : elle affiche déjà le reçu, elle ne va pas geler l'écran le temps que le SMS parte. Flèche à tête ouverte.

Critère de choix : « l'émetteur a-t-il besoin du résultat pour continuer ? ». Si oui → synchrone ; si non → asynchrone.

4.3 Le message de retour : la réponse, en pointillés

DÉFINITION Message de retour

Un **message de retour** représente la réponse à un message synchrone : il ramène le résultat du traitement vers l'émetteur. Il se représente par une flèche en pointillés à tête ouverte, orientée du récepteur vers l'émetteur, généralement étiquetée par la valeur retournée (par exemple `solde = 45 000`).

Le message de retour est souvent **optionnel** : lorsque la réponse est évidente ou sans intérêt pour la compréhension, on peut l'omettre pour alléger le diagramme. En revanche, dès qu'une valeur importante transite (un solde, un résultat de vérification, un numéro de transaction), il faut le dessiner et l'étiqueter clairement.

4.4 Le message réflexif (auto-message)

Il arrive qu'un objet s'envoie un message à lui-même : par exemple, le distributeur calcule lui-même les frais de retrait avant de débiter, ou un objet `Compte` appelle sa propre méthode `verifierDecouvert`. On parle de **message réflexif** (ou auto-message) : la flèche part de la ligne de vie de l'objet, forme un petit **coude** et revient sur cette même ligne de vie, créant éventuellement une seconde barre d'activation superposée à la première.

4.5 Tableau récapitulatif des quatre types de flèches

Notation graphique	Nom du message	Signification
	Synchrone	Flèche pleine, tête pleine. L'émetteur se bloque et attend la réponse du récepteur.
	Asynchrone	Flèche pleine, tête ouverte. L'émetteur continue immédiatement sans attendre de réponse.
	Retour	Flèche en pointillés, tête ouverte. Réponse à un message synchrone ; souvent optionnel.
	Réflexif	Flèche en pointillés revenant sur la même ligne de vie : l'objet s'envoie un message à lui-même.

Les quatre types de messages UML : notation graphique et signification.

4.6 Numérotation des messages et imbrication

Quand un diagramme contient beaucoup de messages, la seule position verticale ne suffit plus à exprimer clairement qui a déclenché quoi. UML propose donc une **numérotation hiérarchique** des messages : 1, 2, 3... pour les messages de premier niveau, et 2.1, 2.2, 2.3... pour les messages émis pendant le traitement du message 2 (on dit qu'ils sont imbriqués dans 2).

MÉTHODE Numéroté les messages et vérifier la chronologie

1. Parcourir le diagramme de haut en bas : le temps s'écoule verticalement, donc le premier message reçoit le numéro 1.
2. Numéroté les messages de premier niveau dans l'ordre chronologique : 1, 2, 3... Un message est de premier niveau s'il est émis par l'acteur ou hors de toute activation imbriquée.
3. Numéroté les messages imbriqués : tout message émis pendant la barre d'activation déclenchée par le message n reçoit un numéro de la forme $n.1, n.2...$ dans l'ordre chronologique. On peut descendre d'un niveau supplémentaire : 2.1.1.
4. Associer les retours : le retour du message n peut être noté n en pointillés (ou simplement étiqueté par la valeur retournée), et il doit apparaître après tous les messages imbriqués $n.x$.
5. Vérifier la cohérence chronologique : pour tout couple de messages, si a est dessiné au-dessus de b sur la même ligne de vie, alors a doit avoir un numéro antérieur à b . Toute contradiction (message 3 dessiné au-dessus du message 2 sur la même ligne de vie) signale une erreur de construction.

4.7 Exemple complet : scénario « Retirer de l'argent » au distributeur

Mettons tout cela en œuvre sur le scénario nominal du cas d'utilisation Retirer de l'argent, rencontré dans les sections précédentes. Le client insère sa carte et saisit son code ; le distributeur (DAB) demande au système central de la banque de vérifier le code (synchrone), puis de débiter le compte (synchrone, avec retour du nouveau solde), calcule lui-même les frais (message réflexif), délivre les billets et déclenche l'envoi d'un SMS (asynchrone).

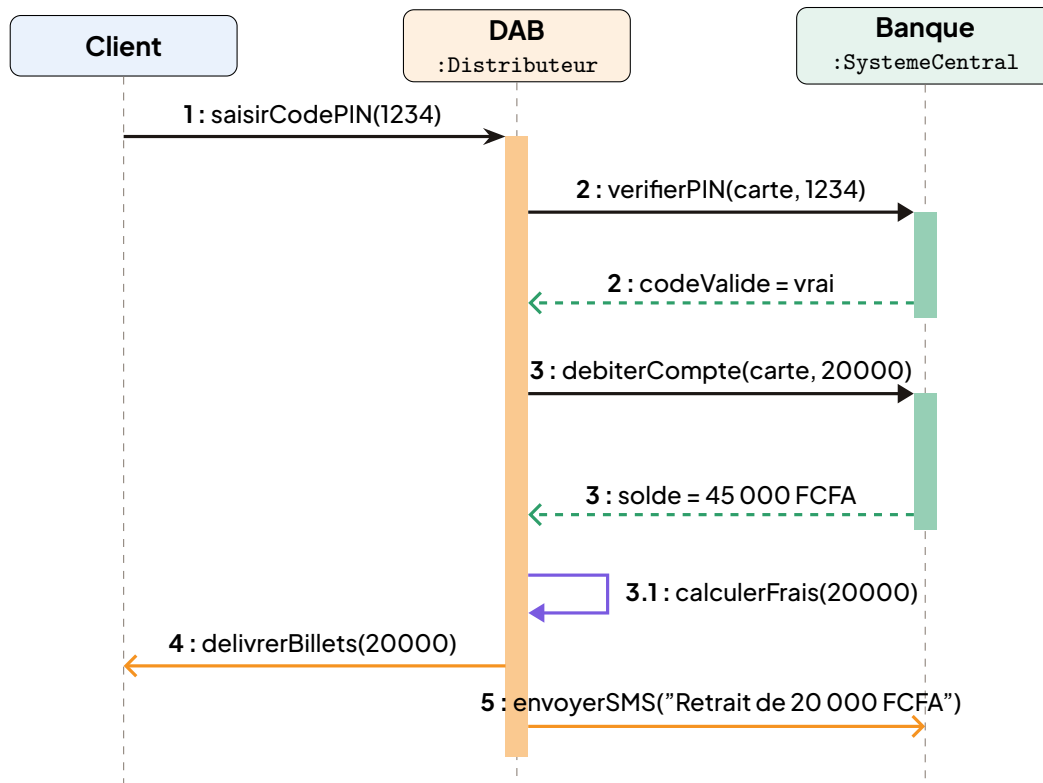


Diagramme de séquence du scénario nominal « Retirer de l'argent » : messages numérotés, vérification synchrone du PIN, retour du solde en pointillés, auto-message de calcul des frais et SMS asynchrone.

Observons la cohérence de la numérotation : le message 3.1 (calcul des frais) est bien imbriqué dans le traitement du débit, d'où son numéro à deux niveaux ; le retour du message 3 (le solde) arrive après 3.1, ce qui respecte la chronologie ; le SMS (message 5) est asynchrone car le distributeur n'a pas besoin d'attendre sa confirmation pour terminer la session.

4.8 Du scénario nominal aux scénarios alternatifs

Un cas d'utilisation possède en général un **scénario nominal** (tout se passe bien) et des **scénarios alternatifs** ou d'exception (code PIN faux, solde insuffisant, distributeur en panne de billets). La règle méthodologique est la suivante :

À RETENIR Un diagramme de séquence par scénario

On dessine un diagramme de séquence par scénario, en commençant toujours par le scénario nominal. Les variantes simples peuvent être indiquées sur le même diagramme par des notes UML (petits rectangles à coin plié) ; les variantes complexes donnent lieu à un diagramme séparé. En complément utile pour le Bac, UML 2 propose les **fragments combinés** : un rectangle étiqueté **alt** (alternative, avec plusieurs compartiments séparés

par des pointillés, chacun gardé par une condition entre crochets comme `[code faux]`) ou `opt` (fragment optionnel exécuté seulement si la condition est vraie) encadre la partie du diagramme concernée. C'est l'équivalent du `si...sinon` de l'algorithmique, directement dessiné sur le diagramme.

EXERCICE RÉSOLU 4 Cohérence entre cas d'utilisation et diagrammes de séquence

Énoncé. Le diagramme de cas d'utilisation d'une application de gestion de bibliothèque scolaire contient quatre cas d'utilisation : Emprunter un livre, Rendre un livre, Consulter le catalogue et Gérer les adhérents. Le concepteur n'a produit qu'un seul diagramme de séquence, pour Emprunter un livre. (1) Cette modélisation est-elle complète ? (2) Pour Emprunter un livre, le message `verifierAdhesion(adherent)` est dessiné avec une flèche en pointillés et son retour `adhesionValide` avec une flèche pleine à tête pleine. Corriger.

SOLUTION

Solution. (1) Non. La règle de cohérence entre les deux diagrammes exige que chaque cas d'utilisation important donne lieu à au moins un diagramme de séquence (celui de son scénario nominal). Il manque donc au minimum trois diagrammes de séquence : un pour Rendre un livre, un pour Consulter le catalogue et un pour Gérer les adhérents. Sans eux, le comportement dynamique du système n'est pas décrit et les développeurs ne savent pas quels objets collaborent pour ces services. (2) Les deux flèches sont inversées. `verifierAdhesion(adherent)` est une demande dont le système attend le résultat avant de prêter le livre : c'est un message synchrone, à dessiner en flèche pleine à tête pleine. `adhesionValide` est la réponse : c'est un message de retour, à dessiner en flèche en pointillés à tête ouverte. Rappel : le retour est toujours en pointillés, jamais en trait plein.

EXERCICE 3 À toi de jouer

On considère le cas d'utilisation Recharger son compte d'une application de type Orange Money. Scénario nominal : l'utilisateur saisit le montant ; l'application demande au serveur de vérifier le solde du compte source (l'application attend la réponse) ; le serveur répond `soldeOK` ; l'application demande le transfert (attente de la confirmation) ; le serveur confirme avec le nouveau solde ; l'application déclenche l'envoi d'une notification push sans attendre. Dessine le diagramme de séquence complet : objets, lignes de vie, barres d'activation, types de flèches corrects et numérotation des messages.

CORRIGÉ Exercice : Recharger son compte

Objets : Utilisateur, :Application, :Serveur. Messages, de haut en bas : 1 :

saisirMontant(m) de l'utilisateur vers l'application (asynchrone, tête ouverte); 2 : verifierSolde(compte, m) synchrone (flèche pleine à tête pleine) vers le serveur, avec barre d'activation; 2 : retour soldeOK en pointillés; 3 : transferer(compte, m) synchrone; 3 : retour nouveauSolde en pointillés; 4 : envoyerNotification(msg) asynchrone (tête ouverte, trait continu). La barre d'activation du serveur couvre chaque traitement synchrone jusqu'à son retour; la numérotation 1, 2, 3, 4 est croissante de haut en bas, donc chronologiquement cohérente.

À RETENIR L'essentiel de la section

- Message synchrone : flèche pleine à tête pleine; l'émetteur attend la réponse (blocage) et le récepteur active une barre d'activation jusqu'au retour.
- Message asynchrone : flèche à tête ouverte; l'émetteur continue sans attendre (notifications, SMS, événements).
- Message de retour : flèche en pointillés; toujours en pointillés, souvent optionnel, étiqueté par la valeur rendue.
- Message réflexif : flèche en coude sur la même ligne de vie.
- Numérotation hiérarchique (1, 2, 2.1...): elle exprime l'imbrication et permet de vérifier la chronologie.
- Un diagramme de séquence par scénario; fragments alt/opt ou notes pour les alternatives; chaque cas d'utilisation important possède au moins un diagramme de séquence.

5 PRODUCTION AVEC UN LOGICIEL PROFESSIONNEL

Dans la section précédente, tu as appris à construire à la main des diagrammes de séquence riches : messages synchrones et asynchrones, messages de retour, activations, scénarios avec alternatives (alt, opt, loop). C'est un savoir-faire indispensable pour comprendre la sémantique UML. Mais dans la vie professionnelle, personne ne livre à un client un diagramme griffonné au crayon : on utilise un **logiciel de modélisation** (ou modèleur UML). Cette section te montre comment passer du brouillon papier à un modèle propre, vérifiable, exportable, partageable et versionnable, avec les outils de référence du programme : ArgoUML, StarUML et WinDesign.

5.1 Pourquoi un logiciel de modélisation ?

Imaginons la situation suivante : la startup doualaïse EduPay développe une application de paiement des frais de scolarité par MTN Mobile Money et Orange Money. L'équipe compte quatre développeurs répartis entre Douala et Buea. Si chacun dessine ses diagrammes à la main, les

versions divergent en une semaine : tel cas d'utilisation s'appelle « Payer scolarité » chez l'un, « Effectuer paiement » chez l'autre, et personne ne sait quelle version est la bonne. Pire, une modification du cahier des charges impose de renommer l'acteur « Parent » en « Tuteur légal » : sur papier, c'est la chasse au trésor risquée ; dans un modèleur, c'est une opération unique propagée partout.

Un modèleur UML résout exactement ces problèmes. Il offre :

- une **représentation normalisée** : les symboles UML (ellipses, acteurs, flèches, fragments combinés) sont imposés par l'outil, donc identiques pour tous les membres de l'équipe ;
- un **dictionnaire des éléments** (appelé Model Explorer ou Explorateur de modèle) : chaque acteur, chaque cas d'utilisation, chaque classe, chaque objet n'existe qu'une seule fois dans le modèle, et est réutilisable par glisser-déposer dans plusieurs diagrammes ;
- des **vérifications de cohérence** (ArgoUML parle de Critiques) : le logiciel signale les noms dupliqués, les relations mal formées (ex. une flèche <<include>> sans stéréotype), les éléments orphelins (cas d'utilisation non relié à un acteur) ;
- des **exports** propres (PNG, PDF, SVG, EMF) pour les rapports, les soutenances du Baccalauréat et la documentation client ;
- la **sauvegarde et le partage** du modèle via un format standard (XMI – XML Metadata Interchange) ou natif (.zargo, .mdj), base du travail collaboratif (dépôt Git, dossier partagé).

À RETENIR**Idée directrice**

Un modèleur UML n'est pas un logiciel de dessin (type Paint, Draw.io, Visio) : c'est un éditeur de modèle sémantique. Quand tu places une relation <<include>> avec l'outil dédié, le logiciel enregistre une véritable relation dans le dictionnaire, pas seulement une flèche sur l'écran. Cette distinction est la clé de la productivité professionnelle.

5.2 Panorama des logiciels professionnels

Le programme officiel de Terminale TI cite trois outils représentatifs de trois philosophies différentes. Le tableau ci-dessous te permet de choisir selon ton contexte (lycée, maison, stage, future entreprise).

Critère	ArgoUML	StarUML (v4+)	WinDesign
Licence	Libre et gratuit (Open Source, LGPL)	Commerciale, version d'évaluation gratuite illimitée (watermark)	Commerciale (licences éducation / académique possibles)
Technologie	Écrit en Java (nécessite JRE/JDK), multiplateforme (Windows, Linux, macOS)	Application Electron (Node.js + Chromium), multiplateforme moderne, interface soignée	Suite Windows native (32/64 bits), éditeur français (CECIMA)
Points forts	Simplicité, Critiques automatiques (conseils de modélisation), idéal pour apprendre sans coût, support UML 1.4/2.0 partiel	Génération des barres d'activation automatique , fragments combinés (alt, opt, loop) natifs, extensions (C#, Java, Python, SQL), support UML 2.5 complet	Modélisation Merise (MCD, MLD, MPD) et UML 2.5 dans un même projet, très répandu en entreprise francophone, génération de code & SQL
Usage typique	Enseignement secondaire/supérieur, projets scolaires, environnements Linux	Projets professionnels, startups, développeurs full-stack, préparation certifications (OCUP)	Cabinets de conseil, administrations, banques, grands comptes francophones (Cameroun, Côte d'Ivoire, France)
Format natif	.zargo (XML zippé), .uml (XMI)	.mdj (JSON)	.wdp (projet), .wdm (modèle)

DÉFINITION **Modeleur UML**

Un **modeleur UML** est un logiciel qui permet de créer, éditer, vérifier, sauvegarder et exporter des modèles UML (diagrammes de cas d'utilisation, de séquence, de classes, d'activités, d'états-transitions, etc.) en respectant la norme OMG UML 2.x. Il maintient un **dictionnaire des éléments** (repository) du modèle, garantissant qu'un même acteur ou cas d'utilisation est défini une seule fois et réutilisé partout (unicité de définition).

LESAVAIS-TU ? **WinDesign, le champion francophone**

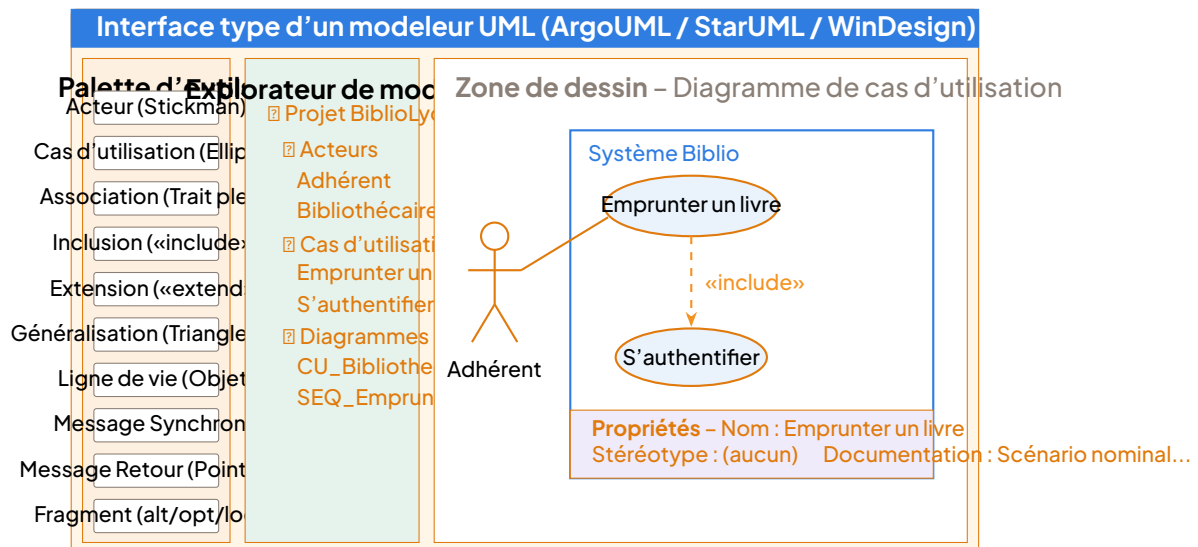
WinDesign, développé par la société française CECIMA, est l'un des outils de modélisation les plus utilisés dans les entreprises françaises et africaines francophones. Sa particularité historique : il couvre à la fois la méthode **Merise** (MCD, MLD, MPD – encore très présents dans les administrations, les banques comme BICEC, UBA, ou les grands comptes comme CAMTEL, SONARA) et la notation **UML**. Beaucoup de cabinets d'études qui travaillent pour des administrations en ligne (impots.cm, ANTIC, CENADI) ou des projets d'e-

gouvernance l'utilisent pour concevoir leurs systèmes d'information. Maîtriser un modeleur au lycée, c'est donc déjà parler le langage des bureaux d'études de Douala, Yaoundé, Abidjan ou Paris.

5.3 L'interface type d'un modeleur UML

Quel que soit le logiciel (ArgoUML, StarUML, WinDesign, Visual Paradigm, Enterprise Architect), l'écran s'organise toujours autour des mêmes quatre zones. Les reconnaître, c'est savoir utiliser n'importe quel modeleur en cinq minutes.

- L'**explorateur de modèle** (souvent à gauche ou à droite) : arborescence de tous les éléments du projet — acteurs, cas d'utilisation, classes, objets, diagrammes, packages. C'est le dictionnaire vivant du modèle (Model Browser).
- La **palette d'outils** (en haut, à gauche ou flottante) : les boutons pour créer un acteur, un cas d'utilisation, une association, une relation d'inclusion (<<include>>), d'extension (<<extend>>), de généralisation, un objet (ligne de vie), un message synchrone, un message de retour, un fragment combiné (alt, opt), une note, etc.
- La **zone de dessin** (au centre) : la feuille (souvent quadrillée) où le diagramme prend vie. C'est une vue sur le modèle.
- Le panneau de **propriétés** (en bas ou à droite) : nom, stéréotype, visibilité, documentation, contraintes, paramètres de l'élément sélectionné dans la zone de dessin ou l'explorateur.



Reconstitution schématique et commentée de l'interface d'un modeleur UML : palette d'outils (gauche), explorateur de modèle (centre-gauche), zone de dessin avec le diagramme en cours (centre-droite) et panneau de propriétés de l'élément sélectionné (bas).

ATTENTION Piège classique : dessiner au lieu de modéliser

L'erreur la plus fréquente des débutants (et piège d'examen) consiste à utiliser le modeleur comme un logiciel de dessin vectoriel : tracer une flèche « libre » (outil Ligne) entre deux ellipses au lieu de choisir l'outil Relation d'inclusion, ou taper « include » dans une note de texte. Visuellement, le résultat se ressemble ; mais pour le logiciel, aucune relation sémantique n'existe : le dictionnaire des éléments reste vide de cette information, les vérifications de cohérence (Critiques) ne fonctionnent pas, la génération de code est impossible, et si tu renommes le cas d'utilisation dans l'explorateur, la fausse flèche ne suit pas. Règle d'or : chaque lien doit être créé avec l'outil de relation normalisé de la palette (Association, <<include>>, <<extend>>, Généralisation, Réalisation), jamais avec l'outil de dessin libre (Line/Connector générique).

5.4 Créer un diagramme de cas d'utilisation

Reprenons notre fil rouge : la bibliothèque du lycée. Le mini-cahier des charges suivant servira de support à toute la section.

EXEMPLE Mini-cahier des charges – Bibliothèque scolaire de Nkongsamba

Le lycée de Nkongsamba veut informatiser sa bibliothèque. L'Adhérent (élève ou enseignant) peut emprunter un livre et consulter le catalogue. Le Bibliothécaire peut enregistrer un retour et gérer les adhérents ; il est aussi un adhérent particulier (généralisation d'acteur). Tout emprunt exige de s'authentifier (inclusion). Lors de l'enregistrement d'un retour, le système peut, si le livre est en retard, calculer une pénalité (extension). Deux diagrammes sont attendus : le diagramme de cas d'utilisation global, et le diagramme de séquence du scénario nominal « Emprunter un livre ».

MÉTHODE Produire un diagramme de cas d'utilisation sous ArgoUML ou StarUML en 6 étapes

1. Créer le projet : menu Fichier → Nouveau projet (ou File → New), nommer le projet (ex. BiblioLycee) et enregistrer immédiatement le fichier (.zargo pour ArgoUML, .mdj pour StarUML) dans ton dossier de travail.
2. Créer le diagramme : dans l'explorateur, clic droit sur le paquetage racine (souvent nommé Model ou Default) → Ajouter un diagramme → Diagramme de cas d'utilisation (Use Case Diagram). Le nommer explicitement, par exemple CU_Bibliotheque.
3. Placer les acteurs : avec l'outil Acteur de la palette, créer Adhérent et Bibliothécaire à gauche de la zone de dessin. Chaque acteur apparaît automatiquement dans l'explorateur sous le nœud Acteurs.

4. Dessiner le cadre du système et les cas d'utilisation : tracer le rectangle du système (outil System Boundary ou Package/Frame) nommé Système Biblio, puis placer les ellipses avec l'outil Cas d'utilisation : Emprunter un livre, Consulter le catalogue, Enregistrer un retour, Gérer les adhérents, S'authentifier, Calculer une pénalité. Nommer chaque cas par un verbe à l'infinitif suivi de son complément (règle de nommage UML).
5. Créer les associations acteur-cas : avec l'outil Association (trait plein simple), relier chaque acteur aux cas qu'il déclenche. Adhérent vers Emprunter un livre et Consulter le catalogue ; Bibliothécaire vers Enregistrer un retour et Gérer les adhérents. Ne jamais utiliser l'outil ligne libre.
6. Créer les relations normalisées inter-cas :
 - Inclusion : outil Include (flèche pointillée, stéréotype <<include>>), de Emprunter un livre (cas de base) vers S'authentifier (cas inclus).
 - Extension : outil Extend (flèche pointillée, stéréotype <<extend>>), de Calculer une pénalité (cas d'extension) vers Enregistrer un retour (cas de base).
 - Généralisation d'acteur : outil Generalization (flèche trait plein, triangle vide), de Bibliothécaire (acteur spécialisé) vers Adhérent (acteur général).

Vérifier dans le panneau de propriétés que les stéréotypes <<include>> et <<extend>> s'affichent bien sur les connecteurs.

ATTENTION Sens des flèches : le piège de l'examen – Rappel vital

Deux erreurs de sens reviennent sans cesse au Baccalauréat.

- Inclusion (<<include>>) : la flèche part du cas de base (celui qui a besoin de l'autre) vers le cas inclus (le service réutilisé). Mnémotechnique : « La base appelle l'inclus » → Flèche Base --> Inclus. Ex. : Emprunter --> S'authentifier.
- Extension (<<extend>>) : la flèche part du cas d'extension (le comportement optionnel/conditionnel) vers le cas de base (celui qui est étendu). Mnémotechnique : « L'extension s'accroche à la base » → Flèche Extension --> Base. Ex. : Calculer pénalité --> Enregistrer retour.
- Généralisation : triangle vide pointant vers le parent (général), départ de l'enfant (spécialisé). Bibliothécaire → Adhérent.

5.5 Créer un diagramme de séquence

Le diagramme de séquence du scénario nominal « Emprunter un livre » fait intervenir trois lignes de vie : l'objet a : Adhérent, l'objet ecran : EcranEmprunt (ou interface) et l'objet gestion : GestionnaireEmprunts. Le bon modeleur (StarUML, WinDesign, ArgoUML récent) génère **automatiquement les barres d'activation** (rectangles fins sur la ligne de vie) quand on pose un message synchrone : c'est l'un de ses plus grands confort et une garantie de justesse temporelle.

MÉTHODE Produire un diagramme de séquence sous StarUML (recommandé pour les séquences)

1. Créer le diagramme : dans l'explorateur, clic droit sur le modèle (ou un package) → Add Diagram → Sequence Diagram. Le nommer SEQ_EmprunterLivres (convention : un diagramme par scénario nominal ou alternatif).
2. Créer les lignes de vie (Lifelines) : avec l'outil Lifeline (ou Object Lifeline) de la palette, déposer trois lignes de vie verticales en haut de la zone de dessin, espacées régulièrement. Dans le panneau de propriétés, saisir pour chacune le nom et le type (classe) au format UML standard : `nomObjet : NomClasse` (ex. `a : Adhérent`, `ecran : EcranEmprunt`, `gestion : GestionnaireEmprunts`). L'underline (soulignement) indique une instance.
3. Envoyer les messages synchrones (Appels d'opération) : sélectionner l'outil Synchronous Message (flèche pleine, tête triangulaire fermée), cliquer sur la ligne de vie émettrice puis glisser jusqu'à la ligne de vie réceptrice (respecter l'ordre chronologique de haut en bas). Le modèleur crée automatiquement la barre d'activation sur le récepteur. Nommer le message par l'opération appelée avec ses paramètres : `demandeEmprunt(codeLivre)`.
4. Ajouter les messages de retour (Reply/Return) : avec l'outil Reply Message (flèche en pointillés, tête ouverte), repartir du bas de la barre d'activation du récepteur vers l'émetteur : `confirmationEmprunt` ou `return livreDisponible`. Le retour se rattache automatiquement à la fin de l'activation.
5. Enchaîner le scénario nominal complet : poursuivre dans l'ordre vertical :
 - (a) `demandeEmprunt(codeLivre)` – synchrone, `a` → `ecran`;
 - (b) `verifierDisponibilite(codeLivre)` – synchrone, `ecran` → `gestion` (activation sur `gestion`);
 - (c) `disponible` – retour, `gestion` --> `ecran` (fin activation `gestion`);
 - (d) `enregistrerEmprunt()` – synchrone, `ecran` → `gestion` (nouvelle activation);
 - (e) `confirmationEmprunt` – retour, `ecran` --> `a` (fin activation `ecran`).
6. (Optionnel) Ajouter des fragments combinés : pour les scénarios alternatifs (ex. livre indisponible), utiliser l'outil Combined Fragment `alt / opt / loop` de la palette, englober les messages concernés, et renseigner la garde (condition) dans l'opérande (ex. `[livre indisponible]`).
7. Documenter, vérifier et exporter : ajouter une Note (outil Note) rappelant le scénario, l'auteur, la date, la version. Vérifier l'orthographe des opérations dans l'explorateur (elles deviennent les méthodes des classes plus tard). Exporter : File → Export Diagram As → PNG / PDF / SVG pour l'insérer dans le rapport de soutenance.

EXPÉRIENCE TP guidé – Modéliser l'emprunt de la bibliothèque

Objectif : produire les deux diagrammes attendus du mini-cahier des charges et les exporter en PNG prêts pour le rapport de projet.

Matériel et logiciels : un poste informatique de la salle multimédia (Windows/Linux), ArgoUML (libre, téléchargeable sur argouml.tigris.org ou via gestionnaire de paquets Linux) ou StarUML (téléchargeable sur staruml.io, version d'évaluation fonctionnelle).

Manipulation :

1. Lance le modeleur et crée le projet `BiblioLycee`. Sauvegarde (Ctrl+S).
2. Suis la méthode en 6 étapes pour construire le diagramme de cas d'utilisation complet (2 acteurs, cadre système, 6 cas, 4 associations, 1 inclusion, 1 extension, 1 généralisation).
3. Crée le diagramme de séquence `SEQ_EmprunterLivre` avec les 3 lignes de vie et les 5 messages du scénario nominal (voir méthode ci-dessus). Observe la création automatique des barres d'activation.
4. Test de la puissance du modèle : dans l'explorateur, renomme le cas `Emprunter un livre` en `Emprunter un document`. Observe : le nom se met à jour partout (diagramme CU, explorateur, futures références). C'est la preuve que tu manipules un modèle unique, pas des dessins indépendants.
5. Déplace l'acteur `Adhérent` sur le diagramme : les associations le suivent. Déplace un message sur le diagramme de séquence : l'activation se redimensionne.
6. Exporte chaque diagramme en PNG (File → Export → PNG) et insère-les dans ton compte rendu (LibreOffice Writer / Word / LaTeX).

Observations attendues : les relations créées avec les bons outils suivent les éléments quand on les déplace (rubber-banding) ; le renommage dans l'explorateur se propage à toutes les vues ; les activations du diagramme de séquence se créent et se dimensionnent automatiquement selon la hauteur des messages.

Interprétation : le modeleur maintient la cohérence du modèle (un élément = une définition unique dans le repository) et automatise les tâches graphiques fastidieuses (alignement, activation, propagation de nom). C'est exactement ce qui distingue un outil professionnel d'un logiciel de dessin.

5.6 Vérifications, export et bonnes pratiques professionnelles

Un modeleur sérieux t'aide à contrôler la qualité de ton travail avant de le livrer (revue de conception, soutenance Bac, livraison client).

- **Cohérence des noms** : le logiciel signale les doublons (deux cas portant le même nom dans le même paquetage) et propage les renommages. ArgoUML va plus loin avec ses Critiques (onglet Critique ou ToDo) : de petites suggestions automatiques (« Ce cas d'utilisation n'est relié à aucun acteur », « Ce message n'a pas de nom d'opération »).

- **Dictionnaire des éléments (Repository)** : l'explorateur fait office de référentiel unique. **Avant de créer un élément, vérifie qu'il n'existe pas déjà** : on réutilise l'acteur existant en le glissant depuis l'explorateur vers le nouveau diagramme, on n'en crée pas un second (sinon doublon sémantique).
- **Export** : PNG ou SVG (vectoriel, zoomable) pour un rapport ou un diaporama, PDF pour un dossier d'analyse formel. Au Baccalauréat comme en entreprise, un diagramme exporté proprement (fond blanc, police lisible, pas de pixels) fait toujours meilleure impression qu'une capture d'écran floue (Impr. Écran).
- **Échange standard (XMI)** : pour l'interopérabilité entre outils (ex. ArgoUML → StarUML → WinDesign), utilise File → Export → XMI (version 2.1 ou 2.4). C'est le format norme OMG.

PROPRIÉTÉ**Bonnes pratiques professionnelles de modélisation UML**

1. Nommer chaque cas d'utilisation par un verbe à l'infinitif suivi d'un complément d'objet : Emprunter un livre, Consulter le catalogue – jamais Livre, Emprunt, Gestion livres.
2. Consacrer un diagramme de séquence par scénario (nominal, puis un par alternative majeure `alt`) plutôt qu'un diagramme unique surchargé et illisible.
3. Légender et dater chaque diagramme exporté (titre du diagramme, auteur, version, date) : un modèle évolue, il faut pouvoir identifier la version de référence (ex. `v1.0_2025-05-15`).
4. Nommer les messages par des opérations explicites avec leurs paramètres types : `verifierDisponibilite(codeLivre : String) : Boolean`. Cela prépare le diagramme de classes et le code.
5. Sauvegarder le projet à chaque étape majeure et conserver les versions horodatées (`Biblio_v1.zargo`, `Biblio_v2_seq_added.zargo`, etc.) ou utiliser Git sur le fichier projet.

EXERCICE RÉSOLU 5**Du cahier des charges aux diagrammes livrés – Sujet type Bac**

Énoncé. À partir du mini-cahier des charges de la bibliothèque de Nkongsamba, indique : (a) les relations normalisées à créer dans le modèleur et leur sens précis, (b) les lignes de vie et les messages du diagramme de séquence du scénario nominal « Emprunter un livre », (c) deux vérifications obligatoires avant l'export final.

SOLUTION

(a) Relations du diagramme de cas d'utilisation.

- Associations (trait plein) : Adhérent – Emprunter un livre ; Adhérent – Consulter le catalogue ; Bibliothécaire – Enregistrer un retour ; Bibliothécaire – Gérer les adhérents.
- Inclusion (`<<include>>`, flèche pointillée) : Emprunter un livre --> S'authentifier (flèche du cas de base vers le cas inclus, stéréotype affiché).

- **Extension** (<<extend>>, flèche pointillée) : Calculer une pénalité → Enregistrer un retour (flèche du cas d'extension vers le cas de base, stéréotype affiché).
- **Généralisation d'acteur** (flèche trait plein, triangle vide) : Bibliothécaire → Adhérent (triangle pointant vers l'acteur général).

(b) **Diagramme de séquence (Scénario nominal)**. Lignes de vie (instances) : a : Adhérent, ecran : EcranEmprunt, gestion : GestionnaireEmprunts. Messages dans l'ordre chronologique (haut → bas) :

1. demanderEmprunt(codeLivre) – Message synchrone (flèche pleine), de a vers ecran (création activation sur ecran);
2. verifierDisponibilite(codeLivre) – Message synchrone, de ecran vers gestion (création activation sur gestion);
3. disponible : Boolean – Message de retour (flèche pointillés), de gestion vers ecran (fin activation gestion);
4. enregistrerEmprunt() – Message synchrone, de ecran vers gestion (nouvelle activation sur gestion);
5. confirmationEmprunt – Message de retour, de ecran vers a (fin activation ecran).

(c) **Vérifications avant export (Checklist qualité)**.

1. Explorateur : contrôler l'absence de doublons de noms (acteurs, cas, opérations), l'orthographe exacte des opérations (cohérence avec le futur diagramme de classes), et que chaque cas d'utilisation est relié à au moins un acteur.
2. Diagrammes : cliquer sur chaque flèche de relation (<<include>>, <<extend>>, Généralisation) : le panneau de propriétés doit afficher le stéréotype correct, pas « Association » ou « Ligne ». Vérifier que les barres d'activation du diagramme de séquence couvrent bien la durée de vie des traitements.

Ensuite seulement : export PNG/PDF haute résolution, ajout de la légende (Titre, Auteur, Date, Version) sur chaque figure dans le rapport.

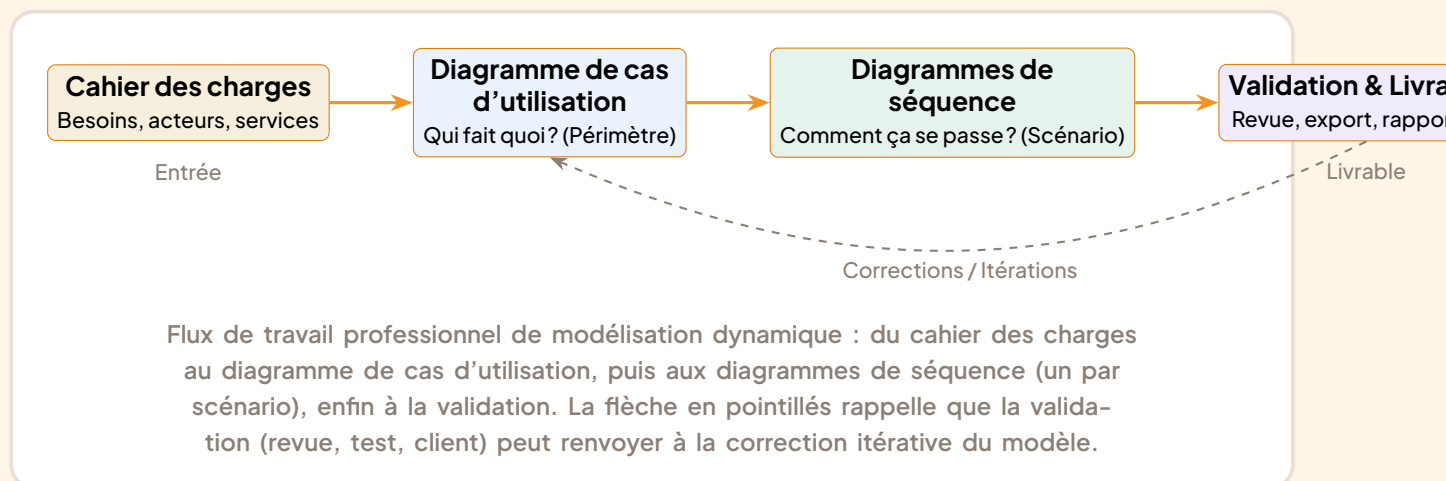
5.7 Sauvegarde, réutilisation et travail collaboratif

Un modèle n'est pas un jetable : c'est un **capital** intellectuel pour le projet, base de la traçabilité (exigences → conception → code → tests).

- **Sauvegarde et versioning** : chaque outil a son format de projet natif (.zargo ou .uml pour ArgoUML, .mdj pour StarUML, .wdp pour WinDesign). Sauvegarde fréquemment (Ctrl+S) et garde des versions numérotées ou, mieux, place le dossier du projet sous **Git** (init, commit à chaque étape). Le format .mdj (JSON) de StarUML se diffère bien dans Git.
- **Réutilisation inter-diagrammes** : tout élément du dictionnaire (acteur, cas, classe, interface) peut être glissé depuis l'explorateur vers un nouveau diagramme. Le diagramme de séquence réutilise ainsi les acteurs du diagramme de cas d'utilisation : la cohérence entre la vue statique (qui fait quoi) et la vue dynamique (comment ça se passe) est garantie par

construction.

- **Travail collaboratif réel** : le fichier de modèle se partage comme tout document (clé USB, dossier partagé Nextcloud/Seafile, dépôt Git/GitHub/GitLab). Dans une équipe, on se répartit les diagrammes – un membre pour les cas d'utilisation, un autre pour les séquences, un troisième pour le diagramme de classes – puis on fusionne (merge) en vérifiant que le dictionnaire reste unique (pas de doublons Adhérent / Adherent). C'est exactement le mode de travail des équipes d'EduPay ou des bureaux d'études qui conçoivent les applications des administrations en ligne (impots.cm, ANTIC, CENADI).



EXERCICE 4 À toi de jouer – Sujet d'entraînement Bac

Le cybercafé « NetPlus » de Bafoussam veut un logiciel de gestion. Le Client peut se connecter à un poste et recharger son compte ; le Gérant peut consulter les statistiques. Toute connexion inclut vérifier le solde ; la recharge peut, si le client est abonné, être étendue par appliquer une remise.

1. Installe ArgoUML ou ouvre StarUML et produis le diagramme de cas d'utilisation complet avec les relations normalisées (associations, inclusion, extension). Vérifie les stéréotypes dans le panneau propriétés.
2. Produis le diagramme de séquence du scénario nominal « Se connecter à un poste » (lignes de vie : c : Client, accueil : EcranAccueil, sessions : GestionnaireSessions). Gère les activations automatiques.
3. Exporte les deux diagrammes en PNG, légende-les (titre du diagramme, ton nom, date, version v1.0) et rassemble-les dans un compte rendu d'une page (contexte, choix de relations, capture des exports).

CORRIGÉ Exercice NetPlus – Éléments attendus pour l'auto-évaluation

1. Diagramme de cas d'utilisation :

- Acteurs : Client, Gérant.
- Cas d'utilisation (verbe infinitif) : Se connecter à un poste, Recharger son compte, Consulter les statistiques, Vérifier le solde, Appliquer une remise.
- Relations :
 - Associations : Client – Se connecter à un poste, Client – Recharger son compte, Gérant – Consulter les statistiques.
 - Inclusion (<<include>>) : Se connecter à un poste --> Vérifier le solde (Base → Inclus).
 - Extension (<<extend>>) : Appliquer une remise --> Recharger son compte (Extension → Base, condition [client abonné]).
- Toutes les flèches créées avec les outils normalisés (vérifier stéréotypes dans propriétés).

2. Diagramme de séquence (Scénario nominal « Se connecter ») :

- Lignes de vie : c : Client, accueil : EcranAccueil, sessions : GestionnaireSessions.
- Messages (ordre vertical) :
 - demanderConnexion(identifiant : String) – Synchrones, c → accueil (activation accueil);
 - verifierSolde(identifiant : String) : Boolean – Synchrones, accueil → sessions (activation sessions);
 - soldeOK : Boolean – Retour, sessions --> accueil (fin activation sessions);
 - ouvrirSession() : Poste – Synchrones, accueil → sessions (activation sessions);
 - posteAttribue : Poste – Retour, sessions --> accueil (fin activation sessions);
 - connexionReussie(poste : Poste) – Retour, accueil --> c (fin activation accueil).
- Les barres d'activation doivent apparaître automatiquement sur les récepteurs des messages synchrones.

3. Compte rendu : Deux figures exportées (PNG/SVG, pas de captures d'écran floues), chacune titrée (Figure 1 : Diagramme de cas d'utilisation NetPlus, Figure 2 : Diagramme de séquence Connexion), datées, signées, plus un court paragraphe justifiant le choix de l'inclusion (service obligatoire réutilisé) et de l'extension (comportement optionnel conditionnel).

À RETENIR L'essentiel de la section – Fiche de révision Bac

- **Modeleur UML** = Éditeur de modèle (repository + dictionnaire unique), pas logiciel de dessin. Outils officiels : ArgoUML (libre, Java, critiques), StarUML (moderne, activations auto, fragments combinés), WinDesign (Merise+UML, standard francophone).
- **Interface universelle** : Explorateur (dictionnaire), Palette (outils normalisés), Zone de dessin (vues), Propriétés (détails sémantiques).
- **Diagramme CU** : Acteurs (bâtonnets), Cas (ellipses, verbe infinitif), Cadre système, Associations (trait plein), <<include>> (Base --> Inclus, obligatoire), <<extend>> (Extension --> Base, conditionnel), Généralisation acteur (Enfant → Parent).
- **Diagramme de séquence** : Lignes de vie (objet : Classe), Messages synchrones (plein, activation auto), Retours (pointillés, fin activation), Fragments combinés (alt/opt/loop) pour alternatives. Un diagramme par scénario.
- **Qualité** : Réutilisation via explorateur (pas de doublons), Vérification (Critiques, stéréotypes), Export propre (PNG/PDF/SVG), Légende+Date+Version, Sauvegarde versionnée (Git).

6 SYNTHÈSE ET MÉTHODOLOGIE D'EXAMEN

Dans la section précédente, tu as appris à produire tes diagrammes de cas d'utilisation et de séquence à l'aide de logiciels professionnels comme ArgoUML ou StarUML. Mais le jour du Baccalauréat, pas d'ordinateur : c'est toi, ta règle, ton stylo et ta compréhension profonde du formalisme UML qui feras la différence. Cette dernière section est donc ta **boîte à outils d'examen** : elle récapitule l'essentiel de la leçon, te donne une méthode infaillible de résolution, des grilles de relecture pour vérifier tes diagrammes, et un sujet type Bac entièrement corrigé.

6.1 Bilan : les deux diagrammes dynamiques au programme

Tout au long de cette leçon, nous avons manipulé deux diagrammes qui se répondent comme les deux faces d'une même pièce. Il est fondamental de ne jamais les confondre, car l'examineur teste précisément ta capacité à choisir le bon outil pour la bonne question.

À RETENIR Les deux vues complémentaires

- Le **diagramme de cas d'utilisation** offre une vue externe du système : il répond aux questions qui utilise le système et quoi fait le système pour lui. Il est statique dans le temps : il montre les fonctionnalités offertes, pas leur déroulement.
- Le **diagramme de séquence** offre une vue interne et temporelle : il répond aux questions quand et comment les objets collaborent pour réaliser un scénario précis d'un

cas d'utilisation. Le temps s'écoule de haut en bas.

La correspondance est directe : un cas d'utilisation se détaille en un ou plusieurs diagrammes de séquence (un par scénario significatif : scénario nominal, scénarios alternatifs, scénarios d'erreur).

Critère	Cas d'utilisation	Séquence
Question posée	Qui fait quoi ?	Quand et comment ?
Point de vue	Externe (utilisateur)	Interne (objets du système)
Éléments clés	Acteurs, cas, relations	Lignes de vie, messages, activations
Dimension temporelle	Absente	Essentielle (haut → bas)
Usage type	Recueillir les besoins	Décrire un scénario précis

6.2 Carte mentale récapitulative de la leçon

Avant d'aller plus loin, voici une carte mentale qui condense l'intégralité des savoirs de cette leçon. Reproduis-la de mémoire sur un brouillon : si tu y arrives, tu maîtrises le chapitre.



Carte mentale récapitulative de la leçon : les cinq branches à connaître pour le Bac.

6.3 Démarche complète de modélisation dynamique

Face à un sujet d'examen, le danger n'est pas de ne pas savoir dessiner une ellipse : c'est de se lancer sans méthode et d'oublier un acteur ou d'inverser une flèche. Voici la démarche professionnelle, celle que suivent les analystes du CENADI comme les développeurs des startups de Douala.

MÉTHODE Fiche de résolution type Bac en 6 étapes

1. Lire l'énoncé deux fois et surligner tous les noms de personnes ou de systèmes (candidats aux acteurs) et tous les verbes d'action (candidats aux cas d'utilisation).
2. Identifier les acteurs et le périmètre : pour chaque candidat acteur, pose la question « est-il à l'extérieur du système et dialogue-t-il directement avec lui ? ». Trace le rectangle du système et place les acteurs dehors.

3. Lister les cas d'utilisation : transforme chaque verbe surligné en une ellipse dont le libellé commence par un verbe à l'infinitif (« Consulter les horaires », « Réserver un billet »).
4. Relier les cas : cherche les factorisations (*include*), les comportements optionnels (*extend*) et les cas qui sont des variantes d'un cas général (généralisation). Vérifie l'orientation de chaque flèche.
5. Décrire les scénarios : pour chaque cas important, rédige le scénario nominal puis les scénarios alternatifs et d'erreur, sous forme d'étapes numérotées.
6. Tracer les diagrammes de séquence : un par scénario demandé. Place les objets en haut, fais descendre le temps, numérote les messages et termine chaque appel synchrone par son retour en pointillés.

6.4 Grilles de relecture : ton filet de sécurité

Les cinq dernières minutes de l'épreuve doivent être consacrées à la relecture. Voici les deux grilles utilisées par les correcteurs eux-mêmes.

Grille de relecture d'un diagramme de cas d'utilisation

- ☐ Tous les acteurs sont à l'**extérieur** du rectangle du système ; aucun acteur n'est une ellipse.
- ☐ Tous les cas sont à l'**intérieur** du rectangle et leur libellé commence par un **verbe à l'infinitif**.
- ☐ Chaque acteur est relié à au moins un cas par une ligne pleine (association).
- ☐ Les flèches *include* partent du cas **de base** vers le cas **inclus** ; les flèches *extend* partent du cas **d'extension** vers le cas **de base**.
- ☐ Les flèches de généralisation (triangle vide) pointent du cas **spécialisé** vers le cas **général**.
- ☐ Les stéréotypes <<include>> et <<extend>> sont écrits sur les flèches en pointillés.

Grille de relecture d'un diagramme de séquence

- ☐ Les objets sont nommés et soulignés (format `nom:Classe`) et disposés en haut.
- ☐ Les messages se succèdent **de haut en bas**, dans l'ordre chronologique, et sont numérotés.
- ☐ Un appel synchrone est une flèche pleine à **tête pleine** ; un retour est une flèche en **pointillés** à tête ouverte.
- ☐ Chaque message synchrone reçu démarre un rectangle d'**activation** sur la ligne de vie.
- ☐ Chaque appel synchrone se termine par un message de retour (même implicite, il est apprécié de le dessiner).
- ☐ Aucun message ne « remonte le temps » : une flèche ne part jamais d'un point bas vers un point haut.

ATTENTION Les 5 erreurs les plus sanctionnées au Bac

1. Placer un acteur à l'intérieur du système. L'acteur est par définition externe : le guichetier de l'agence de voyage n'est pas un composant du logiciel, il l'utilise.
2. Inverser les flèches `include` et `extend`. Retiens : `include` va de la base vers l'inclus (la base tire l'inclusion) ; `extend` va de l'extension vers la base (l'extension s'accroche à la base).
3. Écrire un cas d'utilisation sans verbe. « Billet » n'est pas un cas ; « Réserver un billet » en est un. Un cas exprime une action rendue à un acteur.
4. Désordonner les messages d'un diagramme de séquence. Le temps descend : le message 3 ne peut pas être dessiné au-dessus du message 2.
5. Dessiner un retour en flèche pleine. Le retour est toujours en pointillés ; une flèche pleine signifie un nouvel appel synchrone bloquant, ce qui change totalement le sens du scénario.

6.5 Du langage courant au formalisme UML

L'énoncé du Bac est rédigé en français ; ta copie doit être en UML. Ce tableau de correspondance est ton dictionnaire de traduction : apprends à repérer les formulations qui « annoncent » chaque élément du formalisme.

Formulation dans l'énoncé	Élément UML correspondant
« le client demande la disponibilité et attend la réponse »	message synchrone (flèche pleine, tête pleine)
« le système envoie un SMS de confirmation sans attendre »	message asynchrone (flèche pleine, tête ouverte)
« le système retourne le numéro de réservation »	message de retour (flèche en pointillés)
« le client, l'agence ou le partenaire »	acteur (bonhomme, hors du système)
« le système doit permettre de réserver »	cas d'utilisation (ellipse, verbe à l'infinitif)
« toute réservation comprend obligatoirement un paiement »	relation <<include>>
« le client peut éventuellement souscrire une assurance »	relation <<extend>>
« un paiement peut être un paiement Mobile Money ou un paiement en espèces »	généralisation/spécialisation entre cas
« ensuite », « puis », « après vérification »	ordre chronologique des messages (haut → bas)

6.6 Entraînement type Bac : sujet complet corrigé

EXEMPLE Sujet type Bac — Agence de voyage « Express Voyage » (Yaoundé–Douala)

Énoncé. L'agence « Express Voyage » de Yaoundé souhaite informatiser la réservation de ses billets de bus sur la ligne Yaoundé–Douala. Le client se présente à l'agence ou se connecte en ligne ; il consulte les horaires et les tarifs des départs. S'il le souhaite, il réserve un billet : toute réservation exige obligatoirement un paiement, qui peut se faire par MTN Mobile Money ou en espèces au guichet. Le client peut, s'il le désire, souscrire une assurance voyage. Une fois le paiement validé, le système édite le billet et envoie un SMS de confirmation au client sans attendre d'accusé de réception. Le comptable de l'agence consulte chaque jour les statistiques de vente.

Questions. 1) Identifier les acteurs et tracer le diagramme de cas d'utilisation. 2) Tracer le diagramme de séquence du scénario nominal « Réserver un billet ».

EXERCICE RÉSOLU 6 Corrigé détaillé du sujet « Express Voyage »

Rappel de l'énoncé : voir le cadre ci-dessus.

SOLUTION

Étape 1 — Lecture active. Noms candidats acteurs : client, comptable. Verbes candidats cas : consulter, réserver, payer, souscrire, éditer, envoyer, consulter les statistiques.

Étape 2 — Acteurs et périmètre. Acteurs : Client et Comptable, tous deux externes. Le système est le logiciel de réservation de l'agence.

Étapes 3 et 4 — Cas et relations.

- Cas : « Consulter les horaires et tarifs », « Réserver un billet », « Payer », « Souscrire une assurance », « Éditer le billet », « Envoyer un SMS de confirmation », « Consulter les statistiques de vente ».
- « toute réservation exige obligatoirement un paiement » ⇒ Réserver un billet
 $\xrightarrow{\ll include \gg}$ Payer.
- « peut, s'il le désire » ⇒ Souscrire une assurance $\xrightarrow{\ll extend \gg}$ Réserver un billet.
- « par MTN Mobile Money ou en espèces » ⇒ Payer par Mobile Money et Payer en espèces héritent du cas général Payer (généralisation, triangle vide vers Payer).

Étape 5 — Scénario nominal « Réserver un billet ».

1. Le client consulte les horaires et choisit un départ.
2. Le client demande la réservation d'un billet.
3. Le système vérifie la disponibilité et retourne le montant.
4. Le client paie (par Mobile Money ou en espèces).

5. Le système valide le paiement, édite le billet.

6. Le système envoie le SMS de confirmation (sans bloquer).

Étape 6 — Diagramme de séquence. Objets : `c:Client`, `guichet:InterfaceReservation`, `ctrl:ControleurReservation`, `bd:BaseDeDonnees`. Messages numérotés de haut en bas :

1. 1: `demanderReservation(depart)` — synchrone, client → interface.
2. 2: `verifierDisponibilite(depart)` — synchrone, interface → contrôleur.
3. 3: `rechercherPlaces(depart)` — synchrone, contrôleur → base de données.
4. 4: `montant` — retour en pointillés vers le contrôleur, puis propagé jusqu'au client.
5. 5: `payer(montant)` — synchrone, client → interface.
6. 6: `editerBillet()` — synchrone, contrôleur → base de données.
7. 7: `envoyerSMS(numero)` — asynchrone (tête ouverte), car l'énoncé précise « sans attendre d'accusé ».

Piège évité : le SMS est asynchrone, et chaque retour est en pointillés.

EXERCICE 5 À toi de jouer — Cybercafé de Bafoussam

Un cybercafé informatise sa gestion. Le client peut consulter le tarif horaire, ouvrir une session de navigation et imprimer des documents. Toute ouverture de session exige obligatoirement l'achat d'un ticket de temps. Le client peut, en option, acheter une boisson. Le gérant consulte les recettes quotidiennes. Trace le diagramme de cas d'utilisation complet (acteurs, périmètre, relations) et le diagramme de séquence du scénario « Ouvrir une session ».

CORRIGÉ Exercice — Cybercafé de Bafoussam

Acteurs : Client, Gerant. **Cas :** « Consulter le tarif horaire », « Ouvrir une session », « Acheter un ticket de temps », « Imprimer des documents », « Acheter une boisson », « Consulter les recettes quotidiennes ». **Relations :** Ouvrir une session $\xrightarrow{\ll include \gg}$ Acheter un ticket de temps; Acheter une boisson $\xrightarrow{\ll extend \gg}$ Ouvrir une session. **Diagramme de séquence « Ouvrir une session » :** 1: `acheterTicket(duree)` (client → caisse, synchrone); 2: `validerTicket()` (caisse → gestionnaire de sessions); 3: retour `codeTicket` (pointillés); 4: `ouvrirSession(code)` (synchrone); 5: retour confirmation (pointillés). Vérifie : acteurs hors du rectangle, flèches orientées correctement, messages numérotés de haut en bas.

6.7 Complément utile au Bac (hors programme strict)

LESAVAIS-TU ? Un diagramme bien orienté rapporte des points, même partiel

Au Baccalauréat TI, le barème valorise la forme autant que le contenu : un diagramme de cas d'utilisation dont le périmètre est tracé, dont les acteurs sont correctement placés à l'extérieur et dont les flèches `<<include>>` et `<<extend>>` sont bien orientées et étiquetées rapporte des points même si la liste des cas est incomplète. Ne rends jamais une question de modélisation blanche : dessine le squelette (rectangle, acteurs, ellipses, flèches légendées), c'est déjà des points gagnés. À l'inverse, un diagramme riche mais aux flèches inversées est lourdement pénalisé, car il révèle une incompréhension du formalisme.

Articulation avec le diagramme de classes (complément hors programme). Tu as étudié la modélisation statique avec le diagramme de classes. Les deux approches se rejoignent naturellement : les **objets** qui apparaissent sur les lignes de vie d'un diagramme de séquence sont des instances des **classes** du diagramme de classes, et chaque **message** reçu par un objet correspond à une opération (méthode) déclarée dans sa classe. Ainsi, dans notre corrigé, le message `verifierDisponibilite(depart)` reçu par `ctrl:ControleurReservation` suppose que la classe `ControleurReservation` possède une opération du même nom. Cette cohérence entre vue statique et vue dynamique est exactement ce que vérifient les outils professionnels comme StarUML : si ton diagramme de séquence appelle une méthode qui n'existe pas dans la classe, le logiciel te le signale. Retenir ce pont entre les deux diagrammes, c'est comprendre qu'UML n'est pas un empilement de dessins, mais un langage cohérent où chaque vue éclaire les autres.

Tu es désormais armé pour toute épreuve de modélisation dynamique : méthode en 6 étapes, grilles de relecture, tableau de traduction du français vers UML, et un sujet type corrigé. Entraîne-toi régulièrement sur des situations camerounaises concrètes — réservation de transport, paiement Mobile Money, gestion scolaire — et le formalisme UML deviendra pour toi une seconde nature.

7 ACTIVITÉ D'INTÉGRATION

8 LEÇON 13 : MODÉLISATION DYNAMIQUE DES SYSTÈMES D'INFORMATION AVEC UML (CAS D'UTILISATION ET SÉQUENCE)

8.1 Objectifs de la leçon

À l'issue de cette leçon, tu seras capable de :

- identifier les **acteurs** d'un système d'information réel et délimiter son **périmètre** (Savoir : acteurs et périmètre) ;
- construire un **diagramme de cas d'utilisation** complet, comportant les relations <<include>>, <<extend>> et une **généralisation/spécialisation** (Savoir-faire : représenter un cas d'utilisation et ses relations) ;
- rédiger le **scénario nominal** d'un cas d'utilisation, avec ses scénarios alternatifs et d'exception (Savoir : scénarios d'un cas d'utilisation) ;
- tracer le **diagramme de séquence** correspondant, avec lignes de vie, objets, messages numérotés et types de messages corrects (synchrone, retour, asynchrone, création, destruction) (Savoir : composants d'un diagramme de séquence, types de messages ; Savoir-faire : représenter un diagramme de séquence) ;
- reproduire les deux diagrammes dans un **logiciel professionnel** de modélisation (ArgoUML, StarUML ou WinDesign) et exporter le résultat (Savoir-faire : produire un diagramme à l'aide d'un logiciel professionnel ; Savoir : logiciels professionnels).

ATTENTION Organisation du travail

Cette leçon se déroule sous la forme d'une activité d'intégration en groupes de 4 élèves, en trois temps : une phase analyse et conception sur papier (2 h), une phase réalisation logicielle en salle informatique (1 h). Durée totale indicative : 3 heures. Chaque groupe désigne un rapporteur pour la correction croisée finale.

8.2 Situation professionnelle : Informatisation de la cantine du Lycée Bilingue de Nkolbisson

Le Lycée Bilingue de Nkolbisson compte 2400 élèves. Sa cantine, gérée jusque-là de façon manuscrite, connaît des difficultés croissantes : files d'attente interminables à la caisse, erreurs dans la caisse (environ 15000 FCFA d'écarts par semaine selon le compte de gestion), repas préparés « au hasard » (certains jours, 200 repas en trop ; d'autres jours, 150 élèves restent sans repas), et aucune traçabilité pour les parents qui paient par Mobile Money.

Le proviseur décide donc d'informatiser la cantine et confie l'étude préalable à la classe de Terminale TI. Voici le fonctionnement souhaité, tel que décrit par le proviseur et l'intendant :

- Un ticket de repas coûte **500 FCFA**. L'**élève** achète ses tickets à la caisse, soit en **espèces**, soit par **MTN Mobile Money** ou **Orange Money** (au numéro marchand de l'établissement).
- Pour tout achat, le **caissier** saisit la vente dans le système : celui-ci doit systématiquement **vérifier le paiement** (comptage des espèces remises, ou interrogation automatique de l'API de l'opérateur mobile pour confirmer la transaction).
- Si le paiement est validé, le système enregistre la vente et imprime (ou affiche) un ticket numéroté.
- Dans le cas d'un paiement Mobile Money, le système peut, si le parent a souscrit à l'option, **envoyer un SMS de confirmation** au parent (montant, date, solde de tickets restants).
- Chaque jour à **10 h 30**, le **cuisinier** consulte dans le système le nombre de tickets vendus pour la journée afin d'adapter la quantité de repas à préparer.
- L'**intendant** consulte chaque semaine les statistiques de vente (recettes totales, répartition espèces/Mobile Money) pour le compte de gestion.

Données chiffrées utiles : en moyenne, 1800 tickets sont vendus par jour, dont environ 60 % en espèces et 40 % en Mobile Money. Le système doit fonctionner sur le réseau local du lycée, avec une base de données hébergée sur le serveur de la salle informatique.

Mission du groupe : produire la modélisation dynamique du futur système d'information de la cantine : diagramme de cas d'utilisation (avec relations) et diagramme de séquence du cas principal, d'abord sur papier, puis sous ArgoUML ou StarUML.

8.3 Consignes de l'activité

Partie A — Analyse (sur papier, 30 min)

1. À partir de la description du proviseur, dresse la liste de tous les **acteurs** du système. Pour chacun, précise s'il est un acteur principal (il initie l'interaction avec le système pour atteindre un objectif) ou secondaire (le système initie l'interaction vers lui, ex. l'API de l'opérateur mobile).
2. Délimite le **périmètre** du système : qu'est-ce qui est à l'intérieur du système (fonctions automatisées) et qu'est-ce qui reste à l'extérieur ? Justifie en deux phrases pourquoi « préparer les repas » ne constitue pas un cas d'utilisation du système.
3. Recense les **cas d'utilisation** du système et associe chacun à son (ses) acteur(s) initiateur(s).

Partie B — Diagramme de cas d'utilisation (sur papier, 45 min)

4. Construis le diagramme de cas d'utilisation complet : cadre du système (rectangle), acteurs (bonhommes ou rectangles stéréotypés <<actor>> pour systèmes externes), ellipses (cas d'utilisation), associations (traits pleins).
5. Ajoute une relation <<include>> entre « Acheter un ticket » et « Vérifier le paiement ». Justifie ce choix plutôt que <<extend>> en te référant à la définition formelle.

6. Ajoute une relation <<extend>> entre « Envoyer un SMS de confirmation » et « Acheter un ticket ». Justifie ce choix et précise la condition d'extension.
7. Modélise par une **généralisation/spécialisation** le fait que « Payer en espèces » et « Payer par Mobile Money » sont deux spécialisations de « Payer ». Représente la flèche à triangle blanc du cas fils vers le cas père.

Partie C — Scénario et diagramme de séquence (sur papier, 45 min)

8. Rédige le **scénario nominal** du cas « Acheter un ticket » (paiement Mobile Money), sous forme d'étapes numérotées alternant actions de l'acteur et réponses du système. Ajoute au moins un scénario alternatif (paiement refusé) et un scénario d'exception (panne réseau API).
9. Trace le **diagramme de séquence** correspondant : lignes de vie (Élève, Caissier, Système, API Mobile Money), barres d'activation (foyers de contrôle), messages numérotés dans l'ordre chronologique (de haut en bas), en distinguant messages synchrones (flèche pleine, tête pleine), messages de retour (flèche pointillée, tête ouverte), message asynchrone (flèche pleine, tête ouverte), et message de création le cas échéant.

Partie D — Réalisation logicielle et correction croisée (salle informatique, 60 min)

10. Reproduis les deux diagrammes sous **ArgoUML**, **StarUML** ou **WinDesign**. Vérifie que l'outil impose la syntaxe UML (ex. sens des flèches include/extend). Exporte chaque diagramme en image PNG.
11. Échange ta production avec un autre groupe et évalue-la à l'aide de la **grille de relecture** ci-dessous. Rédige un court compte rendu des erreurs relevées et corrige ton propre diagramme en conséquence.

À RETENIR**Grille de relecture croisée (à utiliser en Partie D)**

Critère	Conforme	Commentaire
	(Oui/-Non)	Correc-tion
Acteurs principaux et secondaires identifiés et bien placés (hors cadre)		
Périmètre du système délimité (cadre rectangle + nom)		
Cas d'utilisation nommés verbe à l'infinitif + complément		
Relation <<include>> : flèche pointillée base → inclus, stéréotype visible		
Relation <<extend>> : flèche pointillée extension → base, stéréotype + condition		
Généralisation : trait plein, triangle blanc du fils vers le père		
Diagramme de séquence : lignes de vie (pointillés), barres d'activation		
Messages numérotés chronologiquement (haut → bas)		
Types de messages respectés (synchrone, retour, asynchrone)		
Cohérence avec le scénario nominal (mêmes étapes, mêmes acteurs)		

8.4 Ressources à mobiliser : Rappels essentiels

À RETENIR Rappels essentiels pour l'activité

- **Acteur** : entité externe (personne, organisation ou système) qui interagit avec le système. Représentation : bonhomme (acteur humain) ou rectangle <<actor>> (système externe). Un acteur est toujours hors du cadre du système.
- **Cas d'utilisation (Use Case)** : description d'une séquence d'actions qu'un système effectue pour produire un résultat observable de valeur pour un acteur. Nommage : verbe à l'infinitif + complément d'objet (ex. « Acheter un ticket », pas « Achat » ni « Ticket »).
- **Relation <<include>>** : le cas de base contient toujours le cas inclus (exécution obligatoire). Notation : flèche en pointillés du cas de base vers le cas inclus, stéréotype <<include>>. Factorise un comportement commun à plusieurs cas de base.
- **Relation <<extend>>** : le cas d'extension s'exécute parfois, sous condition (point d'extension). Notation : flèche en pointillés de l'extension vers le cas de base, stéréotype <<extend>>, condition notée en note ou sur la flèche.
- **Généralisation / Spécialisation** : factorisation de comportements communs. Le cas fils (spécialisation) hérite du cas père (généralisation) et peut ajouter/redéfinir des comportements. Notation : trait plein avec triangle blanc (non rempli) du cas fils vers

le cas père.

- Diagramme de séquence (Sequence Diagram) : vue temporelle (verticale : temps ↓) d'un scénario.
 - Ligne de vie (Lifeline) : participant (acteur ou objet) représenté par un rectangle (tête) + ligne pointillée verticale.
 - Barre d'activation (Execution Specification / Focus of Control) : rectangle fin sur la ligne de vie indiquant la période où l'objet exécute une opération.
 - Message synchrone (Appel d'opération) : flèche pleine, tête pleine (triangle rempli). L'émetteur attend la fin de l'exécution.
 - Message de retour (Reply) : flèche pointillée, tête ouverte (bâton ou triangle ouvert). Retour de contrôle/valeur.
 - Message asynchrone (Signal / Appel non bloquant) : flèche pleine, tête ouverte (triangle ouvert). L'émetteur continue sans attendre.
 - Message de création / destruction : stéréotypes <<create>> / <<destroy>>, flèche vers la tête ou une croix (X) en bas de la ligne de vie.
- Logiciels professionnels : ArgoUML (Java, open source), StarUML (multi-plateforme, freemium), WinDesign (Merise/UML, historique en établissement camerounais). Ils valident la syntaxe UML à la création des liens.

8.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 7 Corrigé commenté de l'activité d'intégration

Étape 1 — Acteurs et périmètre.

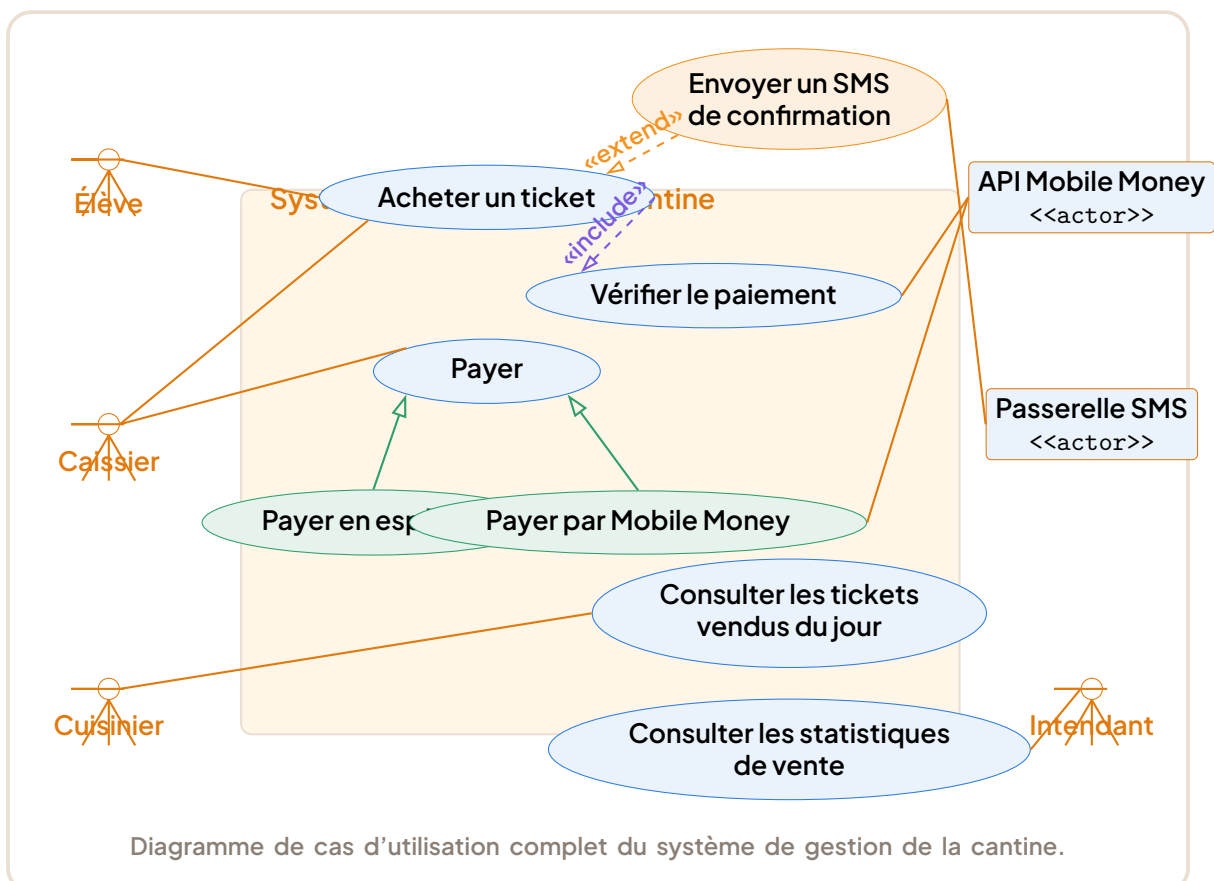
- Acteurs principaux (initient l'interaction) : Élève (achète tickets), Caissier (saisit ventes), Cuisinier (consulte ventes jour), Intendant (consulte statistiques).
- Acteurs secondaires (systèmes externes sollicités) : API Mobile Money (MTN MoMo / Orange Money) — interrogée pour vérifier la transaction ; Passerelle SMS — utilisée pour l'envoi de confirmation (optionnel).
- Périmètre (Système « Gestion Cantine ») : Automatisation de la vente (saisie, vérification paiement, édition ticket), notification SMS, consultation ventes (cuisinier), consultation statistiques (intendant), gestion base de données locale.
- Hors périmètre : Préparation physique des repas (activité humaine du cuisinier), gestion des stocks alimentaires, comptabilité générale de l'établissement, infrastructure réseau/serveur.
- Justification : « Préparer les repas » est une action humaine et physique réalisée par le cuisinier après avoir consulté l'information fournie par le système. Le système ne

pilote pas les casseroles ; il ne fait que fournir le chiffre (nombre de tickets vendus) qui aide à la décision.

Étape 2 — Cas d'utilisation identifiés.

1. Acheter un ticket (acteurs : Élève, Caissier) — cas principal.
2. Vérifier le paiement (inclus dans Acheter un ticket).
3. Payer (généralisation) — spécialisé en :
 - Payer en espèces (Caissier).
 - Payer par Mobile Money (Caissier, API Mobile Money).
4. Envoyer un SMS de confirmation (extension de Acheter un ticket, condition : paiement MoMo et parent abonné).
5. Consulter les tickets vendus du jour (acteur : Cuisinier).
6. Consulter les statistiques de vente (acteur : Intendant).

Étape 3 — Diagramme de cas d'utilisation (version complète).



Étape 4 — Justification rigoureuse des relations.

PROPRIÉTÉ Choix Include vs Extend vs Généralisation

- `<<include>>` (Acheter → Vérifier) : La vérification est obligatoire pour tout achat (espèces ou MoMo). C'est un fragment de comportement commun, factorisé pour éviter la duplication. La flèche part du cas utilisateur (base) vers le cas utilisé (inclus).
- `<<extend>>` (Envoyer SMS → Acheter) : L'envoi de SMS est conditionnel : il ne se produit que si (paiement = Mobile Money) ET (parent abonné à l'option). C'est une extension optionnelle à un point d'extension (après validation paiement). La flèche part de l'extension vers le cas de base.
- Généralisation (Payer ← Payer en espèces, Payer par Mobile Money) : Les deux modes de paiement partagent une structure commune (montant, date, référence) mais diffèrent dans la réalisation (comptage vs appel API). Le cas père « Payer » est abstrait (jamais instancié directement). Flèche trait plein, triangle blanc du fils vers le père.

Étape 5 — Scénario nominal et alternatifs du cas « Acheter un ticket » (Mobile Money).

MÉTHODE Rédaction d'un scénario structuré

1. Numéroté les étapes séquentiellement.
2. Alterné Acteur (initiative) et Système (réponse).
3. Préciser les données échangées (entrées/sorties).
4. Indiquer les inclusions (`<<include>>`) et extensions (`<<extend>>`) aux étapes concernées.
5. Lister les scénarios alternatifs (chemin alternatif nominal) et d'exception (erreur technique) séparément, en référençant l'étape de branchement.

EXEMPLE Scénario nominal — Acheter un ticket (Mobile Money)

1. Élève : Présente sa carte d'élève et demande n tickets au caissier.
2. Caissier : Saisit la vente dans le système (n tickets, montant = $n \times 500$ FCFA, mode « Mobile Money »), transmet la référence de transaction fournie par l'élève.
3. Système : <<include>> Déclenche « Vérifier le paiement » : interroge l'API Mobile Money avec la référence de transaction et le montant attendu.
4. API Mobile Money : Retourne `transactionConfirmée` (statut SUCCESS, identifiant transaction, horodatage).
5. Système : Enregistre la vente en base (ticket(s) numérotés, date, mode, caissier), met à jour le solde de tickets de l'élève.
6. Système : Édite/affecte les tickets numérotés (impression ou affichage écran).
7. Caissier : Remet les tickets à l'élève.
8. Système : <<extend>> Si (parent abonné option SMS) ALORS envoie SMS de confirmation asynchrone au parent (montant, date, solde restant).

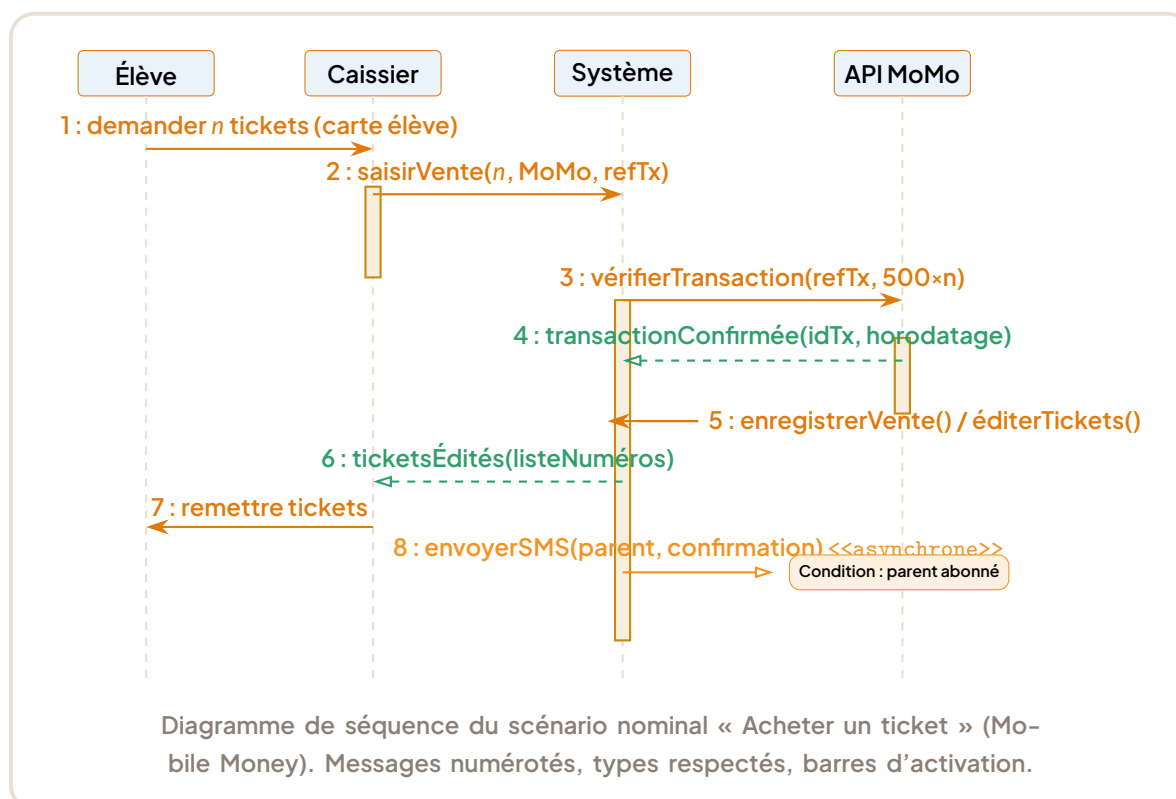
EXEMPLE Scénario alternatif 3a — Paiement refusé / Transaction invalide

3. API Mobile Money : Retourne `transactionRefusée` (statut FAILED, code erreur : solde insuffisant / référence invalide / délai dépassé).
4. Système : Affiche message d'erreur explicite au caissier (« Paiement non confirmé : solde insuffisant »).
5. Caissier : Informe l'élève, annule la saisie, propose paiement espèces ou nouvelle tentative.
6. Fin du cas d'utilisation (aucun ticket émis, aucune écriture en base).

EXEMPLE Scénario d'exception 3b — Panne réseau / API indisponible

3. Système : Timeout ou exception réseau lors de l'appel API (pas de réponse après 10 s).
4. Système : Affiche alerte critique « Service Mobile Money indisponible ».
5. Caissier : Bascule en mode dégradé : propose paiement espèces uniquement, note la vente en attente de régularisation (procédure papier de secours).
6. Fin du cas d'utilisation (pour le mode MoMo).

Étape 6 — Diagramme de séquence (scénario nominal Mobile Money).



Étape 7 — Réalisation logicielle (StarUML / ArgoUML / WinDesign).

MÉTHODE Procédure StarUML (exemple)

1. File > New Project > UML Project.
2. Diagramme de cas d'utilisation : Model > Add Diagram > Use Case Diagram.
 - Palette : Actor (bonhomme), Use Case (ellipse), System Boundary (rectangle).
 - Relations : Association (trait plein), Include (flèche pointillée base→inclus), Extend (flèche pointillée extension→base + propriété condition), Generalization (trait plein triangle blanc fils→père).
3. Diagramme de séquence : Model > Add Diagram > Sequence Diagram.
 - Palette : Lifeline (acteur/objet), Message (synchrone, flèche pleine), Reply Message (retour, flèche pointillée), Async Message (asynchrone, flèche tête ouverte), Execution Specification (barre d'activation), Create Message / Destroy Message.
 - Numérotation : propriété sequenceNumber (auto ou manuel).
4. Export : File > Export Diagram As Image... > PNG. Vérifier l'affichage des stéréotypes <<include>> / <<extend>> sur les connecteurs.

8.6 Erreurs fréquentes et pièges de l'examen (Attention!)**ATTENTION** Erreurs classiques relevées en correction croisée et au Baccalauréat

- Sens des flèches Include/Extend inversé : <<include>> part du cas de base vers le cas inclus ; <<extend>> part de l'extension vers le cas de base. C'est l'erreur n°1 au Bac. Mnémotechnique : « J'inclus dedans (flèche rentre), j'étends dehors (flèche sort) ».
- Nommage incorrect des cas d'utilisation : Noms communs (« La caisse », « Le SMS », « Gestion ») au lieu de verbe à l'infinitif + complément (« Enregistrer la vente », « Envoyer un SMS », « Gérer les tickets »).
- Acteurs à l'intérieur du cadre système ou acteurs « Base de données », « Serveur » dessinés comme acteurs principaux. La BD et le serveur sont des ressources internes, pas des acteurs externes.
- Confusion Généralisation / Include : La généralisation structure une hiérarchie de cas (factorisation « est-un »), l'include factorise un fragment de comportement commun (« a-besoin-de »).

- Diagramme de séquence :
 - Messages non numérotés ou numérotation non chronologique (le temps va strictement de haut en bas).
 - Types de messages mélangés : retour dessiné en flèche pleine, asynchrone en flèche fermée.
 - Absence de barres d'activation (foyers de contrôle) : on ne voit pas qui fait quoi et pendant combien de temps.
 - Lifelines d'acteurs humains (Élève, Caissier) qui envoient des messages techniques (SQL, API) : l'acteur humain parle au Système (interface), le Système parle à l'API.
- Condition d'extension manquante sur la flèche <<extend>> (ex. « [paiement MoMo && parent abonné] »).
- Triangle de généralisation rempli (noir) au lieu de blanc (creux). UML : triangle blanc = généralisation ; triangle noir = réalisation d'interface (pas au programme Terminale TI).

8.7 Bilan et transfert vers l'examen

Cette activité d'intégration a mobilisé l'ensemble des compétences de la Famille de situations : « Analyse et conception d'un système ; présentation du comportement dynamique ».

À RETENIR Ce qu'il faut retenir pour le Bac

- Analyse préalable : Lister acteurs (principal/secondaire) et périmètre avant de dessiner. C'est la clé pour ne pas mettre la BD en acteur.
- Relations UC : <<include>> = toujours (obligatoire, factorisation) ; <<extend>> = parfois (conditionnel, optionnel) ; Généralisation = est-un (héritage de structure/comportement).
- Cohérence UC / Séquence : Le diagramme de séquence détaille un seul cas d'utilisation (un scénario). Chaque message = une étape du scénario. La numérotation est le lien direct.
- Types de messages : Synchrone (appel bloquant, flèche pleine), Retour (flèche pointillée), Asynchrone (signal, flèche tête ouverte), Création/Destruction (stéréotypes).
- Outils pros : ArgoUML, StarUML, WinDesign ne sont pas des « paint » : ils valident la syntaxe (ex. refusent un include inversé). Les maîtriser est un savoir-faire évalué.
- Contexte camerounais : Intégration Mobile Money (MTN/Orange API), SMS, gestion cantine/école, réseau local serveur salle info. Ces exemples sont types de sujets d'examen.

EXERCICE 6 Entraînement Bac — Sujet type

Sujet : On souhaite modéliser le système de « Réservation de places d'examen en ligne » pour l'Office du Baccalauréat du Cameroun (OBC).

1. Identifier les acteurs (Candidat, Surveillant, OBC, API Paiement, Serveur Mail).
2. Lister les cas d'utilisation : « S'inscrire », « Payer les frais » (Espèces, Mobile Money, Carte), « Choisir centre », « Imprimer convocation », « Valider présence » (Surveillant), « Publier résultats » (OBC).
3. Proposer les relations : Include (Payer dans S'inscrire), Extend (Imprimer convocation après Paiement validé), Généralisation (Payer).
4. Écrire le scénario nominal de « S'inscrire » (paiement Mobile Money).
5. Tracer le diagramme de séquence correspondant (Candidat, Système, API Paiement, Serveur Mail).

CORRIGÉ Exercice Entraînement Bac

1. Acteurs : Principaux : Candidat, Surveillant, Administrateur OBC. Secondaires : API Paiement (MTN/Orange/Banque), Serveur Mail (SMTP). 2. Cas d'utilisation : « S'inscrire à l'examen », « Payer les frais », « Choisir son centre », « Imprimer convocation », « Valider présence (appel) », « Publier résultats », « Consulter statistiques ». 3. Relations :

- <<include>> : « S'inscrire » → « Payer les frais » (paiement obligatoire pour valider inscription).
- <<extend>> : « Imprimer convocation » → « S'inscrire » (condition : inscription validée && paiement confirmé).
- Généralisation : « Payer les frais » (père) ← « Payer en espèces », « Payer par Mobile Money », « Payer par Carte bancaire » (fils).

4. Scénario nominal « S'inscrire » (Mobile Money) : 1. Candidat remplit formulaire (nom, série, centre souhaité). 2. Système vérifie places dispo au centre. 3. Candidat choisit Mobile Money, reçoit référence. 4. Système <<include>> appelle API Paiement. 5. API confirme. 6. Système enregistre inscription, génère n° candidat. 7. Système <<extend>> déclenche envoi Mail convocation (asynchrone). 8. Candidat reçoit Mail. 5. Diagramme de séquence : Lifelines : Candidat, Système, API Paiement, Serveur Mail. Messages numérotés 1 à 8. Types : 1,2,3,4,6,7 synchrone ; 5 retour ; 8 asynchrone (vers Serveur Mail). Barres d'activation sur Système (longue), API Paiement (courte), Serveur Mail (courte).

LESAVAIS-TU ? Le Cameroun numérique et l'UML

L'Agence Nationale des Technologies de l'Information et de la Communication (ANTIC) recommande l'utilisation de normes ouvertes (UML, Merise) pour les marchés publics de développement logiciel. Les grands projets camerounais (plateforme `impots.cm` de la DGI, système de gestion des concours CAMPOST/MINFOPRA, interopérabilité CENADI) imposent une phase de modélisation UML (cas d'utilisation, séquence, classes) avant tout codage. Maîtriser ces diagrammes, c'est accéder aux métiers d'Analyste Programmeur, d'Architecte Logiciel ou de Consultant SI au Cameroun et à l'international. Les logiciels ArgoUML (gratuit) et StarUML (standard industriel) sont installés dans les salles informatiques des lycées techniques camerounais pour cet apprentissage.

9 MÉTHODES ET EXERCICES RÉSOLUS

9.1 Méthode 1 : Identifier les acteurs et délimiter le périmètre du système

DÉFINITION Acteur et périmètre

Un **acteur** est un rôle joué par une entité externe au système (personne, autre système informatique, dispositif matériel, horloge) qui interagit directement avec lui. Le **périmètre** (ou frontière) du système délimite ce que le système réalise (à l'intérieur du rectangle) de ce qui reste à l'extérieur. Un **cas d'utilisation** est un ensemble d'actions réalisées par le système et produisant un résultat observable et utile pour un acteur ; il se nomme par un verbe à l'infinitif.

MÉTHODE Identifier les acteurs et le périmètre d'un système

1. Lire l'énoncé en soulignant les verbes d'action : chaque verbe (« consulter », « payer », « enregistrer ») est un candidat cas d'utilisation ; chaque sujet de ces verbes est un candidat acteur.
2. Distinguer acteur et utilisateur interne : un acteur est un rôle externe au système qui interagit avec lui (personne, autre système, horloge). Un programme interne au système n'est **jamais** un acteur.
3. Repérer les acteurs secondaires : systèmes externes sollicités par le système (serveur de paiement MTN MoMo, serveur d'impression, base distante). Ils se placent à **droite** du diagramme, les acteurs principaux à gauche.
4. Tracer le rectangle du système : il matérialise le **périmètre** (frontière). Tout ce qui est dans le rectangle est réalisé par le système ; tout ce qui est dehors ne l'est pas.
5. Vérifier la cohérence : chaque acteur doit être relié à au moins un cas d'utilisation, et chaque cas d'utilisation doit être déclenché par au moins un acteur. Un cas « orphelin » est une erreur de modélisation.
6. Rédiger la réponse au Bac : présenter les acteurs dans un tableau (acteur / rôle / type principal ou secondaire) avant de dessiner.

Réflexe : se poser la question « qui appuie sur le bouton ? » pour chaque cas d'utilisation.

EXERCICE RÉSOLU 8 Acteurs et périmètre d'une plateforme de paiement scolaire

Le lycée de Nkol-Eton veut informatiser le paiement des frais de scolarité. Les parents

paient via MTN Mobile Money ; le comptable consulte les paiements et édite les quittances ; le système interroge la plateforme MTN pour valider chaque transaction. Identifier les acteurs (en précisant leur type) et délimiter le périmètre du système.

SOLUTION

Étape 1 — Extraction des verbes et des sujets. Verbes : « paient » (parents), « consulte », « édite » (comptable), « interroge/valide » (système vers MTN).

Étape 2 — Classement des acteurs.

Acteur	Rôle	Type
Parent	Initie le paiement des frais via son téléphone	Principal
Comptable	Consulte les paiements, édite les quittances	Principal
Plateforme MTN MoMo	Valide les transactions (système externe)	Secondaire

Étape 3 — Périmètre. Le rectangle « Système de paiement des frais » contient les cas : Payer les frais, Consulter les paiements, Éditer une quittance, Valider la transaction. La plateforme MTN reste à l'extérieur : elle n'est pas développée par le lycée, seulement sollicitée.

Conclusion : le système compte 2 acteurs principaux (Parent, Comptable) et 1 acteur secondaire (Plateforme MTN MoMo) ; le périmètre exclut la plateforme MTN, qui est un système externe avec lequel on communique.

9.2 Méthode 2 : Construire un diagramme de cas d'utilisation

MÉTHODE**Tracer un diagramme de cas d'utilisation conforme UML**

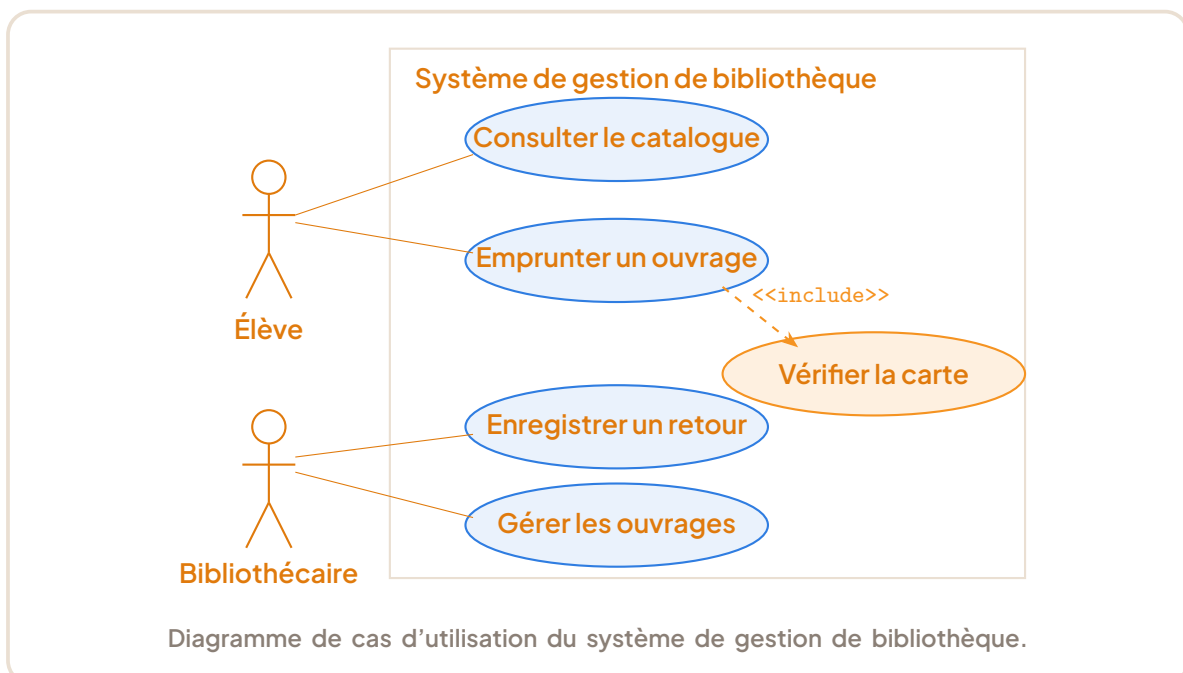
1. Dessiner le rectangle du système et écrire son nom en haut.
2. Placer les acteurs : bonhommes (stickmen) à gauche pour les principaux, à droite pour les secondaires ; un acteur non humain peut être représenté par un rectangle stéréotypé `<<system>>`.
3. Dessiner chaque cas d'utilisation dans une **ellipse**, avec un nom commençant par un verbe à l'infinitif (« Payer les frais », jamais « Paiement »).
4. Relier acteurs et cas par des traits pleins (associations), sans flèche en général.
5. Ajouter les relations entre cas si nécessaire : `<<include>>` et `<<extend>>` en flèches pointillées, généralisation en flèche pleine à triangle vide.
6. Relire : aucune ellipse hors du rectangle, aucun acteur à l'intérieur, aucun lien direct entre deux acteurs.

EXERCICE RÉSOLU 9 Diagramme de cas d'utilisation d'une bibliothèque scolaire

Une bibliothèque permet à l'élève de consulter le catalogue et d'emprunter un ouvrage. Le bibliothécaire peut enregistrer un retour et gérer les ouvrages. Tout emprunt exige de vérifier la carte de l'élève. Représenter le diagramme de cas d'utilisation.

SOLUTION

Analyse : 2 acteurs principaux (Élève, Bibliothécaire), 5 cas d'utilisation. « Vérifier la carte » est systématiquement exécutée lors d'un emprunt : relation `<<include>>` de « Emprunter un ouvrage » vers « Vérifier la carte de l'élève ».



Justifications : « Vérifier la carte » est obligatoire à chaque emprunt, donc `<<include>>` (et non `<<extend>>`, qui est optionnel). La flèche pointillée part du cas de base (Emprunter) vers le cas inclus (Vérifier). « Consulter le catalogue » ne dépend d'aucun autre cas : simple association.

Conclusion : le diagramme comporte 2 acteurs, 5 cas d'utilisation, 4 associations et 1 relation d'inclusion correctement orientée.

9.3 Méthode 3 : Choisir et représenter les relations entre cas d'utilisation**PROPRIÉTÉ** Les trois relations entre cas d'utilisation

- Inclusion `<<include>>` : le cas de base A exécute obligatoirement le cas inclus B à chaque exécution. Flèche pointillée de A vers B.

- **Extension** <<extend>> : le cas d'extension B s'exécute optionnellement, sous condition, pendant le cas de base A. Flèche pointillée de B vers A.
- **Généralisation/spécialisation** : un cas enfant hérite du comportement du cas parent et le spécialise. Flèche pleine à triangle vide, de l'enfant vers le parent. La même relation existe entre acteurs (ex. « Élève » et « Enseignant » spécialisent « Utilisateur »).

MÉTHODE Choisir entre include, extend et généralisation

1. **Test de l'inclusion** <<include>> : le cas B est-il exécuté systématiquement (obligatoirement) quand A s'exécute? Si oui : flèche pointillée de A vers B, étiquetée <<include>>. Sert aussi à factoriser un comportement commun à plusieurs cas.
2. **Test de l'extension** <<extend>> : le cas B est-il optionnel, déclenché seulement sous une condition (ex. « si le stock est épuisé »)? Si oui : flèche pointillée de B (l'extension) vers A (le cas de base), étiquetée <<extend>>.
3. **Test de la généralisation** : plusieurs cas (ou acteurs) sont-ils des variantes d'un même concept (« Payer en espèces » et « Payer par MoMo » sont des spécialisations de « Payer »)? Si oui : flèche pleine à triangle vide, de l'enfant vers le parent.
4. **Mémoriser les sens des flèches** : include : base → inclus ; extend : extension → base ; généralisation : enfant → parent.
5. **Justifier par écrit au Bac** : une relation non justifiée en une phrase perd des points.

EXERCICE RÉSOLU 10 Relations dans une application de vente en ligne

Sur la plateforme d'une startup doualaïse, un client peut Passer une commande. Passer une commande implique toujours de S'authentifier. Si le client le souhaite, il peut Appliquer un code promo pendant la commande. Le client peut Payer la commande, soit par Payer par Orange Money, soit par Payer par carte bancaire. Indiquer, en justifiant, les relations entre ces cas et les représenter schématiquement.

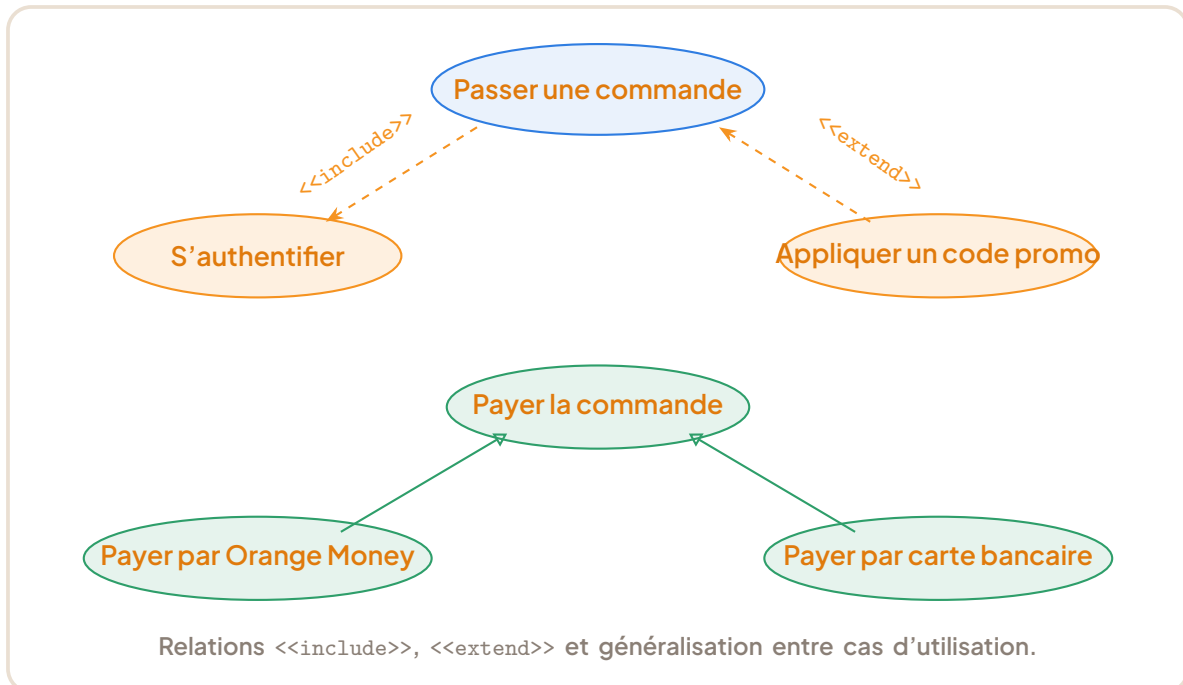
SOLUTION

Relation 1 — Passer une commande / S'authentifier. L'authentification est obligatoire à chaque commande : relation <<include>>, flèche pointillée de « Passer une commande » vers « S'authentifier ».

Relation 2 — Appliquer un code promo / Passer une commande. Le code promo est optionnel (« si le client le souhaite ») : relation <<extend>>, flèche pointillée de « Appliquer un code promo » (extension) vers « Passer une commande » (cas de base).

Relation 3 — Paiements. « Payer par Orange Money » et « Payer par carte bancaire » sont

deux variantes de « Payer la commande » : généralisation/spécialisation, flèches pleines à triangle vide des deux cas fils vers le cas parent.



Conclusion : une inclusion (obligatoire), une extension (optionnelle) et une généralisation à deux spécialisations modélisent fidèlement l'énoncé ; les sens des flèches sont déterminants et doivent être justifiés.

9.4 Méthode 4 : Construire un diagramme de séquence

DÉFINITION Diagramme de séquence

Le **diagramme de séquence** est un diagramme comportemental UML qui représente les interactions entre acteurs et objets du système dans le temps, pour un scénario donné d'un cas d'utilisation. Ses composants sont : les **participants** (acteurs et objets), les **lignes de vie** (traits verticaux en pointillés, le temps s'écoulant de haut en bas), les **messages** échangés et les **barres d'activation** (périodes pendant lesquelles un objet exécute un traitement).

MÉTHODE Tracer un diagramme de séquence conforme UML

1. Identifier les participants (acteur + objets du système) et les placer en haut, dans l'ordre chronologique d'intervention (de gauche à droite).

2. Tracer les lignes de vie : traits verticaux en pointillés sous chaque participant. Le **temps s'écoule de haut en bas**.
3. Nommer les objets dans des rectangles selon la syntaxe `nomObjet : NomClasse` (souligné en UML strict) ou `:NomClasse` si anonyme.
4. Placer les messages dans l'ordre chronologique : flèche pleine à pointe pleine pour un message synchrone (appel avec attente de réponse), flèche à pointe ouverte pour un message asynchrone, flèche pointillée de retour pour la réponse.
5. Dessiner les barres d'activation (rectangles fins sur les lignes de vie) pendant qu'un objet exécute un traitement.
6. Ajouter les fragments combinés si nécessaire : `alt` (alternative), `opt` (option), `loop` (boucle), avec leur garde entre crochets.
7. Vérifier : chaque message part d'une ligne de vie et arrive sur une autre (ou la même pour un auto-message) ; l'ordre vertical respecte le scénario.

EXERCICE RÉSOLU 11**Diagramme de séquence d'un paiement Mobile Money**

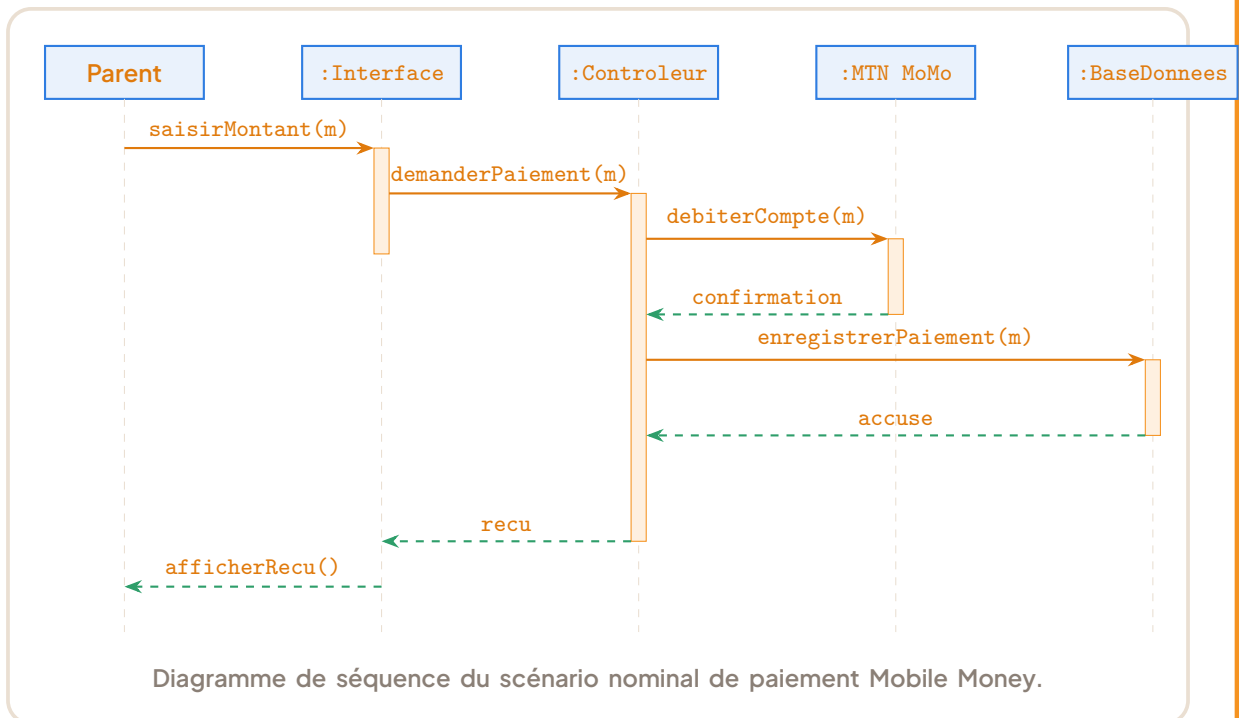
Scénario nominal : le Parent saisit le montant sur l'interface du système de paiement ; le système envoie une demande de débit à la plateforme MTN MoMo ; celle-ci valide et retourne une confirmation ; le système enregistre le paiement dans la base de données puis affiche le reçu au Parent. Représenter le diagramme de séquence.

SOLUTION

Participants (ordre d'apparition) : Parent (acteur), :InterfacePaiement, :ContrôleurPaiement, :PlateformeMTN, :BaseDeDonnees.

Chronologie des messages :

1. Parent → Interface : `saisirMontant(m)` (synchrone)
2. Interface → Contrôleur : `demanderPaiement(m)` (synchrone)
3. Contrôleur → MTN : `debiterCompte(m)` (synchrone)
4. MTN → Contrôleur : `confirmation` (retour, pointillé)
5. Contrôleur → BD : `enregistrerPaiement(m)` (synchrone)
6. BD → Contrôleur : `accusé` (retour, pointillé)
7. Contrôleur → Interface : `recu` (retour, pointillé)
8. Interface → Parent : `afficherRecu()` (retour, pointillé)



Justifications : les appels sont synchrones (flèches pleines) car chaque émetteur attend la réponse avant de continuer ; les réponses sont en pointillés (retours). Les barres d'activation montrent que le Contrôleur reste actif pendant tout l'échange.

Conclusion : le diagramme comporte 5 participants, 4 messages synchrones et 4 messages de retour, ordonnés verticalement selon le scénario nominal.

9.5 Méthode 5 : Modéliser les scénarios complexes (alt, opt, loop)

PROPRIÉTÉ Types de messages et fragments combinés

Types de messages : synchrone (flèche pleine à pointe pleine : l'émetteur se bloque en attendant la réponse), asynchrone (flèche à pointe ouverte : l'émetteur continue sans attendre), retour (flèche pointillée), auto-message (flèche revenant sur la même ligne de vie), création *create* (flèche pointillée vers la tête d'un objet nouvellement créé).
Fragments combinés : rectangle à cinq pans portant l'opérateur en haut à gauche — alt (alternative exclusive, opérandes séparés par une ligne pointillée, chacun gardé par [condition]), opt (option, un seul opérande gardé), loop (boucle, garde d'itération).

MÉTHODE Utiliser les fragments combinés et les types de messages

1. Repérer dans l'énoncé les mots déclencheurs : « si... sinon » ⇒ fragment `alt` avec deux opérandes et des gardes `[condition]` ; « éventuellement / le cas échéant » ⇒ `opt` ; « pour chaque / tant que » ⇒ `loop`.
2. Encadrer la zone concernée par un rectangle à cinq pans portant l'opérateur en haut à gauche ; séparer les opérandes d'un `alt` par une ligne pointillée horizontale.
3. Choisir le bon type de message : synchrone (pointe pleine, l'émetteur bloque en attente), asynchrone (pointe ouverte, l'émetteur continue), retour (pointillé), auto-message (flèche qui revient sur la même ligne de vie), message de création `create` (pointillé vers la tête d'un objet).
4. Ne pas oublier le scénario alternatif : au Bac, un énoncé avec « en cas d'échec » exige un `alt` avec la branche d'erreur (ex. solde insuffisant).

EXERCICE RÉSOLU 12 Scénario alternatif : solde insuffisant

Compléter le scénario de paiement MoMo : si le solde du compte est suffisant, la plateforme confirme et le système enregistre le paiement ; sinon, elle retourne un refus et le système affiche « Solde insuffisant ». Indiquer l'opérateur de fragment à utiliser et décrire les deux opérandes.

SOLUTION

Choix de l'opérateur : l'énoncé contient « si... sinon » : il s'agit d'une alternative exclusive entre deux comportements. On utilise le fragment combiné `alt` avec deux opérandes séparés par une ligne pointillée.

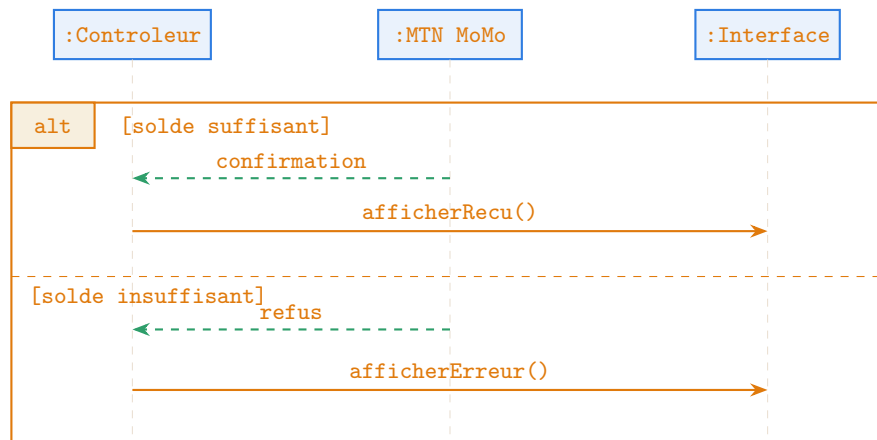
Opérande 1 — garde `[solde suffisant]` :

1. MTN → Contrôleur : `confirmation` (retour)
2. Contrôleur → BaseDeDonnees : `enregistrerPaielement(m)`
3. Contrôleur → Interface : `afficherRecu()`

Opérande 2 — garde `[solde insuffisant]` :

1. MTN → Contrôleur : `refus` (retour)
2. Contrôleur → Interface : `afficherErreur("Solde insuffisant")`

Schéma du fragment :



Fragment alt : deux opérandes gardés pour le paiement MoMo.

Conclusion : un fragment alt à deux opérandes gardés [solde suffisant] / [solde insuffisant] couvre exhaustivement les deux scénarios ; les gardes sont mutuellement exclusives, ce qui garantit un comportement déterministe.

9.6 Méthode 6 : Produire les diagrammes avec un logiciel professionnel

LESAVAIS-TU ? Des outils de modélisation utilisés par les professionnels

ArgoUML (libre, écrit en Java), StarUML (multiplateforme, très répandu en entreprise) et WinDesign (suite française de modélisation Merise et UML) sont cités par le programme officiel. Les startups camerounaises et les cabinets d'études (projets ANTIC, CENADI) utilisent ces outils — ou leurs équivalents en ligne comme PlantUML et Lucidchart — pour concevoir les systèmes d'information avant tout développement : modéliser d'abord, coder ensuite.

MÉTHODE Réaliser les diagrammes avec StarUML ou ArgoUML

1. Créer le projet : dans StarUML, File > New Project, choisir le modèle UML ; dans ArgoUML, Fichier > Nouveau.
2. Ajouter le bon type de diagramme : clic droit sur le modèle → Add Diagram > Use Case Diagram (ou Sequence Diagram). Ne jamais mélanger les deux dans le même diagramme.
3. Placer les éléments depuis la boîte à outils : Actor, UseCase, System Boundary (rec-

tangle du système) pour le diagramme de cas ; Lifeline, Message pour le diagramme de séquence.

4. Créer les relations avec les outils dédiés : Include, Extend, Generalization — le logiciel oriente automatiquement les flèches selon la norme UML ; vérifier le sens.
5. Nommer chaque élément dans le panneau de propriétés (un cas = verbe à l'infinitif ; un message = verbe + paramètres).
6. Exporter en image (File > Export Diagram as > PNG) pour l'insérer dans le rapport ou la copie d'examen pratique.
7. Réflexe WinDesign : même logique par modules (module UML) ; penser à sauvegarder régulièrement (Ctrl+S).

EXERCICE RÉSOLU 13 Production du diagramme de cas d'utilisation sous StarUML

À l'aide de StarUML, produire le diagramme de cas d'utilisation de la bibliothèque scolaire (exercice résolu précédent) : 2 acteurs, 5 cas, 1 inclusion. Décrire la démarche complète.

SOLUTION

Étape 1 — Projet. Lancer StarUML, File > New Project, sélectionner UML Standard. Dans l'explorateur de modèles (panneau droit), le Model racine apparaît.

Étape 2 — Diagramme. Clic droit sur Model → Add Diagram > Use Case Diagram. Renommer le diagramme « CU_Bibliotheque ».

Étape 3 — Frontière. Glisser l'outil System Boundary sur la zone de dessin, le redimensionner, taper « Système de gestion de bibliothèque » comme nom.

Étape 4 — Acteurs. Glisser deux Actor à gauche de la frontière ; les nommer « Élève » et « Bibliothécaire » dans le panneau de propriétés.

Étape 5 — Cas d'utilisation. Glisser cinq UseCase à l'intérieur de la frontière : « Consulter le catalogue », « Emprunter un ouvrage », « Vérifier la carte », « Enregistrer un retour », « Gérer les ouvrages ».

Étape 6 — Associations. Avec l'outil Association, relier Élève à « Consulter le catalogue » et « Emprunter un ouvrage » ; Bibliothécaire à « Enregistrer un retour » et « Gérer les ouvrages ».

Étape 7 — Inclusion. Avec l'outil Include, cliquer d'abord sur « Emprunter un ouvrage » (source) puis sur « Vérifier la carte » (cible) : StarUML trace la flèche pointillée stéréotypée <<include>> dans le bon sens.

Étape 8 — Export. File > Export Diagram as > PNG, nommer le fichier cu_bibliotheque.png.

Conclusion : le diagramme produit est conforme à la norme UML 2 ; l'ordre des clics (source puis cible) garantit le sens correct de la flèche d'inclusion, point classique d'erreur en épreuve pratique.

À RETENIR L'essentiel de la leçon

- Un **acteur** est externe au système ; le **rectangle du système** matérialise le périmètre ; un **cas d'utilisation** se nomme par un verbe à l'infinitif dans une ellipse.
- `<<include>>` = obligatoire (base → inclus) ; `<<extend>>` = optionnel (extension → base) ; généralisation = flèche pleine à triangle vide (enfant → parent).
- Le **diagramme de séquence** décrit un scénario dans le temps (de haut en bas) : participants, lignes de vie en pointillés, barres d'activation, messages synchrones (pointe pleine), asynchrones (pointe ouverte) et retours (pointillés).
- Les fragments combinés `alt`, `opt`, `loop` modélisent les scénarios complexes avec des gardes `[condition]`.
- ArgoUML, StarUML et WinDesign permettent de produire ces diagrammes conformément à la norme UML.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Inverser les flèches `<<include>>` et `<<extend>>` : l'inclusion va du cas de base vers le cas inclus ; l'extension va de l'extension vers le cas de base. C'est l'erreur la plus fréquente : vérifier le sens en se demandant « qui dépend de qui? ».
2. Confondre `include` et `extend` : `include` = comportement obligatoire (systématique) ; `extend` = comportement optionnel (sous condition). Utiliser `extend` pour une étape obligatoire (comme l'authentification) est une faute de modélisation.
3. Placer un acteur à l'intérieur du rectangle du système ou un cas d'utilisation à l'extérieur : le rectangle matérialise le périmètre ; les acteurs sont toujours externes par définition.
4. Nommer un cas d'utilisation par un nom commun (« Paiement », « Gestion ») au lieu d'un verbe à l'infinitif (« Payer les frais », « Gérer les ouvrages ») : un cas d'utilisation exprime une action, pas un objet.
5. Négliger la chronologie du diagramme de séquence : messages non ordonnés de haut en bas, retours tracés en flèches pleines au lieu de pointillés, gardes `[condition]` oubliées dans un fragment `alt`, ou scénario d'erreur (solde insuffisant, carte invalide) ignoré alors que l'énoncé l'exigeait.

10 EXERCICES

10.1 Je vérifie mes connaissances

EXERCICE 7 QCM : notations UML

Pour chaque question, une seule réponse est correcte. Recopie la lettre correspondante sur ta copie.

1. Dans un diagramme de cas d'utilisation, la relation `<<include>>` signifie :
 - a) le cas source peut éventuellement être complété par le cas cible ;
 - b) le cas source contient obligatoirement le cas cible à chaque exécution ;
 - c) le cas cible est une spécialisation du cas source ;
 - d) les deux cas sont exécutés en parallèle.
2. Dans un diagramme de séquence, un message synchrone est représenté par :
 - a) une flèche pleine à tête pleine ;
 - b) une flèche en pointillés à tête ouverte ;
 - c) une flèche pleine à tête ouverte ;
 - d) un rectangle sur la ligne de vie.
3. La flèche de la relation `<<extend>>` est dirigée :
 - a) du cas de base vers le cas d'extension ;
 - b) du cas d'extension vers le cas de base ;
 - c) dans les deux sens ;
 - d) de l'acteur vers le cas.
4. Un acteur d'un système d'information est :
 - a) toujours une personne physique ;
 - b) toujours interne à l'entreprise ;
 - c) toute entité externe qui interagit avec le système ;
 - d) un composant matériel du système.

EXERCICE 8 Vrai ou faux, avec justification

Indique si chacune des affirmations suivantes est vraie ou fausse, puis justifie ta réponse en une ou deux phrases.

1. Un acteur peut être un autre système informatique, par exemple l'API de MTN Mobile Money pour une application de vente en ligne.
2. Le périmètre d'un système est matérialisé dans le diagramme de cas d'utilisation par un rectangle qui contient les acteurs.
3. Dans un diagramme de séquence, la ligne de vie d'un objet est une ligne verticale en pointillés.
4. La relation de généralisation entre acteurs permet à un acteur fils d'hériter des cas d'utilisation de l'acteur père.
5. Le message de retour dans un diagramme de séquence est obligatoire et toujours synchrone.

EXERCICE 9 Définitions

Défins précisément, en deux ou trois phrases chacune, les termes suivants : **acteur**, **cas d'utilisation**, **scénario nominal**, **ligne de vie**, **périmètre du système**.

EXERCICE 10 Lecture directe d'un diagramme

On considère le diagramme de séquence simplifié d'un paiement par Orange Money : le Client envoie le message synchrone `initierPaiement(montant)` à l'objet `Application`, qui envoie à son tour `verifierSolde()` au `ServeurOM`; celui-ci retourne `soldeOK`, puis l'`Application` retourne `confirmation` au Client.

1. Combien ce diagramme contient-il de lignes de vie ?
2. Combien de messages synchrones et combien de messages de retour y figurent ?
3. Quel objet déclenche le scénario ?

10.2 Je m'entraîne

EXERCICE 11 Identifier acteurs et cas d'utilisation

Une startup doualaïse développe une application web de réservation de tickets de bus entre Douala et Yaoundé. Le voyageur consulte les horaires, réserve une place et paie en ligne via un service de paiement mobile externe. L'employé de l'agence peut annuler une

réservation et consulter la liste des passagers d'un départ.

1. Dresse la liste des acteurs du système, en distinguant acteurs principaux et acteurs secondaires.
2. Dresse la liste des cas d'utilisation, en veillant à ce que chaque intitulé commence par un verbe à l'infinitif.
3. Trace le diagramme de cas d'utilisation complet (périmètre, acteurs, associations).

EXERCICE 12 Chasse aux erreurs

Le diagramme de cas d'utilisation d'un système de gestion de cybercafé, reproduit ci-dessous en description, contient cinq erreurs volontaires :

- l'acteur `Caissier` est dessiné à l'intérieur du rectangle du système ;
 - le cas d'utilisation est intitulé `Facture` (un nom commun) ;
 - la flèche `<<extend>>` part du cas de base `Imprimer un document` vers le cas `Numériser en couleur` ;
 - le cas `Consulter le solde` est relié à `Se connecter` par une relation `<<include>>` dont la flèche part de `Se connecter` vers `Consulter le solde` ;
 - une flèche de généralisation relie deux cas d'utilisation avec une tête de flèche pleine (au lieu d'une tête triangulaire creuse).
1. Repère chaque erreur et explique pourquoi elle viole la notation UML.
 2. Propose la correction de chaque erreur, en justifiant.

EXERCICE 13 Diagramme de séquence à compléter

Voici le scénario nominal du cas `Emprunter un livre` dans la bibliothèque d'un lycée : l'Adhérent présente sa carte au Guichet ; le système vérifie que l'adhérent n'a pas de pénalité ; le système vérifie que l'ouvrage est disponible ; le système enregistre l'emprunt ; le système retourne un reçu à l'adhérent.

1. Identifie les objets/lignes de vie nécessaires au diagramme de séquence.
2. Pour chaque échange, précise s'il s'agit d'un message synchrone ou d'un message de retour.
3. Trace le diagramme de séquence complet : lignes de vie, activations, messages numérotés dans l'ordre chronologique.

EXERCICE 14 Relations <<include>> et <<extend>>

Dans le système de gestion des notes d'un lycée de Bafoussam, on considère les cas d'utilisation suivants : Saisir les notes, Se connecter, Consulter le bulletin, Imprimer le bulletin, Corriger une note.

1. Sachant que toute saisie et toute consultation exigent obligatoirement une authentification, place les relations <<include>> adéquates (sens des flèches exigé).
2. L'impression du bulletin est une possibilité offerte lors de la consultation, mais elle n'est pas systématique : place la relation adéquate et justifie le choix entre <<include>> et <<extend>>.
3. Trace le diagramme partiel correspondant.

10.3 Je me prépare au Bac

EXERCICE 15 Sujet type Bac : gestion d'une pharmacie

La pharmacie Bon Santé de Yaoundé souhaite informatiser sa gestion. Le dossier d'analyse fournit les informations suivantes :

- le Pharmacien et le Préparateur peuvent tous deux Vendre un médicament ; le Pharmacien peut en outre Gérer le stock (commandes, inventaires) ;
- toute vente exige obligatoirement de Vérifier la disponibilité du produit et de Enregistrer la vente en caisse ;
- lors de la vente, si le médicament est soumis à prescription, le vendeur peut être amené à Contrôler l'ordonnance : ce contrôle n'a lieu que dans ce cas particulier ;
- la gestion du stock comprend systématiquement l'opération Mettre à jour les quantités.

Travail à faire :

1. Identifie les acteurs du système et précise la relation de généralisation/spécialisation qui les lie. Justifie. (2 pts)
2. Dresse la liste des cas d'utilisation en respectant la règle de nommage. (2 pts)
3. Construis le diagramme de cas d'utilisation complet : périmètre, acteurs, associations, généralisation entre acteurs, au moins deux relations <<include>> et une relation <<extend>> correctement orientées. (6 pts)
4. Pour le cas Vendre un médicament, rédige le scénario nominal (vente d'un produit disponible, sans ordonnance) sous forme d'étapes numérotées. (3 pts)
5. Trace le diagramme de séquence correspondant à ce scénario nominal, avec les objets Vendeur, InterfaceVente, Stock et Caisse, en distinguant messages synchrones et

messages de retour. (5 pts)

6. Propose un scénario alternatif (produit en rupture de stock) et indique comment il modifierait le diagramme de séquence. (2 pts)

EXERCICE 16 Sujet type Bac : retrait au guichet automatique

Une banque camerounaise veut modéliser le fonctionnement de ses guichets automatiques de billets (GAB). Le client insère sa carte, saisit son code confidentiel, choisit un montant ; le système interroge le serveur central de la banque pour vérifier le code et le solde, puis délivre les billets et édite un ticket.

1. Identifie les acteurs du système, y compris l'acteur secondaire. (2 pts)
2. Rédige le scénario nominal du cas `Retirer de l'argent` sous forme d'étapes numérotées alternant actions du client et réponses du système. (4 pts)
3. Rédige deux scénarios alternatifs : (a) code confidentiel erroné trois fois de suite ; (b) solde insuffisant. Précise pour chacun le point du scénario nominal où il se déclenche. (4 pts)
4. Trace le diagramme de séquence du scénario nominal avec les lignes de vie `Client`, `GAB` et `ServeurBanque` : messages synchrones, messages de retour, barres d'activation. (6 pts)
5. Le cas `Consulter le solde` nécessite, comme le retrait, une authentification : propose une factorisation à l'aide d'une relation UML, nomme-la, oriente la flèche et justifie. (4 pts)

EXERCICE 17 Travail pratique noté : notes en ligne sous StarUML ou ArgoUML

Le proviseur d'un lycée de Buea veut mettre en place une plateforme de notes en ligne. Les règles de gestion sont les suivantes :

- l'Enseignant saisit les notes de ses évaluations après s'être connecté ;
- l'Élève consulte ses notes après s'être connecté ; il peut, s'il le souhaite, télécharger son bulletin en PDF ;
- le Proviseur valide les bulletins avant leur publication ; la validation déclenche systématiquement l'envoi d'une notification SMS aux parents via une passerelle SMS externe.

Travail à faire (sur machine, compte rendu à rendre sous forme de captures et d'un fichier de projet) :

1. Lance StarUML (ou ArgoUML) et crée un nouveau projet ; crée un Use Case Diagram nommé `PlateformeNotes`. (2 pts)

2. Réalise le diagramme de cas d'utilisation complet : acteurs (y compris l'acteur secondaire `PasserelleSMS`), périmètre, une généralisation d'acteurs si tu la juges pertinente (justifie-la), au moins deux relations `<<include>>` et une relation `<<extend>>`. (8 pts)
3. Crée un Sequence Diagram pour le scénario nominal Valider les bulletins : lignes de vie `Provisueur`, `Plateforme`, `BaseDeDonnees`, `PasserelleSMS`; messages synchrones, messages de retour et activations correctement placés. (8 pts)
4. Exporte les deux diagrammes en image, puis rédige en cinq lignes maximum un commentaire expliquant la différence de point de vue entre les deux diagrammes produits. (2 pts)

11 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : notations UML

Question 1 : réponse b. La relation `<<include>>` exprime une inclusion obligatoire : chaque exécution du cas source comporte l'exécution du cas cible. La réponse a) décrit `<<extend>>` (comportement optionnel), c) décrit la généralisation, d) n'a aucun sens en UML.

Question 2 : réponse a. Un message synchrone (l'émetteur attend la fin du traitement) se note par une flèche au trait plein et à la tête pleine (triangle noir). La flèche en pointillés à tête ouverte (b) est un message de retour ; la flèche pleine à tête ouverte (c) est un message asynchrone ; le rectangle (d) est une barre d'activation.

Question 3 : réponse b. Dans `<<extend>>`, la flèche en pointillés part du cas d'extension (optionnel) et pointe vers le cas de base : c'est l'extension qui « connaît » le cas qu'elle peut compléter. C'est le sens inverse de `<<include>>`, piège classique à l'examen.

Question 4 : réponse c. Un acteur est toute entité externe au système qui interagit avec lui : personne physique (client, employé) ou système informatique tiers (API de paiement, serveur central). Il n'est donc ni nécessairement humain (a), ni nécessairement interne (b), et il n'est jamais un composant du système (d) : par définition, il est à l'extérieur du périmètre.

CORRIGÉ Exercice 2 — Vrai ou faux, avec justification

1. Vrai. Un acteur est toute entité externe interagissant avec le système ; un service tiers comme l'API MTN Mobile Money, sollicitée par l'application pour encaisser un paiement, est un acteur secondaire (il répond aux sollicitations du système).
2. Faux. Le rectangle du système contient les cas d'utilisation (les ellipses) ; les acteurs sont placés à l'extérieur du rectangle, précisément parce qu'ils sont externes au système.
3. Vrai. La ligne de vie est une ligne verticale en pointillés partant du symbole de l'objet (rectangle en haut) ; le temps s'écoule du haut vers le bas. Les barres d'activation sont de fins rectangles posés sur cette ligne.
4. Vrai. La généralisation entre acteurs (flèche à tête triangulaire creuse, du fils vers le père) signifie que l'acteur fils hérite de toutes les associations du père : il peut réaliser tous les cas d'utilisation du père, plus éventuellement les siens.
5. Faux. Le message de retour (flèche en pointillés à tête ouverte) est optionnel : on l'omet quand il n'apporte pas d'information. De plus, « synchrone » qualifie l'envoi (tête pleine), pas le retour ; un retour est par nature une réponse, on ne dit pas qu'il est synchrone.

ATTENTION Points de vigilance — Exercices 1 et 2

Retenir absolument : `<<include>>` = obligatoire (flèche base → inclus); `<<extend>>` = optionnel (flèche extension → base). Acteurs jamais dans le rectangle. Message synchrone = trait plein + triangle plein; retour = pointillés + tête ouverte; asynchrone = trait plein + tête ouverte. Généralisation = trait plein + triangle creux (vide) vers le parent.

CORRIGÉ Exercice 3 — Définitions

Acteur. Entité externe au système (personne, organisation ou autre système informatique) qui interagit directement avec lui en émettant ou en recevant des informations. On distingue les acteurs principaux (qui déclenchent les cas) des acteurs secondaires (sollicités par le système).

Cas d'utilisation. Ensemble cohérent d'actions réalisées par le système pour produire un résultat observable et utile à un acteur. Il se représente par une ellipse et se nomme par un verbe à l'infinitif suivi d'un complément (*Retirer de l'argent*).

Scénario nominal. Déroulement « standard » d'un cas d'utilisation, lorsque tout se passe bien, sans erreur ni cas particulier : c'est la séquence d'étapes la plus fréquente, dont dérivent les scénarios alternatifs (cas particuliers) et les scénarios d'exception (erreurs).

Ligne de vie. Dans un diagramme de séquence, ligne verticale en pointillés associée à un objet (ou acteur) participant à l'interaction; elle matérialise son existence dans le temps, qui s'écoule de haut en bas, et porte les barres d'activation.

Périmètre du système. Frontière qui délimite ce qui relève du système à développer (les cas d'utilisation, placés dans le rectangle) de ce qui lui est extérieur (les acteurs). Le définir est la première étape de la modélisation : il fixe les responsabilités du futur logiciel.

CORRIGÉ Exercice 4 — Lecture directe d'un diagramme

1. Le diagramme contient 3 lignes de vie : *Client*, *Application* et *ServeurOM*.
2. On compte 2 messages synchrones : `initierPaieement(montant)` (*Client* → *Application*) et `verifierSolde()` (*Application* → *ServeurOM*); et 2 messages de retour : `soldeOK` (*ServeurOM* → *Application*) et `confirmation` (*Application* → *Client*).
3. C'est le *Client* qui déclenche le scénario : il émet le premier message. C'est l'acteur principal de l'interaction.

ATTENTION Point de vigilance — Exercice 4

Ne pas confondre le nombre de messages avec le nombre de lignes de vie : ici 4 flèches mais seulement 3 participants. À l'examen, compte d'abord les objets (rectangles en haut), puis classe chaque flèche selon sa notation (pleine/tête pleine = synchrone ; pointillés/tête ouverte = retour).

CORRIGÉ Exercice 5 — Identifier acteurs et cas d'utilisation

1. Acteurs.

- **Acteurs principaux** : le *Voyageur* (il déclenche la consultation, la réservation et le paiement) et l'*EmployéAgence* (il déclenche l'annulation et la consultation des passagers).
- **Acteur secondaire** : le *ServicePaiementMobile* (système externe sollicité pour encaisser le paiement ; il n'initie rien).

2. Cas d'utilisation (verbe à l'infinitif + complément) :

- **côté Voyageur** : Consulter les horaires, Réserver une place, Payer en ligne ;
- **côté EmployéAgence** : Annuler une réservation, Consulter la liste des passagers.

Le cas *Payer en ligne* est associé au *ServicePaiementMobile*.

3. Diagramme de cas d'utilisation. On peut en outre relier *Réserver une place* à *Payer en ligne* par `<<include>>` si le cahier des charges impose le paiement à la réservation ; ce n'était pas exigé ici.

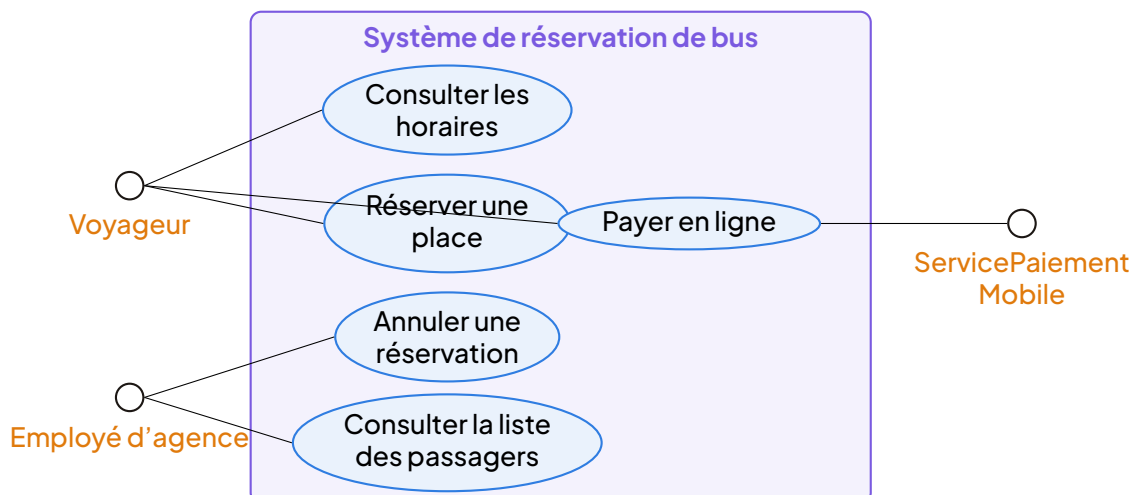


Diagramme de cas d'utilisation du système de réservation de bus.

CORRIGÉ Exercice 6 — Chasse aux erreurs

1. Acteur à l'intérieur du système. Par définition, un acteur est externe au système. Correction : placer le bonhomme `Caissier` à l'extérieur du rectangle, relié à ses cas par des associations.
2. Cas nommé par un nom commun (`Facture`). Un cas d'utilisation exprime un service rendu : il doit commencer par un verbe à l'infinitif. Correction : renommer en `Éditer une facture`.
3. Sens de la flèche `<<extend>>`. La flèche d'extension part du cas d'extension vers le cas de base. Correction : inverser la flèche : `Numériser en couleur` --> `Imprimer un document`, car l'option couleur complète éventuellement l'impression.
4. Sens de la flèche `<<include>>`. L'inclusion part du cas qui exige l'autre vers le cas inclus. Correction : la flèche doit partir de `Consulter le solde` vers `Se connecter` : consulter le solde exige obligatoirement d'être connecté, et non l'inverse.
5. Tête de flèche de généralisation. La généralisation se note par un trait plein terminé par un triangle creux (vide) pointant vers l'élément parent ; une tête pleine correspond à un message synchrone (diagramme de séquence) ou à une composition (diagramme de classes). Correction : remplacer la tête pleine par un triangle creux orienté vers le cas parent.

ATTENTION Pièges à retenir — Exercice 6

Les deux confusions les plus sanctionnées au Bac : inverser les sens de `<<include>>` (base → inclus) et `<<extend>>` (extension → base), et dessiner les acteurs dans le rectangle du système.

CORRIGÉ Exercice 7 — Diagramme de séquence à compléter

1. Lignes de vie. Trois participants suffisent : l'acteur `Adhérent`, l'objet frontière `Guichet` (interface de saisie) et l'objet `SystèmeBiblio` (qui concentre les vérifications et l'enregistrement). On peut raffiner en séparant `FichierAdhérents` et `Catalogue`, mais ce n'est pas exigé.
2. Nature des messages.

N°	Message	Type
1	présenterCarte() : Adhérent → Guichet	synchrone
2	vérifierPénalité() : Guichet → SystèmeBiblio	synchrone
3	pasDePénalité : SystèmeBiblio → Guichet	retour
4	vérifierDisponibilité(ouvrage) : Guichet → SystèmeBiblio	synchrone
5	disponible : SystèmeBiblio → Guichet	retour
6	enregistrerEmprunt() : Guichet → SystèmeBiblio	synchrone
7	reçu : SystèmeBiblio → Guichet puis Guichet → Adhérent	retour

3. Diagramme de séquence. Les messages sont numérotés de haut en bas, dans l'ordre chronologique ; les barres d'activation du Guichet et du SystèmeBiblio couvrent la durée de leur participation.

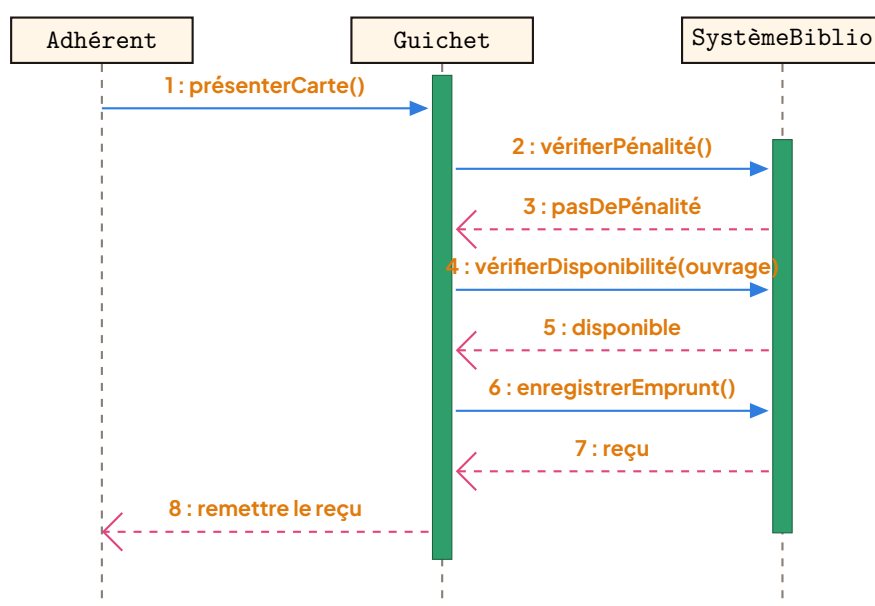


Diagramme de séquence du scénario nominal « Emprunter un livre ».

CORRIGÉ Exercice 8 — Relations <<include>> et <<extend>>

1. Relations <<include>>. L'authentification est obligatoire pour la saisie et la consultation : on trace deux flèches en pointillés étiquetées <<include>>, chacune partant du cas exigeant vers le cas inclus :

- Saisir les notes --> Se connecter ;

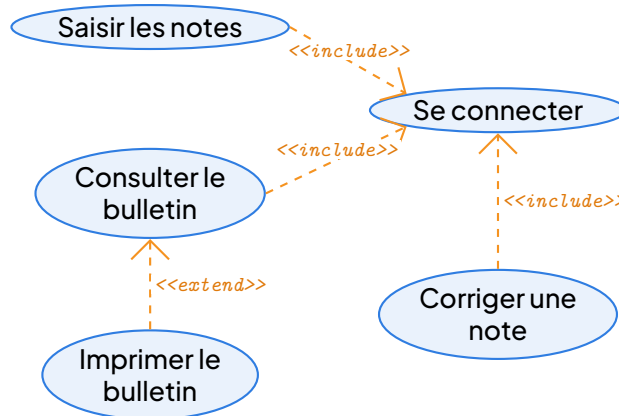
- Consulter le bulletin --> Se connecter.

La flèche part toujours du cas de base vers le cas inclus.

2. Impression du bulletin. L'impression est optionnelle (elle peut avoir lieu lors de la consultation, mais pas systématiquement) : c'est donc une relation `<<extend>>`, et non `<<include>>` qui imposerait l'impression à chaque consultation. La flèche part du cas d'extension vers le cas de base : Imprimer le bulletin --> Consulter le bulletin.

3. Correction d'une note. Corriger une note exige systématiquement une authentification : on ajoute Corriger une note --> Se connecter (`<<include>>`).

4. Diagramme partiel. Justification du choix : `<<include>>` factorise un comportement systématique partagé par plusieurs cas ; `<<extend>>` décrit un comportement conditionnel qui enrichit un cas de base sans le modifier. Ici, la connexion est systématique (include), l'impression est conditionnelle (extend).



Relations d'inclusion et d'extension (correction : Corriger une note inclut directement Se connecter).

CORRIGÉ Exercice 9 — Sujet type Bac : gestion d'une pharmacie

1. Acteurs et généralisation. (2 pts) Acteurs : Pharmacien et Préparateur. Les deux peuvent Vendre un médicament : on factorise ce comportement commun en introduisant un acteur général Vendeur (ou Personnel), père de Pharmacien et Préparateur. Le Pharmacien, acteur fils, hérite du cas Vendre un médicament et possède en propre Gérer le stock. La flèche de généralisation (trait plein, triangle creux) va de chaque fils vers le père.

2. Cas d'utilisation. (2 pts) Vendre un médicament, Vérifier la disponibilité du produit, Enregistrer la vente en caisse, Contrôler l'ordonnance, Gérer le stock, Mettre à jour les quantités. Chaque intitulé commence par un verbe à l'infinitif.

3. Diagramme de cas d'utilisation. (6 pts) Orientation exigée : `<<include>>` de Vendre un médicament vers Vérifier la disponibilité et vers Enregistrer la vente en caisse (obligatoires) ; `<<extend>>` de Contrôler l'ordonnance vers Vendre un médicament

(contrôle seulement si le produit est soumis à prescription); <<include>> de Gérer le stock**vers**Mettre à jour les quantités(**systématique**).

4. Scénario nominal de Vendre un médicament. (3 pts)

1. Le vendeur saisit l'identifiant du produit demandé.
2. Le système vérifie la disponibilité du produit : il est en stock.
3. Le vendeur confirme la quantité vendue.
4. Le système enregistre la vente en caisse et calcule le montant.
5. Le client règle ; le système édite le reçu et met à jour le stock.

5. Diagramme de séquence. (5 pts)

6. Scénario alternatif : rupture de stock. (2 pts) À l'étape 2 du scénario nominal, le produit est indisponible : le Stock retourne indisponible au lieu de disponible ; l'InterfaceVente affiche un message d'erreur au Vendeur et le scénario se termine (ou propose un produit équivalent). Sur le diagramme de séquence, on ajoute un fragment combiné alt : une branche [disponible] contenant les messages 4 à 8, une branche [indisponible] contenant le message d'erreur.

Barème indicatif et points de vigilance. Q1 : 1 pt pour les deux acteurs, 1 pt pour la justification de la généralisation (comportement commun factorisé). Q3 : 1 pt périmètre, 1 pt acteurs/associations, 1 pt généralisation correctement orientée, 2 pts pour les <<include>>, 1 pt pour le <<extend>> dans le bon sens. Q5 : 1 pt lignes de vie, 2 pts messages synchrones corrects, 1 pt retours en pointillés, 1 pt activations. Vigilance : citer la condition d'application de chaque relation (« obligatoire » ⇒ include ; « seulement si soumis à prescription » ⇒ extend) ; les retours se dessinent en pointillés à tête ouverte.

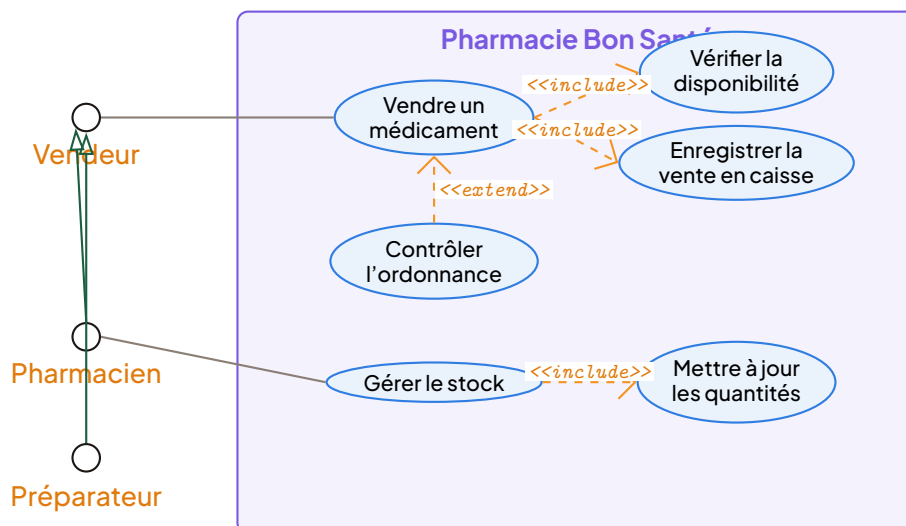


Diagramme de cas d'utilisation de la pharmacie (généralisation d'acteurs, inclusions, extension).

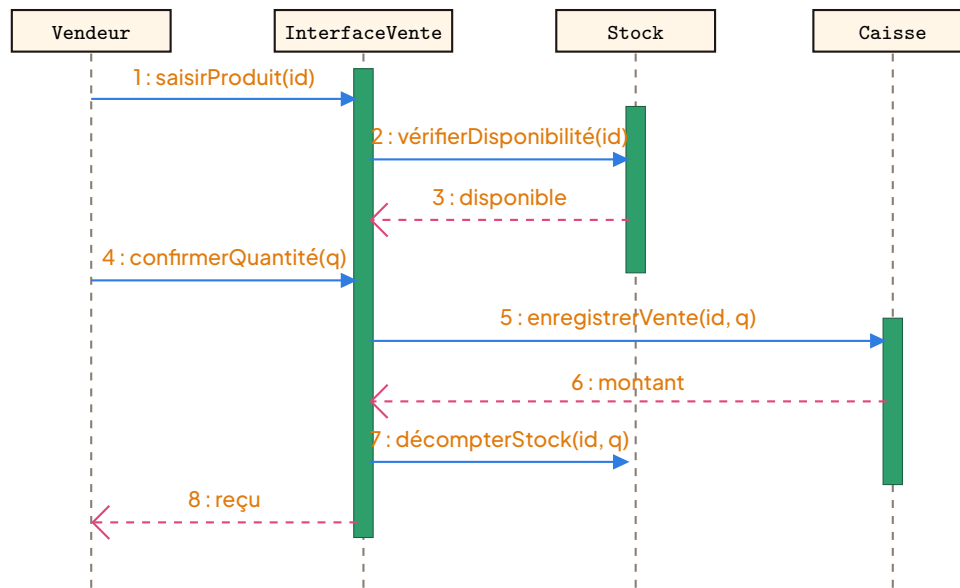


Diagramme de séquence du scénario nominal « Vendre un médicament ».

CORRIGÉ Exercice 10 — Sujet type Bac : retrait au guichet automatique

1. Acteurs. (2 pts) Acteur principal : le Client (il déclenche le retrait). Acteur secondaire : le ServeurCentral de la banque (système externe au GAB, sollicité pour vérifier le code et le solde). Le GAB lui-même n'est pas un acteur : c'est le système étudié.

2. Scénario nominal de Retirer de l'argent. (4 pts)

1. Le client insère sa carte dans le GAB.
2. Le système lit la carte et invite à saisir le code confidentiel.
3. Le client saisit son code.
4. Le système interroge le serveur central : le code est valide.
5. Le client choisit l'opération « retrait » puis saisit le montant.
6. Le système demande au serveur central de vérifier le solde : le solde est suffisant.
7. Le serveur central débite le compte et autorise le retrait.
8. Le système délivre les billets, édite le ticket et restitue la carte.

3. Scénarios alternatifs. (4 pts)

- (a) Code erroné : se déclenche à l'étape 4. Le serveur répond « code invalide » ; le système propose une nouvelle saisie. Après trois échecs consécutifs, le système confisque la carte et le scénario se termine (scénario d'exception).
- (b) Solde insuffisant : se déclenche à l'étape 6. Le serveur refuse le débit ; le système affiche « solde insuffisant », propose un montant inférieur ou annule ; la carte est res-

tituée sans délivrance de billets.

4. Diagramme de séquence du scénario nominal. (6 pts)

5. Factorisation de l'authentification. (4 pts) Le retrait et la consultation du solde exigent tous deux systématiquement une authentification : on factorise avec une relation <<include>>. On crée le cas S'authentifier, et on trace deux flèches en pointillés : Retirer de l'argent-->S'authentifier et Consulter le solde-->S'authentifier. Justification : <<include>> s'emploie précisément pour un comportement obligatoire partagé par plusieurs cas ; <<extend>> serait faux car l'authentification n'est pas optionnelle.

Barème indicatif et points de vigilance. Q1 : 1 pt par acteur avec son rôle (principal/secondaire). Q2 : 1 pt par tranche de deux étapes, alternance client/système exigée. Q3 : 2 pts par scénario (1 pt déclenchement localisé, 1 pt déroulement). Q4 : 1 pt lignes de vie, 2 pts messages synchrones, 1 pt retours en pointillés, 1 pt activations, 1 pt chronologie. Q5 : 1 pt nom de la relation, 1 pt sens des flèches, 2 pts justification (obligation + mutualisation). Vigilance : le serveur central est un acteur secondaire du diagramme de cas, mais devient une ligne de vie dans le diagramme de séquence ; ne pas dessiner le GAB comme acteur.

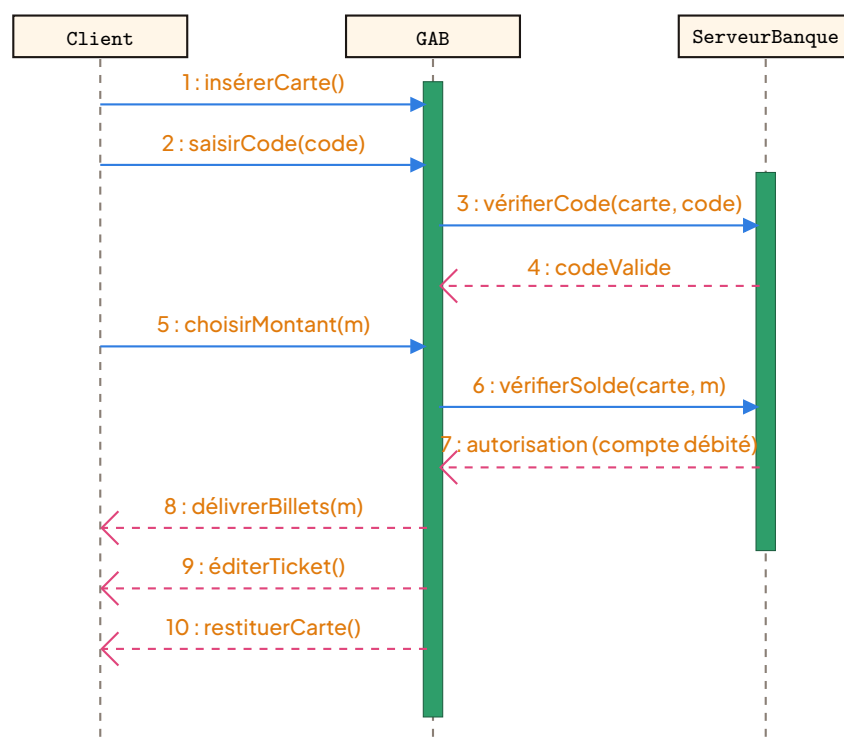


Diagramme de séquence du retrait au GAB (scénario nominal).

CORRIGÉ Exercice 11 — Travail pratique noté : notes en ligne sous StarUML ou ArgoUML

1. Création du projet. (2 pts) Sous StarUML : File > New Project, puis dans l'explorateur de modèles, clic droit sur le modèle Add Diagram > Use Case Diagram, renommé PlateformeNotes. Sous ArgoUML : File > New, puis Create Diagram > Use Case Diagram. Le fichier est enregistré (.mdj pour StarUML, .zargo pour ArgoUML).

2. Diagramme de cas d'utilisation attendu. (8 pts)

- **Acteurs principaux** : Enseignant, Élève, Proviseur ; **acteur secondaire** : PasserelleSMS (système externe).
- **Cas** : Saisir les notes, Consulter ses notes, Télécharger le bulletin en PDF, Valider les bulletins, Se connecter, Envoyer une notification SMS.
- **Relations <<include>> (obligatoires)** : Saisir les notes → Se connecter ; Consulter ses notes → Se connecter ; Valider les bulletins → Envoyer une notification SMS (envoi systématique).
- **Relation <<extend>> (optionnelle)** : Télécharger le bulletin en PDF → Consulter ses notes (« s'il le souhaite »), flèche de l'extension vers le cas de base.
- **Association** : Envoyer une notification SMS reliée à l'acteur PasserelleSMS.
- **Généralisation d'acteurs (valorisée)** : un acteur Utilisateur père de Enseignant, Élève et Proviseur, associé à Se connecter — justifiée par le comportement commun d'authentification.

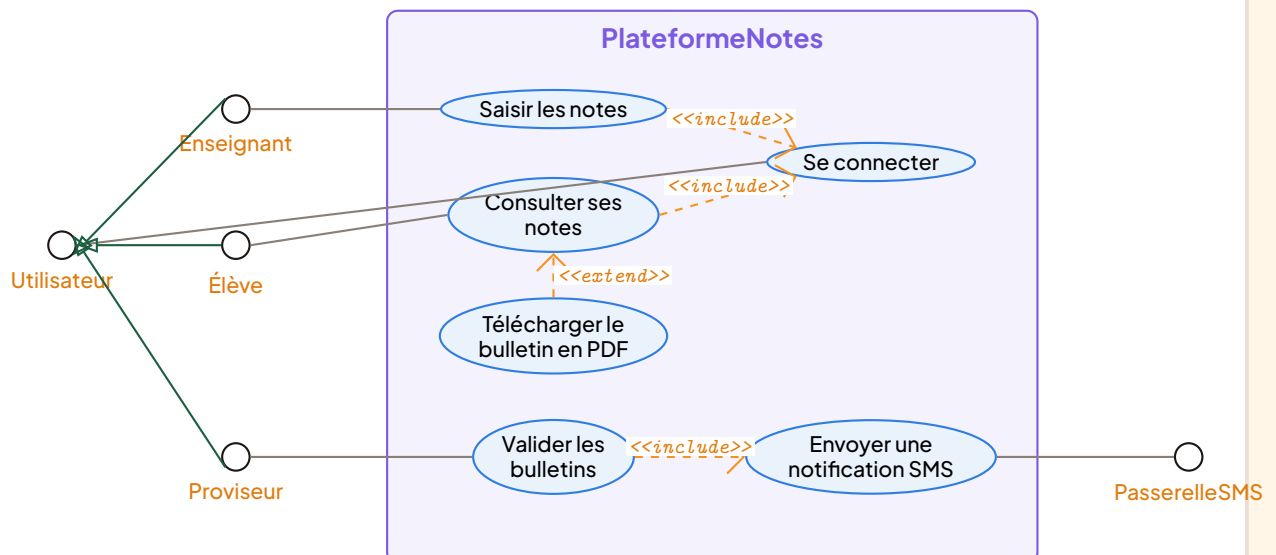


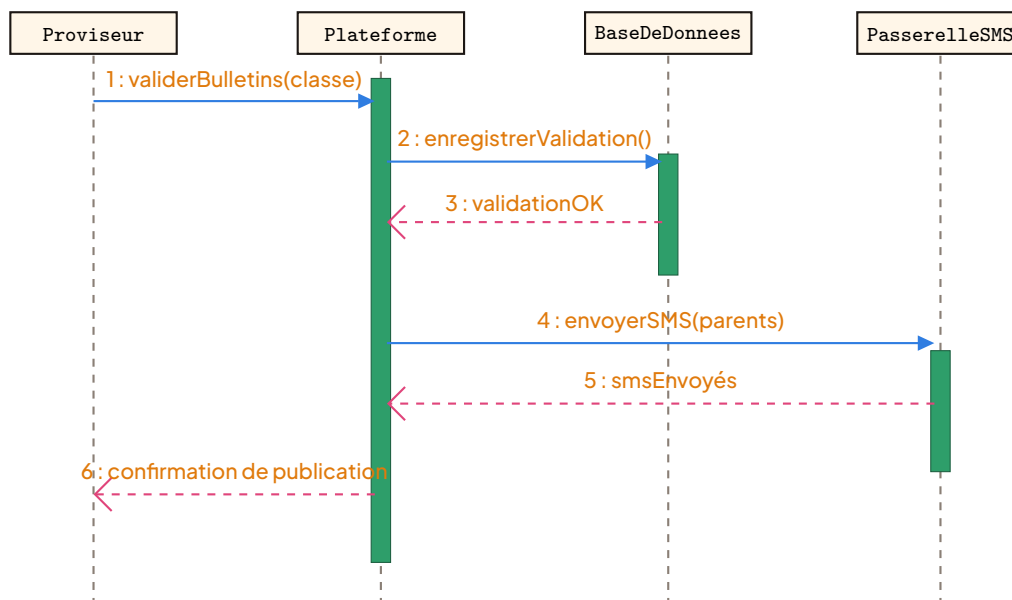
Diagramme de cas d'utilisation attendu pour la plateforme de notes (avec généralisation d'acteurs).

CORRIGÉ Exercice 11 — Suite : Diagramme de séquence Valider les bulletins. (8 pts)

Scénario nominal : le proviseur demande la validation ; la plateforme enregistre la validation en base ; la base confirme ; la plateforme demande à la passerelle l'envoi des SMS ; la passerelle confirme ; la plateforme notifie le proviseur.

4. Export et commentaire. (2 pts) Export : File > Export Diagram As > PNG (StarUML) ou File > Export Graphics (ArgoUML). Commentaire attendu (exemple) : « Le diagramme de cas d'utilisation offre une vue statique et fonctionnelle : il recense qui utilise le système et quels services il rend, sans dire quand ni comment. Le diagramme de séquence offre une vue dynamique et temporelle : il décrit, pour un scénario précis, l'ordre chronologique des messages échangés entre les objets. Les deux sont complémentaires : le premier définit le périmètre, le second détaille le comportement. »

Barème indicatif et points de vigilance. Q2 : 2 pts acteurs (dont le secondaire), 2 pts cas bien nommés, 2 pts inclusions correctement orientées, 1 pt extension, 1 pt association avec la passerelle, 1 pt généralisation acteurs. Q3 : 2 pts lignes de vie, 2 pts messages synchrones, 2 pts retours en pointillés, 2 pts activations et chronologie. Vigilance : la passerelle SMS est un acteur dans le diagramme de cas mais une ligne de vie dans le diagramme de séquence ; l'envoi du SMS est inclus (systématique) dans la validation, tandis que le téléchargement PDF étend (optionnel) la consultation ; vérifier que chaque flèche en pointillés porte bien son stéréotype <<include>> ou <<extend>>.



12 FICHE BILAN



Carte mentale de la leçon : la modélisation dynamique avec UML.

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- **Acteur** : entité externe (personne ou système) qui interagit avec le système ; il joue un rôle, représenté par un bonhomme (stickman).
- **Périmètre** (frontière) : rectangle qui délimite ce que le système fait (cas d'utilisation à l'intérieur) de ce qui est extérieur (acteurs).
- **Cas d'utilisation** : ensemble d'actions réalisées par le système pour fournir un résultat observable à un acteur ; ellipse contenant un verbe à l'infinitif (ex. : Consulter son

solde).

- **Scénario** : déroulement concret d'un cas ; on distingue le scénario nominal, les scénarios alternatifs et les scénarios d'erreur.
- **Diagramme de séquence** : représentation temporelle (de haut en bas) des échanges de messages entre objets pour réaliser un scénario.

Relations entre cas d'utilisation

Relation	Sens	Exemple
<<include>>	Le cas de base contient toujours le cas inclus (obligatoire) ; flèche pointillée vers le cas inclus.	Effectuer un retrait → S'authentifier
<<extend>>	Le cas d'extension s'exécute parfois (optionnel) ; flèche pointillée de l'extension vers le cas de base.	Imprimer le reçu ---> Effectuer un retrait
Généralisation	Le cas fils spécialise le cas père ; flèche pleine à triangle blanc vers le père.	Payer par MoMo → Payer

Composants et messages du diagramme de séquence

- **Objet/acteur** : rectangle en haut ; **ligne de vie** : pointillés verticaux ; **activation** : rectangle fin sur la ligne de vie (période d'activité).
- Message **synchrone** : flèche pleine, triangle noir (l'émetteur attend la réponse) ; **asynchrone** : flèche à pointe ouverte ; **retour** : flèche pointillée ; **réflexif** : message d'un objet vers lui-même.
- Fragments combinés : alt (alternative), opt (option), loop (boucle).

Méthodes express

1. Cas d'utilisation : lister les acteurs → lister les services attendus (verbes) → placer les cas dans la frontière → relier acteurs/cas → factoriser avec include, extend, généralisation.
2. Séquence : choisir un scénario → placer acteur et objets en haut → tracer les messages chronologiquement de haut en bas → ajouter retours et fragments alt/loop.
3. Logiciel : créer un projet dans ArgoUML/StarUML → choisir le type de diagramme → glisser acteurs, cas, objets → nommer chaque élément → exporter en image.

À RETENIR Checklist avant le Bac

- ☐ Je sais identifier les **acteurs** (externes au système) et tracer la **frontière** du système.

- ☐ Je nomme chaque cas d'utilisation par un verbe à l'infinitif complété.
- ☐ Je distingue scénario **nominal**, **alternatif** et **d'erreur**.
- ☐ Je place correctement le sens des flèches <<include>> (vers le cas inclus) et <<extend>> (vers le cas de base).
- ☐ Je sais dessiner une relation de **généralisation** (triangle blanc vers le cas parent).
- ☐ Je connais les composants d'un diagramme de séquence : objet, **ligne de vie**, **activation**.
- ☐ Je différencie message **synchrone**, **asynchrone**, **retour** et **réflexif** par leur flèche.
- ☐ Je sais utiliser les fragments alt, opt et loop pour les scénarios complexes.
- ☐ Je lis le temps de haut en bas dans un diagramme de séquence.
- ☐ Je sais produire ces diagrammes avec **ArgoUML**, **StarUML** ou **WinDesign**.
- ☐ Je vérifie toujours la cohérence : chaque cas d'utilisation est relié à au moins un acteur.
- ☐ Je relis mon schéma : pas de cas isolé, pas de flèche sans stéréotype, légendes présentes.

**Signé • OnBuch+**Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC**OnBuch+**